

INDIRA GANDHI NATIONAL OPEN UNIVERSITY

MCSP - 232

**AI-POWERED LEGAL DOCUMENT ANALYZER
AND RISK ASSESSMENT SYSTEM**

by

Nitish Kumar

Enrolment No: 2451971556

Under Guidance of

Rajat Kumar Moharana

Submitted to the School of Computer and Information Sciences, IGNOU in partial fulfilment
of the requirements for the award of the degree

Master of Computer Applications (MCA)

2026

Indira Gandhi National Open University
Maidan Garhi
New Delhi – 110068

APPROVED PROFORMA

*(Insert the original, signed Proforma for Approval of the MCA Project Proposal — MCSP-232
— here)*

PROJECT PROPOSAL / SYNOPSIS

(Insert the approved Project Proposal / Synopsis here)

BIO-DATA OF THE PROJECT GUIDE

(Insert the Project Guide's signed Bio-data here)

CERTIFICATE OF ORIGINALITY

This is to certify that the project report titled "AI-Powered Legal Document Analyzer and Risk Assessment System" submitted to Indira Gandhi National Open University in partial fulfilment of the requirement for the award of the degree of MASTER OF COMPUTER APPLICATIONS, is an authentic and original work carried out by Mr. Nitish Kumar with Enrolment No. 2451971556 under my guidance.

The matter embodied in this project is genuine work done by the student and has not been submitted whether to this University or to any other University / Institute for the fulfilment of the requirements of any course of study.

.....

(Signature of the Student)

Date:

Name and Address of the Student

Nitish Kumar
Sec-57, Khora Colony, Noida

Enrolment No: 2451971556

.....

(Signature of the Guide)

Date:

Name, Designation & Address of the Guide

Rajat Kumar Moharana
Software Development Engineer-I
Sec-62, Noida

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my project guide, Mr. Rajat Kumar Moharana, for his continuous guidance, technical direction, and encouragement throughout the development of this project. His feedback at every stage — from the initial synopsis through to the final deployed system — shaped both the scope and the quality of the work.

I am also grateful to the School of Computer and Information Sciences, IGNOU, and to my Study Centre, for the structure and support provided through the MCA programme, which gave me the foundation to undertake a project of this scale.

Finally, I would like to thank my family and friends for their patience and support during the course of this work.

Nitish Kumar
Enrolment No: 2451971556

TABLE OF CONTENTS

1. Introduction and Objectives	9
1.1 Introduction.....	9
1.2 Objectives.....	9
2. System Analysis.....	12
2.1 Identification of Need	12
2.2 Project Planning and Scheduling	14
2.3 Software Requirement Specifications (SRS)	19
2.4 Software Engineering Paradigm Applied.....	24
2.5 Data Models.....	25
3. System Design.....	49
3.1 System Architecture.....	49
3.2 Module Decomposition	51
3.3 Database Design	56
3.4 Data Integrity and Constraints.....	56
3.5 User Interface Design	58
3.6 Procedural Design.....	61
3.7 Component Diagram	64
3.8 Test Case Design	66
4. Coding.....	67
4.1 Coding Standards and Practices.....	67
4.2 Representative Code Extracts.....	67
4.3 Database Creation, Data Insertion, and Access Rights	68
4.4 Standardization of the Coding.....	70
5. Testing.....	71
5.1 Testing Strategy	71
5.2 Testing Plan	71
5.3 Unit Testing Coverage	72
5.4 System / Integration Testing Coverage	72
5.5 Representative Test Cases.....	72
5.6 Defects Identified During Testing.....	73
5.7 Debugging and Code Improvement	74
5.8 Worked Test Case Walkthrough.....	74
5.9 Test Coverage by Module.....	76
6. System Security Measures.....	78
6.1 Authentication and Session Security.....	78

6.2 Authorisation and Access Control.....	78
6.3 Input Validation and Transport Security.....	78
6.4 Data and Infrastructure Security	78
6.5 OWASP Top 10 Mapping	79
6.6 Security Verification Checklist.....	80
7. Reports	82
7.1 Document Risk Analysis Report.....	82
7.2 Legal Assistant Grounded Answer.....	83
7.3 Consultation Summary Report.....	84
8. Future Scope and Further Enhancement	86
8.1 Lessons Learned.....	87
8.2 Conclusion	88
8.3 Maintenance and Support Plan	88
9. Bibliography	90
9.1 Primary Legal Sources	90
9.2 Technical References	90
10. Appendices.....	92
Appendix A — Complete Prisma Schema	92
Appendix B — Environment Configuration Reference	101
Appendix C — API Endpoint Reference	102
Appendix D — Deployment and Installation Guide.....	107
Appendix E — User Manual.....	109
Appendix F — Error Handling and Status Code Reference	110
Appendix G — Sample API Requests	112
Appendix H — List of Figures and Tables	114
11. Glossary	116
COMPLETE SOURCE CODE — CORE MODULES.....	120
I.1 Authentication / OTP Module (Complete).....	121
I.2 Document Module (Complete)	126
I.3 Analysis Module (Complete)	135
I.4 RAG Grounding (Complete)	146
I.5 Shared Middleware and Error Handling (Complete).....	152

1. Introduction and Objectives

1.1 Introduction

Legal documents — rental agreements, employment contracts, loan agreements, terms of service, privacy policies — are written in dense legal language that is difficult for a layperson to fully understand. A tenant signing a lease, an employee accepting an offer letter, or a consumer agreeing to an online service's terms rarely has the legal training to notice a one-sided termination clause, a non-refundable security deposit buried in a long paragraph, or a dispute-resolution clause that quietly waives their right to approach a court. This risk is not hypothetical — unfavourable clauses routinely go unnoticed until the moment they are actually enforced against the person who signed them.

Separately, when a person does need to understand their rights under Indian law — for a tenancy dispute, a criminal complaint, a case of cyber fraud, or the procedure for anticipatory bail — the realistic alternatives are limited: a lawyer's consultation, which is often costly and not immediately available; or a generic internet search, whose results are not grounded in the actual, current text of Indian statutes and offer no way to verify what they cite.

"AI-Powered Legal Document Analyzer and Risk Assessment System" is this project's formal title, consistent with the approved project proposal, and addresses the first problem directly. During implementation, the system was given the end-user-facing brand name NyayMitra AI (Hindi: न्यायमित्र, "friend of justice") — this is the product's name as seen in the application's interface, screenshots, and its own self-identification; it is not a change to the project's formal title, and both names refer to the same single system throughout this report.

The implemented system lets a user upload or paste a legal document and receive, within seconds, a plain-language summary, an automatically classified document type, a quantified 0–100 risk score, and a clause-by-clause breakdown of what is risky and why. The same system extends this goal to conversational legal assistance: a user can ask a question about Indian law in their own words — in English or Hindi, typed or spoken — and receive an answer grounded in the actual text of the relevant Act, with the exact section cited and verified against the official source rather than only the model's own, unverifiable recollection; or speak live, by voice, with an AI legal advocate, in a video-call-style interface, with the option to instead request a real, verified human advocate when one is available.

This report documents the system as implemented: the identification of need, requirement specifications, system analysis and design, the complete software development lifecycle followed, the testing actually performed — including real defects this testing found and fixed — and the security measures in place.

1.2 Objectives

The primary objective of this project is to design, implement, test and deploy a production-quality — not merely academic-prototype — system that:

1. Accepts a legal document for analysis either as an uploaded file (PDF, or a photograph/scan in JPG or PNG) or as directly pasted text.

2. Reads a scanned or photographed document that has no machine-readable text layer, by falling back to Optical Character Recognition, rather than only supporting born-digital PDFs.
3. Produces, via a Large Language Model, an AI-generated plain-language summary, an automatic document-type classification, a quantified risk score derived deterministically on the server (not left to the model to self-report, for consistency and auditability), and a structured, per-clause list of risk items with severity and category.
4. Processes each analysis as a background job on a durable queue, with automatic retry and bounded concurrency, so that the system stays responsive under load and a transient failure (for example, a momentary AI-provider outage) does not require the user to notice and manually retry.
5. Extends the same legal-assistance goal to open-ended conversational question answering about Indian law, grounded in Retrieval-Augmented Generation over the actual text of central statutes, in both English and Hindi, by typed and spoken input.
6. Provides a live, real-time voice consultation channel with an AI legal advocate, and a parallel channel to connect with a real, verified human advocate, including presence, request/accept/decline, and browser-to-browser audio/video.
7. Authenticates and authorises every request, validates all input, rate-limits cost-bearing operations, and specifically prevents one authenticated user from reading or modifying another user's data (an Insecure Direct Object Reference, or IDOR, class of vulnerability).
8. Is developed following a complete, demonstrable software engineering lifecycle — requirements analysis, system design, modular implementation, and a genuine automated test suite covering both unit and system/integration testing — rather than being verified only by manual, ad hoc use.
9. Documents its own Application Programming Interface directly from the server's real input-validation rules, so the documentation cannot silently drift from what the system actually accepts.

Table 1.1 traces each objective above forward to the specific functional requirement(s) in Section 2.3.1 that operationalise it — so a reader can confirm every objective was actually built, not merely stated, without first reading the full SRS.

Table 1.1 — Objective-to-Functional-Requirement Traceability

Objective	Satisfied by
1. Accept a document (file or pasted text)	FR-4
2. OCR fallback for scanned/photographed documents	FR-5
3. AI summary, classification, deterministic risk score	FR-6
4. Background job queue, retry, bounded concurrency	FR-7
5. Conversational Q&A, RAG-grounded, bilingual	FR-8, FR-9, FR-10
6. Live AI voice advocate + human advocate channel	FR-11, FR-12, FR-13

Objective	Satisfied by
7. AuthN/AuthZ, per-user isolation (IDOR)	FR-1, FR-2, FR-3
8. Complete, tested SDLC (not ad hoc verification)	Section 5 (Testing) — a process objective, not a single FR
9. Self-documenting API from real validation rules	FR-14 (FR-15's security headers/logging support the same “production-quality” framing without mapping to a single numbered objective)

2. System Analysis

2.1 Identification of Need

A legal document — a rental agreement, an employment offer letter, a loan or credit agreement, a freelance or vendor contract, a platform's terms of service — is a binding instrument the moment it is signed, yet the person signing it is very often not in a position to actually evaluate what they are agreeing to. Indian contract drafting conventions favour dense, formal, often one-sided language; a security-deposit clause, a unilateral termination right, an indemnity obligation, or an arbitration clause that quietly forecloses the right to approach a civil court can all sit in plain text, fully enforceable, and still go unnoticed by a layperson reading the document in good faith. The practical result is a persistent information asymmetry between the party that drafted the document and the party that is asked to sign it.

The conventional remedy — a lawyer's review before signing — does not scale to the volume of everyday documents an ordinary person actually encounters: a rental agreement, a gig-work contract, an online purchase's terms, a salary offer letter. A paid consultation for each of these is neither affordable nor realistically available on the timeline most of these documents demand (a landlord or employer rarely gives a prospective tenant or employee a week to have a lawyer review the paperwork). The alternative people actually reach for — searching the internet, or asking a general-purpose chatbot — has its own, different failure mode: neither is grounded in the current, correct text of the applicable Indian statute, and neither offers a way to verify what it claims against an authoritative source. A plausible-sounding but wrong answer about, say, the notice period for anticipatory bail or a tenant's rights under a state Rent Control Act, is arguably worse than no answer, because it is acted upon with the same confidence as a correct one.

This identifies two distinct, concrete needs that an ordinary user — not a law student, not a paralegal — actually has: first, a fast, low-cost way to understand what is risky in a specific document before signing it; and second, a way to ask a general question about Indian law and get an answer that is grounded in, and verifiably traceable to, the actual statutory text, with a clearly signposted escalation path to a real human advocate for anything that genuinely requires professional judgement rather than information retrieval. Existing generic AI assistants address neither need specifically: they are not grounded in Indian statutes by default, they do not produce a structured, quantified risk assessment of an uploaded document, and they offer no mechanism to connect the user to a verified professional when the question exceeds what an AI should answer alone. This gap — between "I have a document or a question right now" and "the only realistic help available is slow, expensive, or unverifiable" — is the specific need this project was proposed and approved to address.

The system's scope is deliberately bounded to keep this achievable and honest about what it is: it analyses documents and answers questions grounded in central Indian statutes (the scope explicitly does not claim state-specific amendments are covered unless a chunk for that state's Act has been ingested); it produces a risk assessment and a cited answer, not legal advice, and both the chat agent and the document analyser are expected to be used alongside — not instead of — a qualified advocate for anything consequential, which is exactly why the Connect Advocate module exists as a first-class escalation path rather than an afterthought.

Scope and Limitations

In scope:

- Document risk analysis for PDF, JPG/PNG (via OCR), and pasted-text input, grounded entirely in central Indian statutes actually ingested into the Legal Knowledge Base (Table 9.1).
- Conversational legal question-answering in English and Hindi, by typed or spoken input, with every citation verified against retrieved source text.
- A live voice channel with an AI legal advocate, and a parallel escalation path to a verified human advocate.
- Authentication via email/password, phone OTP, or Google OAuth; role-based access for a small admin surface.
- A genuine automated test suite, documented security measures, and API documentation generated from the system's own validation schemas.

Explicitly out of scope for this submission:

- State-specific statutes and amendments — only the ten central Acts in Table 9.1 are ingested; a question resting on a state amendment will not be answered correctly, and the system does not claim otherwise.
- Legal advice in the professional sense — every output carries a disclaimer (Section 6, Section 7.2) and is explicitly positioned to sit alongside, not replace, a licensed advocate.
- Multi-document comparison, downloadable report export, and a public advocate-ratings marketplace — named explicitly as Future Scope (Section 8), not silently dropped.
- Automated UI/voice-channel regression testing — the WebRTC and voice paths are verified manually against the running application (Section 5.2, Table 5.2) rather than by an automated browser-level suite, a known and stated limitation, not an oversight.

2.1.1 Feasibility Study

Before committing to the plan in Table 2.1, the project was assessed against the four conventional feasibility dimensions. None of this was a formal, separately-documented sign-off (the project had a single part-time developer and no external stakeholders to sign one), but each question below was concretely answered before Phase A began, and the answers are reproduced honestly here rather than written up after the fact to look more rigorous than the process actually was.

Technical feasibility.

Every component the design calls for — a Node.js/Express API, a React client, PostgreSQL with pgvector, Redis/BullMQ, and the OpenAI API for summarisation, embeddings, and voice — is either free/open-source or available on a pay-as-you-go tier with no upfront cost, and all of it runs on a single ordinary development laptop via Docker Compose (Appendix D.1). No component required specialised hardware, a GPU, or a paid enterprise licence. The one genuinely new technology for this developer, WebRTC (Section 2.3.4), was de-risked by scheduling it as its own late phase (Phase F, Table 2.1) after the core system was already working, so a technical dead end there would not have blocked the rest of the project.

Operational feasibility.

The system was designed to be usable without training: registration, document upload, and the chat interface all follow patterns (Figure 3.3-3.8) any user of a mainstream consumer web

application already knows. The one operational dependency genuinely outside the developer's control — advocates actually being available to accept human-consultation requests (Figure 2.6, uc7) — was treated as a real constraint, not assumed away: Section 8 (Future Scope) lists advocate-supply growth as unsolved rather than claiming the marketplace side of the problem is already complete.

Economic feasibility.

Cost Item	Estimate	Notes
Developer time	65 person-days (Table 2.2)	The project's only large cost, contributed by a single part-time MCA student developer
OpenAI API usage (development)	Low — pay-as-you-go, sandbox-tier usage during development	No fixed/minimum monthly fee; cost scales with actual test traffic only
MSG91 SMS OTP	Pay-per-SMS, sandbox credits during development	No fixed monthly fee at this scale
Hosting (PostgreSQL, Redis, server, client)	Free-tier capable	Every component in Table 3.x/Appendix D runs on free tiers of common hosts (a small VM or PaaS free tier) at this traffic volume
Software licences	Nil	Every dependency in Appendix A/Bibliography (Section 9.2) is open-source

There is accordingly no economic barrier to completing or running the system at the scale of an academic project and a small initial user base; the only cost that would grow materially with real adoption is OpenAI API usage, which is directly proportional to actual user activity rather than a fixed sunk cost.

Schedule/legal feasibility.

Table 2.1's schedule was sized against the real, constrained time available to a part-time student developer (evenings/weekends around coursework and full-time employment, Section 2.2), which is why the two term-time gaps in the Gantt chart were planned around rather than treated as risk to be absorbed later. On the legal side, the system's own scope boundary (above) — grounded statute citation rather than legal advice, with an unconditional disclaimer (Section 4.2, Section 7.2) — was itself adopted specifically so the project would not need a law firm's sign-off or professional-indemnity cover to be built and demonstrated academically.

2.2 Project Planning and Scheduling

The project was planned and executed as seven incremental phases, each delivering a working, demonstrable increment rather than a layer of an unfinished whole. The schedule below is reconstructed directly from the project's own Git commit history (first and last commit date actually recorded against each phase's work), not from a plan written after the fact. The two visible gaps — between Phase A/B and Phase C in April–May, and the long gap between Phase C and Phase D across June–July — correspond to University term-end examinations and semester breaks, during which no development work was carried out; they are shown here rather than concealed, because the schedule is intended to be an honest record of how the project actually progressed as part-time, alongside the author's MCA coursework and full-time employment.

Table 2.1 — Development Phases and Schedule

Phase	Mar	Apr	May	Jun	Jul	Aug	Sep	Oct
A. Backend Setup & Authentication	A	A						
B. Document Upload & AI Integration		B						
C. Frontend Foundation & OAuth			C					
D. Prisma Migration & Analysis Enhancement						D		
E. Legal Assistant Chat Agent + RAG						E		
F. Rebrand & Connect Advocate							F	
G. Documentation & Production Hardening							G	G

Figure 2.1 models the same seven phases as a PERT (Program Evaluation and Review Technique) network, which is the more appropriate tool for reasoning about dependency and schedule risk rather than elapsed calendar time alone. Phase B (Document Upload & AI Integration) and Phase C (Frontend Foundation & OAuth) could, in principle, proceed in parallel once Phase A's authentication layer existed — both depend only on Phase A, not on each other — so they are modelled as two parallel activities between events 2 and the merge event 5. Because Phase C's actual effort (4 weeks) exceeded Phase B's (1 week), Phase C — not Phase B — sits on the critical path: the project's overall schedule was bound by how long the frontend foundation took, even though the backend's document/AI integration finished sooner. This is the standard justification for distinguishing a critical path from a merely long one, and it is a real property of this project's own history, not a constructed example.

Cost and Effort Estimation

Function Point Analysis — the standard method for data-centric business applications of fixed, known scope — fits poorly here: a large share of this system's complexity is in AI integration, a background job pipeline, and a live voice/WebRTC channel, none of which FPA's input/output/inquiry/file counting model was designed to weight. A simpler, task-based effort estimate was used instead, in person-days, directly consistent with the PERT estimate in Figure 2.1 (each phase's weeks of active effort $\times$ 5 working days, for a single part-time developer).

Table 2.2 — Cost and Effort Estimation

Phase	Effort (person-days)	Complexity	Primary Cost Driver
A — Backend Setup & Auth	15	Medium	Auth flows, schema design
B — Document Upload & AI Integration	5	Medium	File handling, first AI integration
C — Frontend Foundation & OAuth	20	High	Full UI scaffold, OAuth integration
D — Prisma Migration & Analysis Enhancement	10	Medium	Schema migration, caching logic

Phase	Effort (person-days)	Complexity	Primary Cost Driver
E — Legal Assistant + RAG	5	High	RAG pipeline, embeddings, streaming
F — Rebrand & Connect Advocate	5	High	Realtime voice, WebRTC, admin panel
G — Documentation & Hardening	5	Medium	Tests, validation, security headers, docs
Total	65 person-days ($\approx$ 13 weeks, solo, part-time)	—	—

Complexity weighting follows the same logic the PERT critical path already demonstrated: Phase C is rated High despite being UI work (not AI or real-time) because it was, in practice, the project's single largest time sink (Figure 2.1) — a useful reminder that “complex” and “time-consuming” do not always coincide, and an estimate built only around novel technology (AI, WebRTC) would have under-weighted it.

Project Risk Management

Table 2.2a records the risks that were actually live during development, not a generic textbook list — each row's mitigation is a real, implemented control already referenced elsewhere in this report, not a future promise.

Table 2.2a — Risk Register

Risk	Likelihood	Impact	Mitigation (Implemented)
AI provider cost or rate-limit pressure under load	Medium	High	Background queue with bounded concurrency (Section 3.1) caps simultaneous OpenAI calls; retry with backoff (Section 4.2) absorbs transient failures instead of resending immediately
Single part-time developer — a bus-factor-of-one risk	High	High	Incremental delivery (Table 2.1) kept every phase independently shippable; the system test suite (Table 5.1) is a safety net for solo refactoring
Academic-term scheduling gaps (exams, semester breaks)	High	Medium	The Gantt chart (Table 2.1) was planned around, not hidden from — phases sized to fit realistic available windows rather than assuming continuous time
AI-generated legal citation is wrong or fabricated	Medium	High	RAG grounding plus server-side citation verification (Figure 2.8) — a citation not actually retrieved is marked unverified, never presented as confirmed
Scope growth beyond the originally approved synopsis	Medium	Medium	New capability (Legal Assistant, Connect Advocate) was added as whole new phases/modules (Table 2.1, Figure 3.2), not folded into existing modules — scope growth stayed visible, not silently absorbed
Third-party SMS gateway (MSG91) cost from abuse or retries	Low	Medium	Per-user rate limiting and a 30-second resend cooldown on OTP (Table 6.1) bound the worst case
Sensitive document content exposed to another user	Low	High	Per-request ownership checks (Section 6.2); no uploaded document is ever served from a static/public path (Section 6.4)

Risk	Likelihood	Impact	Mitigation (Implemented)
WebRTC/real-time voice — new technology for this project	Medium	Medium	Introduced in its own phase (Phase F, Table 2.1) after the core system was stable, and verified manually against the running app (Section 5.2) rather than assumed correct

Figure 2.1 — PERT Chart (Network Diagram)

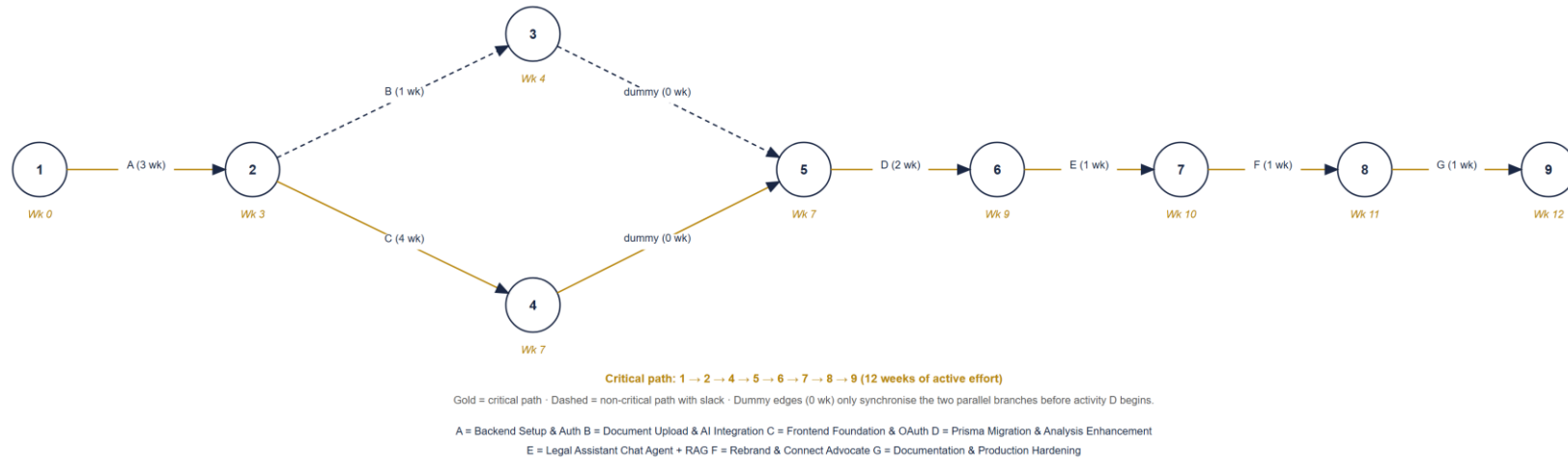


Figure 2.1 — PERT Chart (Network Diagram)

2.3 Software Requirement Specifications (SRS)

2.3.1 Functional Requirements

Module A — Authentication & User Management

- FR-1: The system shall allow a user to register and log in using an email/password pair or Google OAuth.
- FR-2: The system shall support phone-number login via a one-time password (OTP) delivered through the MSG91 SMS gateway.
- FR-3: The system shall scope every data-accessing request to the authenticated user's own records (preventing one user from reading or modifying another user's documents, conversations, or consultations — an IDOR-class control).

Module B — Document Analysis

- FR-4: The system shall accept a legal document as an uploaded PDF, JPG, or PNG file, or as directly pasted text.
- FR-5: The system shall extract text from a scanned or photographed document with no machine-readable text layer by falling back to Optical Character Recognition (Tesseract.js).
- FR-6: The system shall produce, for an analysed document, a plain-language summary, an automatic document-type classification, a 0–100 risk score computed deterministically on the server, and a structured, per-clause list of risk items with severity and category.
- FR-7: The system shall process each analysis as a background job on a durable, Redis-backed queue with automatic retry on transient failure, and shall expose a polling endpoint for the client to observe job status.

Module C — Legal Assistant

- FR-8: The system shall allow a user to ask an open-ended question about Indian law, by typed or spoken input, in English or Hindi.
- FR-9: The system shall ground its answer in the actual text of the relevant central statute using Retrieval-Augmented Generation, and shall cite the specific Act and section, verified against the retrieved source text rather than the model's unverified recollection.
- FR-10: The system shall stream the assistant's reply to the client incrementally (Server-Sent Events) rather than waiting for the full response to be generated.

Module D — Connect Advocate

- FR-11: The system shall provide a live, real-time voice consultation channel with an AI legal advocate.
- FR-12: The system shall allow a user to request a live consultation with a verified human advocate, with request/accept/decline semantics and browser-to-browser audio/video.
- FR-13: The system shall allow an administrator to manage advocate accounts, including verification status.

Module E — Platform

- FR-14: The system shall publish interactive API documentation (OpenAPI/Swagger) generated directly from the server's real input-validation schemas, so the documentation cannot silently drift from what the API actually accepts.
- FR-15: The system shall apply standard HTTP security headers and emit structured, redacted logs for every request.

Table 2.1a gives each requirement an explicit, testable acceptance criterion, rather than leaving "done" to interpretation — most rows are, directly, the condition a specific test case in Table 5.3 actually checks.

Table 2.1a — Functional Requirement Acceptance Criteria

ID	Acceptance Criterion
FR-1	A new account can be created and logged into via email/password or Google OAuth; a duplicate email is rejected (TC-01).
FR-2	An OTP is delivered via MSG91 and accepted exactly once before expiry; a second /send within 30s is rejected (TC-04); a consumed code is rejected on reuse (TC-05).
FR-3	A request for another user's document, conversation, or consultation returns 403, never the resource or a leaking 404 (TC-06).
FR-4	A PDF, JPG, or PNG upload, or pasted text of at least 50 characters, is accepted; shorter pasted text is rejected at the schema (TC-07).
FR-5	A scanned/photographed document with no text layer still yields usable extracted text via the OCR fallback (Figure 2.5).
FR-6	A completed analysis has a summary, a document-type label, a 0–100 score, and at least one risk item when the source text contains flaggable clauses.
FR-7	A run request returns 202 immediately rather than blocking on the AI call; a transient AI failure is retried automatically (TC-10) and a permanent one ends in Failed (TC-11).
FR-8	A typed or spoken question in English or Hindi returns a relevant, grounded reply.
FR-9	Every citation in a reply traces to an actually-retrieved LegalKnowledgeChunk; an unsupported citation is marked unverified, never presented as confirmed.
FR-10	The client renders the reply incrementally as SSE tokens arrive, not only after the full response completes.
FR-11	A user can start and hold a live voice exchange with the AI advocate end-to-end without the call dropping under normal network conditions.
FR-12	A human-advocate request reaches REQUESTED and resolves to exactly one of ACCEPTED/DECLINED/EXPIRED/CANCELLED (Figure 2.11b), never left ambiguous.
FR-13	An admin can verify, suspend, or edit an advocate account; a non-admin attempting the same route receives 403.
FR-14	/api-docs renders a working Swagger UI whose schemas are generated from the same Zod definitions the API enforces, not maintained by hand.
FR-15	A normal API response carries Helmet's baseline headers (TC-14); a logged request with a password or token field shows it redacted (TC-18–TC-20).

2.3.2 Non-Functional Requirements

- Performance: cost-bearing or slow operations (AI analysis, OCR) run on a background queue so the HTTP API itself stays responsive; the client polls for completion at a bounded interval rather than holding a request open.

- **Security:** all request input is validated against explicit Zod schemas at the API boundary; HTTP responses carry Helmet-applied security headers; sensitive fields (passwords, tokens, OTPs) are redacted from logs at the point of logging, not after the fact; cost-bearing endpoints are rate-limited per user.
- **Reliability:** background jobs retry automatically with exponential backoff on transient failure, and a job is only marked failed after its final attempt is exhausted.
- **Usability:** error messages shown to the user are plain-language, not raw HTTP/stack-trace text; the interface and the legal assistant both support English and Hindi.
- **Scalability:** the API is stateless and horizontally scalable; the analysis worker pool's concurrency is independently configurable from the API process.
- **Maintainability:** the codebase is organised into modular feature services; a genuine automated test suite (unit and system/integration, Vitest) covers core business logic; the API documentation is generated from the same schemas that validate requests, so it cannot drift silently out of sync with the implementation.
- **Portability:** local infrastructure dependencies (PostgreSQL, Redis) are defined via Docker Compose, and all environment-specific configuration is externalised to environment variables.

Table 2.1b gives each non-functional requirement a measurable target rather than leaving "fast" or "reliable" to interpretation.

Table 2.1b — Non-Functional Requirement Targets

NFR	Measurable Target
Performance	POST /analysis/run returns within 300ms regardless of document size, since it only enqueues a job (Figure 2.7) — the AI call itself never sits on this request
Security	Zero plaintext secrets in the repository (Section 6.4); every request body validated before reaching a service (Table 5.3, TC-07/TC-15/TC-16)
Reliability	A transient AI failure succeeds within 3 retry attempts (exponential backoff); a document is only ever left Failed after every attempt is exhausted (Table 5.3, TC-10/TC-11)
Usability	No raw HTTP status text or stack trace is ever shown to the user (Table 5.3, TC-13); every screen and the Legal Assistant's own replies work in English and Hindi
Scalability	Analysis worker concurrency (Section 3.1) is a single environment variable, changeable without a code change or redeploying the API process separately
Maintainability	253/253 tests passing (Table 5.1) is a merge gate (Table 5.2); OpenAPI documentation is generated, not hand-maintained, so it cannot silently drift from FR-14
Portability	A fresh clone reaches a running local stack with docker compose up plus one prisma migrate deploy (Appendix D) — no manually-installed local database or queue software

2.3.3 Hardware and Software Requirements

Table 2.3 — Hardware and Software Requirements

Component	Development Environment	Minimum Deployment Target
Server runtime	Node.js 20 LTS, 8 GB RAM, Windows / Linux / macOS	Node.js 20 LTS, 1 vCPU, 1 GB RAM
Database	PostgreSQL 15 (Docker container)	PostgreSQL 15, pgvector extension enabled

Component	Development Environment	Minimum Deployment Target
Queue / cache	Redis 7 (Docker container)	Redis 7, persistent volume
Client build	Vite + React 18, Node.js 20 LTS	Static hosting (any CDN / Node static server)
Browser (client)	Chromium-based browser, latest version	Any modern browser supporting SSE and WebRTC (getUserMedia) for voice/video features
External services	OpenAI API key, MSG91 API key (sandbox)	OpenAI API key, MSG91 API key (production)

2.3.4 Assumptions, Dependencies and Constraints

Assumptions:

- The user has a modern browser and a working internet connection; the system does not support fully offline use.
- A document's text, once extracted (Section 2.5.2), is in a language the configured AI model can process — Hindi and English specifically, consistent with FR-8.
- The phone number used for OTP login can receive SMS; a landline or SMS-blocked number cannot complete phone-based registration.

Dependencies:

- The OpenAI API — for document summarisation, risk scoring, embeddings, and the Realtime voice API — is a hard external dependency; the system has no local fallback model if it is unavailable.
- The MSG91 SMS gateway is a hard dependency for phone-OTP login specifically; email/password and Google OAuth (FR-1) remain available if it is down.
- PostgreSQL with the pgvector extension is required, not optional — RAG retrieval (Figure 2.8) cannot run against a pgvector-less database.

Constraints:

- Development and testing were constrained to the hardware and time available to a single, part-time developer (Table 2.2), which directly shaped the incremental paradigm chosen (Section 2.4).
- The Legal Knowledge Base is limited to the ten central Acts actually ingested (Table 9.1) — a hard data constraint, not a configurable one, until further statutes are ingested (Section 8, item 1).
- No budget existed for a paid, dedicated TURN server; WebRTC connectivity (Figure 2.9) relies on STUN and falls back to whatever TURN configuration is supplied via environment variables (Appendix B).

2.3.5 Interface Requirements

User interfaces.

A single responsive web interface (Section 3.4, Figures 3.3-3.8) serves every actor — User, Human Advocate, and Admin — with the available actions gated by role rather than by a separate application per role. There is no native mobile app; the client is a browser-based single-page application (Appendix D.3) that runs unmodified on a phone's mobile browser.

Hardware interfaces.

The system has no interface to purpose-built hardware. The only hardware-adjacent interfaces are ones every modern browser already mediates: a microphone, accessed through the browser's `getUserMedia` API for the Connect Advocate voice/video call (Figure 2.9, Appendix E.5), and standard client storage for the session token (Section 6.1) — nothing the server talks to directly.

Software interfaces.

Interface	Protocol / Format	Purpose
Client ↔ Server (CRUD)	HTTPS, JSON (REST)	Ordinary request/response operations — documents, auth, admin (Appendix C)
Client ↔ Server (Legal Assistant)	HTTPS, Server-Sent Events	Token-by-token streamed chat replies (Figure 2.8)
Client ↔ Server / Client ↔ Client	WebRTC (SRTP media, DataChannel)	Live audio/video for Connect Advocate (Figure 2.9)
Server ↔ PostgreSQL	Prisma query engine over TCP (Appendix B: DATABASE_URL)	All persistent storage, including pgvector similarity search
Server ↔ Redis	ioredis/BullMQ protocol over TCP (Appendix B: REDIS_URL)	Background job queue and rate-limiter store (Section 3.1)
Server ↔ OpenAI API	HTTPS, JSON / streaming	Chat completion, embeddings, Realtime voice (Appendix B: OPENAI_API_KEY)
Server ↔ MSG91	HTTPS, JSON	OTP SMS delivery (Appendix B: MSG91_AUTH_KEY)

Communication interfaces.

All client-server traffic is expected to run over TLS in any deployed environment (Section 6.3); CORS is restricted to the configured `CLIENT_URL` rather than left open (Appendix B). No interface in this system accepts an unauthenticated write beyond registration, login, and OTP verification themselves (Table 6.1).

2.4 Software Engineering Paradigm Applied

This project followed an Incremental (iterative) software development model rather than a strict, single-pass Waterfall model. Each of the seven phases in Table 2.1 was planned, implemented, and left in a working, demonstrable state before the next phase began — Phase A delivered working authentication end-to-end before Phase B's document upload existed; Phase B's document upload and AI integration worked end-to-end before Phase D's analysis caching and risk-score restructuring revisited and improved it; the legal assistant (Phase E) was added as a self-contained increment on top of an already-working document analyser rather than being designed into the system from day one.

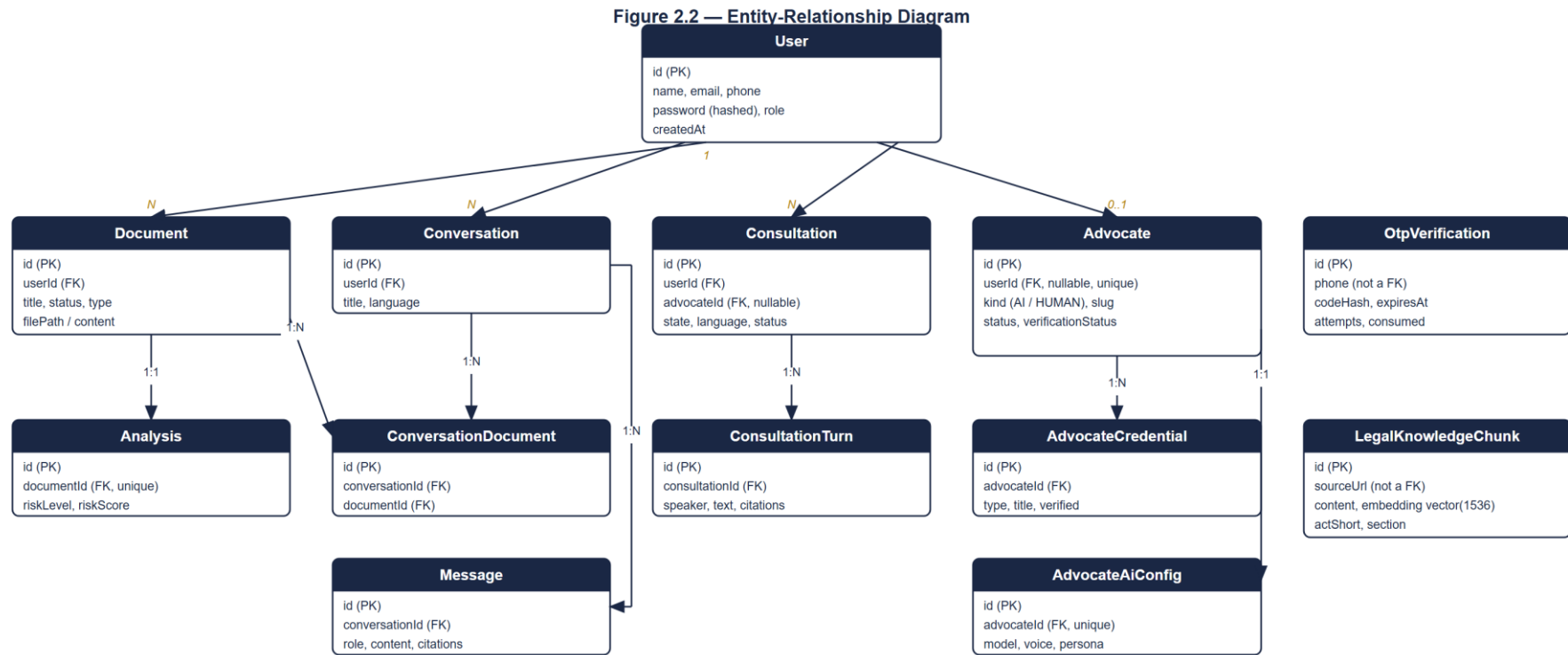
This choice was not arbitrary. The requirements for this project matured over time — the approved synopsis specified document risk analysis as the core deliverable, and the conversational legal assistant and the Connect Advocate module were designed and added once the core analyser was stable, informed by what the working system's own gaps suggested the system still needed. An incremental model accommodates this kind of requirement growth far better than Waterfall, which assumes the complete requirement set is known and fixed before design begins. It also matched the realistic constraint of part-time, single-developer-led execution around University terms: each phase was small enough to design, build, and stabilise within the days or weeks actually available, rather than requiring a long, uninterrupted analysis-and-design stage up front that a part-time schedule could not sustain in one sitting.

Each phase itself followed the same internal shape — a short analysis of what the increment needed, implementation, and manual verification against the running application before being folded into the next phase's starting point — and the final hardening phase (Phase G) retrofitted a genuine automated test suite, input validation layer, and security/observability tooling across the whole, already-functioning system, which is itself a recognised and defensible pattern within an incremental lifecycle: later increments are free to revisit and strengthen earlier ones, rather than every quality gate having to be front-loaded into the very first phase.

2.5 Data Models

2.5.1 Entity-Relationship Diagram

Figure 2.2 shows the system's complete relational schema as implemented in Prisma ORM, covering all thirteen entities. User sits at the centre of the model: a user owns documents, conversations, consultations, and — for an advocate account — an optional Advocate profile. Document has a one-to-one relationship with Analysis (each document is analysed at most once, cached rather than re-run); Conversation relates to both ConversationDocument (the documents grounding a chat) and Message in a one-to-many fashion. Advocate relates one-to-many to AdvocateCredential and one-to-one to AdvocateAiConfig. OtpVerification and LegalKnowledgeChunk are intentionally standalone: the former is keyed by phone number rather than a user foreign key (a user may not exist yet at the point an OTP is requested), and the latter is retrieved by vector-similarity search rather than a relational join.



Standalone entities (no foreign key into the diagram above): OtpVerification (keyed by phone), LegalKnowledgeChunk (retrieved by vector similarity, not joined).

Figure 2.2 — Entity-Relationship Diagram

2.5.2 Data Flow Diagrams

Figure 2.3 is the Level 0 (context) DFD, showing the system as a single process bounded by its four external entities: the User, the MSG91 OTP gateway, the AI/LLM provider, and a human Advocate. Figure 2.4 decomposes that single process into its five major internal processes and six data stores; external-entity flows already established in Figure 2.3 are omitted from Figure 2.4 to keep the internal decomposition legible. Figure 2.5 goes one level further, decomposing process 3.0 (Analyze Document) — the process with the most internal structure worth surfacing, since it is where text extraction, the OCR fallback, the AI call, and the deterministic risk-score derivation (Section 3.3) all actually happen.

Figure 2.3 — DFD: Context Diagram (Level 0)

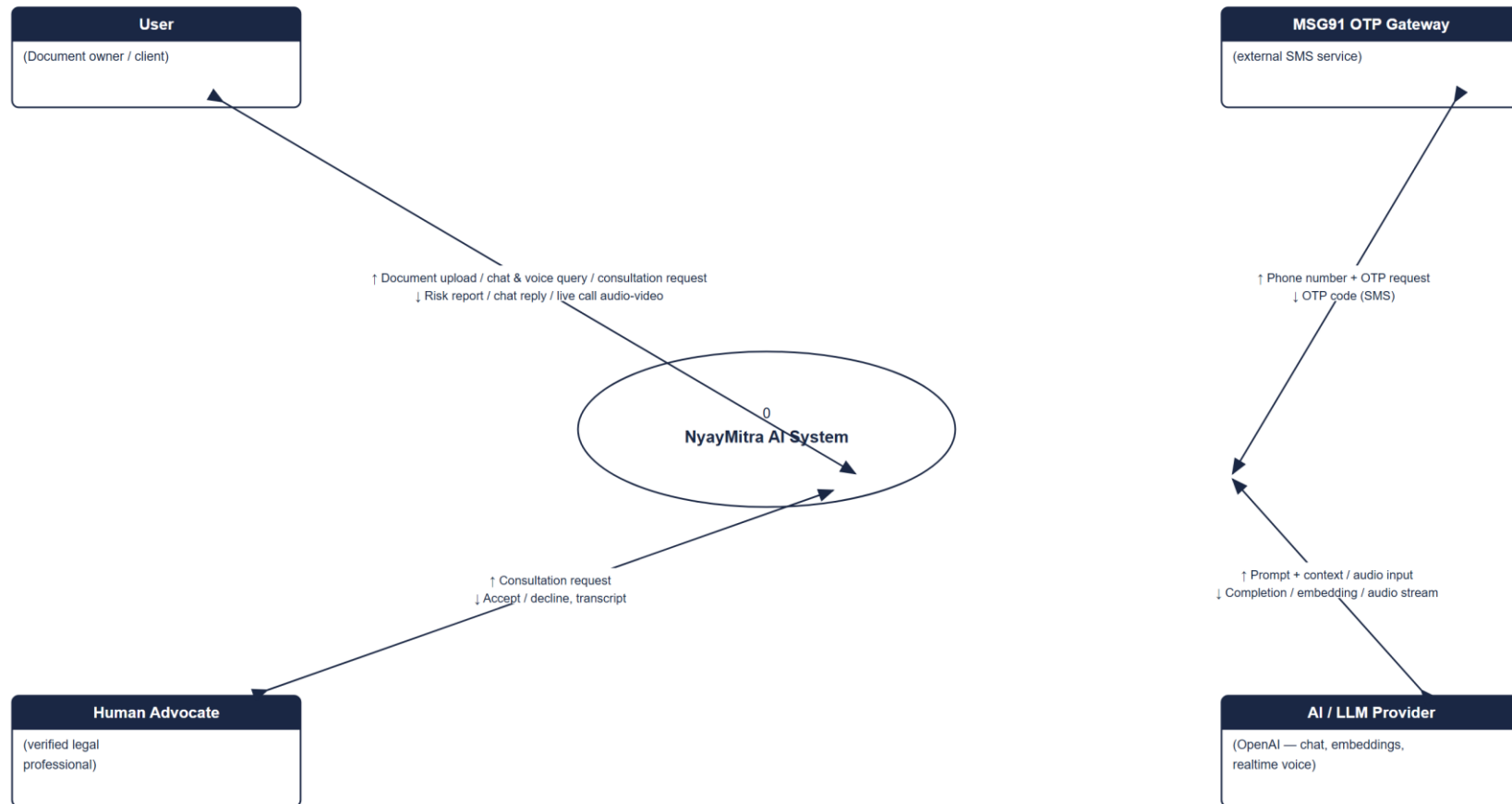
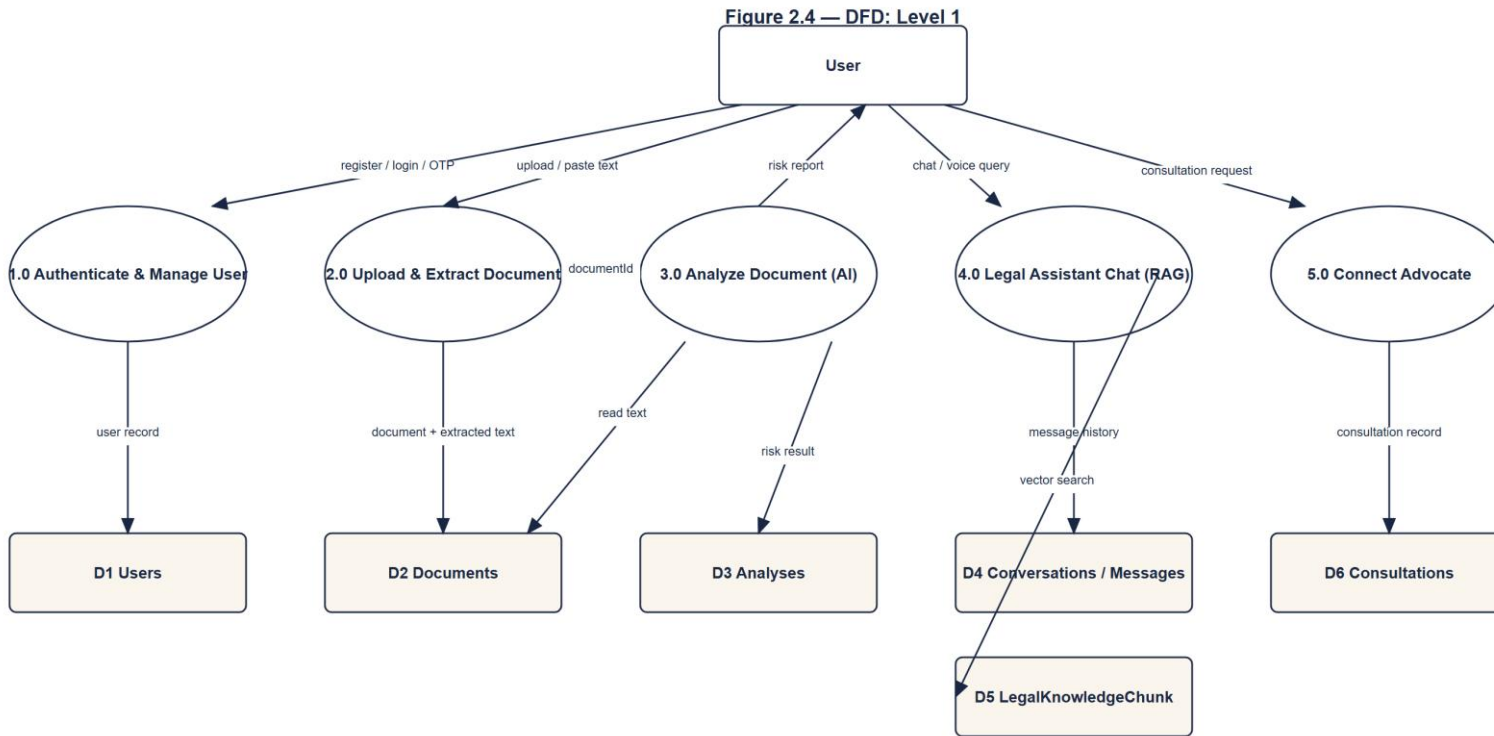


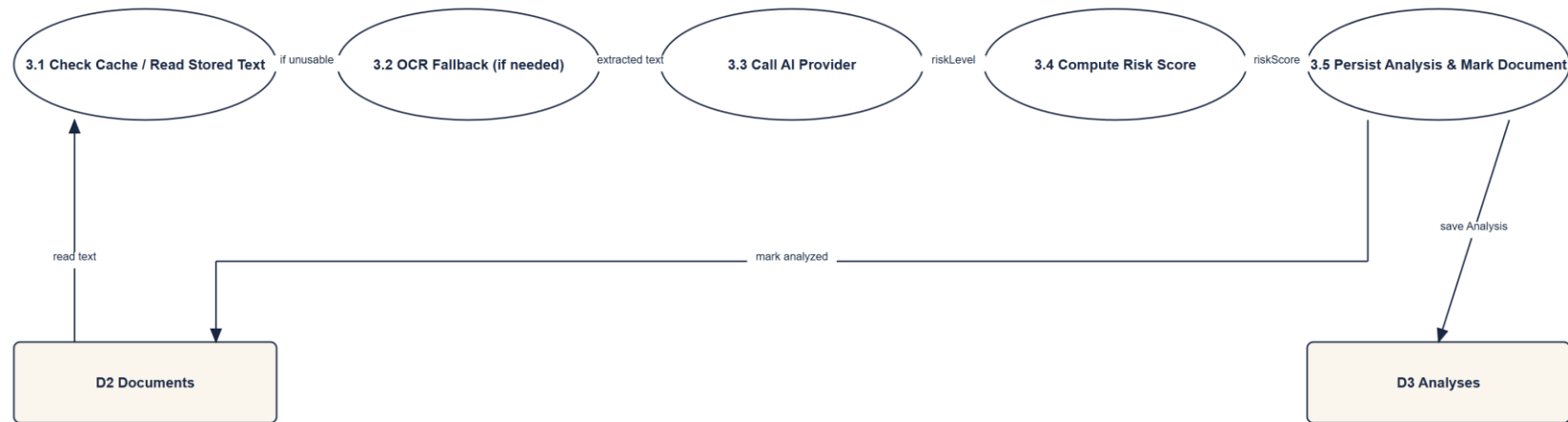
Figure 2.3 — DFD: Context Diagram (Level 0)



External-entity flows (User's risk report / chat reply, MSG91 OTP, the AI/LLM provider, and the human advocate hand-off) are shown in Figure 2.3 (Context Diagram) and omitted here for clarity.
 Process 5.0 also reads the advocate's profile from D1 (Users) when matching a consultation request; omitted above to avoid a diagram-spanning crossing line.

Figure 2.4 — DFD: Level 1

Figure 2.5 — DFD: Level 2 (Decomposition of Process 3.0 — Analyze Document)



Decomposes process 3.0 (Figure 2.4) — the AI/LLM Provider external-entity flow at 3.3 is shown in Figure 2.3 (Context Diagram) and omitted here for clarity.

Figure 2.5 — DFD: Level 2 (Decomposition of Process 3.0)

2.5.3 Use Case Diagram

Figure 2.6 identifies the system's three actors — the registered User, a verified Human Advocate, and an Admin — and the principal use cases each actor drives. Register & Login includes Verify Phone (OTP) as a sub-flow, shown with the standard UML «include» relationship, since OTP verification is a reusable step invoked from registration rather than a standalone, user-initiated action.

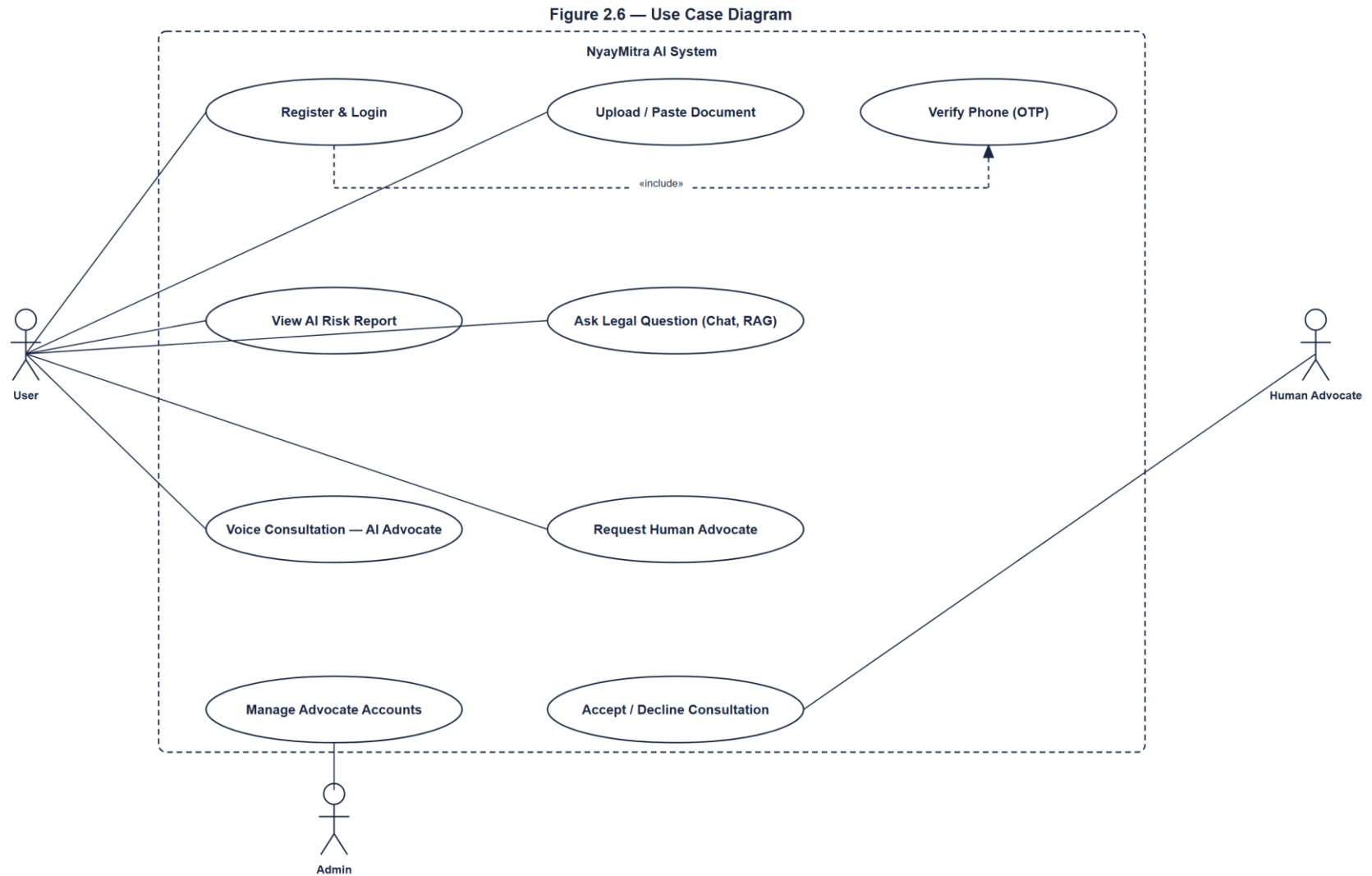


Figure 2.6 — Use Case Diagram

2.5.3.1 Use Case Specifications

A short textual specification for each use case in Figure 2.6, in the conventional actor / precondition / main flow / exception flow / postcondition form, cross-referenced to the functional requirement and sequence diagram each one is ultimately implemented by.

UC-1 — Register & Login

Actor: User. Precondition: none (first visit) or an existing account (returning visit).

1. User opens the Home screen (Figure 3.3) and selects Register or Login.
2. For Register: the user supplies either an email/password pair or a phone number (FR-1); a phone number triggers UC-9 (Verify Phone) as an «include» sub-flow before the account is created.
3. For Login: the user supplies the matching credential; the server validates it and issues a JWT session token (Section 6.1).
4. The client stores the token and attaches it to every subsequent API call (Figure 3.9).

Exception flow: invalid credentials or an unverified phone number return a form-level error (Section 6.1) without revealing which field was wrong, to avoid account enumeration (Section 6.2). Postcondition: the user holds a valid session token.

UC-2 — Upload / Paste Document

Actor: User. Precondition: the user holds a valid session (UC-1).

1. User selects Upload Document on the Dashboard (Figure 3.5) and chooses a PDF/JPG/PNG file, or pastes raw text.
2. The client sends the file to POST /documents/upload (FR-5); the server stores it and creates a Document row with status pending.
3. The client calls POST /analysis/run to enqueue background analysis (Figure 2.7), then begins polling for status.

Exception flow: an unsupported file type or a file exceeding the configured size limit is rejected by the Zod-validated upload route (Section 4.2) before it reaches storage. Postcondition: the document is queued (or rejected with a clear reason).

UC-3 — View AI Risk Report

Actor: User. Precondition: UC-2 has reached status completed.

1. User opens the analysed document from the Dashboard's document list.
2. The client requests the cached Analysis record (Section 3.3, Table 7.1 fields) and renders the risk score, summary, and clause-by-clause risk items (Figure 3.6).

Exception flow: if analysis failed (e.g. OCR could not extract text from an unreadable scan), the Dashboard shows a failed status with a plain-language reason instead of a silent blank report. Postcondition: the user has read the report; no state changes.

UC-4 — Ask Legal Question (Chat, RAG)

Actor: User. Precondition: the user holds a valid session (UC-1).

1. User types or speaks a question on the Legal Assistant screen (Figure 3.7), in English or Hindi (FR-9, FR-10).

2. The client opens an SSE connection to POST /legal-agent/conversations/:id/messages (Appendix C); the server retrieves grounded context (RAG, Figure 2.8) and streams the answer token-by-token.
3. Each citation in the streamed answer is verified against the retrieved source text before being shown (Section 2.5.4).

Exception flow: if retrieval finds no matching statute passage, the answer is still returned but without a citation for that claim, signalling lower confidence rather than fabricating a source (Section 7.2). Postcondition: the conversation (Message rows) is saved for the session.

UC-5 — Voice Consultation (AI Advocate)

Actor: User. Precondition: the user holds a valid session (UC-1).

1. User opens Connect Advocate (Figure 3.8), selects an AI advocate persona, and starts a call (FR-11, FR-12).
2. POST /consultations/:id/connect opens a live voice session with the Realtime API (Figure 2.9); the client polls /consultations/:id/turns for the transcript as the call proceeds.
3. On POST /consultations/:id/end, the server generates the Consultation Summary Report (Section 7.3) from the full transcript and re-verified citations.

Exception flow: a dropped connection ends the call early; the summary is still generated from whatever transcript exists up to that point. Postcondition: a stored, citation-checked consultation summary.

UC-6 — Request Human Advocate

Actor: User. Precondition: at least one advocate is registered and marked available (UC-8).

1. User describes the matter on Connect Advocate and submits a request (FR-13) via POST /human-consultations.
2. The request appears on the matching advocate's Advocate Desk (GET /advocate-desk/consultations/:id) for UC-7.
3. Once accepted, the client negotiates a WebRTC call using ICE servers from GET /human-consultations/:id/ice.

Exception flow: the user may cancel via POST /human-consultations/:id/cancel before it is accepted. Postcondition: a live call in progress, or a cancelled/declined request.

UC-7 — Accept / Decline Consultation

Actor: Human Advocate. Precondition: the advocate is linked to an account (UC-8) and marked available.

1. Advocate opens the Advocate Desk and reviews a pending request (GET /advocate-desk/consultations/:id).
2. Advocate accepts (POST .../accept) or declines (POST .../decline) the request.
3. On acceptance, the advocate may record private or shared notes during/after the call (PUT .../notes).

Postcondition: the consultation's status reflects the advocate's decision; the requesting user is notified of the outcome via the consultation's status field.

UC-8 — Manage Advocate Accounts

Actor: Admin. Precondition: the admin holds a session with the Admin role (Section 6.2).

1. Admin creates, updates, or removes an advocate profile and its credentials (POST/PATCH/DELETE /admin/advocates, Appendix C).
2. Admin configures the AI advocate persona's behaviour (PUT /admin/advocates/:id/ai-config) and links/unlinks a login account to a human advocate profile.

Exception flow: every route in this use case is gated by requireAdmin (Section 6.2); a non-admin session receives 403 Forbidden. Postcondition: the advocate roster used by UC-6/UC-7 reflects the change.

UC-9 — Verify Phone (OTP)

Actor: User. Precondition: invoked only as an «include» sub-flow of UC-1.

1. Server sends a one-time password to the supplied phone number via the MSG91 gateway (Section 6.1).
2. User enters the received code; the server validates it against the stored OTP record within its validity window.

Exception flow: an expired or incorrect OTP is rejected with a friendly error (Section 6.1) and a resend option is offered after a cooldown (Table 2.2a). Postcondition: the phone number is marked verified, and UC-1 proceeds to create the account.

2.5.4 Sequence Diagrams

Figure 2.7 traces the document-analysis flow end to end, specifically to illustrate the background-queue design implemented in Phase D: the API responds 202 Accepted immediately after enqueueing the job rather than blocking on the AI call, and the client polls for the result. Figure 2.8 traces the legal-assistant chat flow implemented in Phase E, showing the Retrieval-Augmented Generation sequence — a vector-similarity search against the LegalKnowledgeChunk store before the AI provider is ever called — and the server-side citation verification step that runs before a reply is persisted and shown to the user.

Figure 2.9 traces the Connect Advocate live-consultation flow (Phase F): a WebRTC audio channel negotiated through a thin signalling relay, with the consultation summary (Section 7.3) generated once the call ends rather than during it. Figure 2.10 traces the phone-OTP authentication flow (FR-2), showing exactly where the code is hashed, where it is compared, and where the single-use and rate-limiting checks from Table 6.1 actually run.

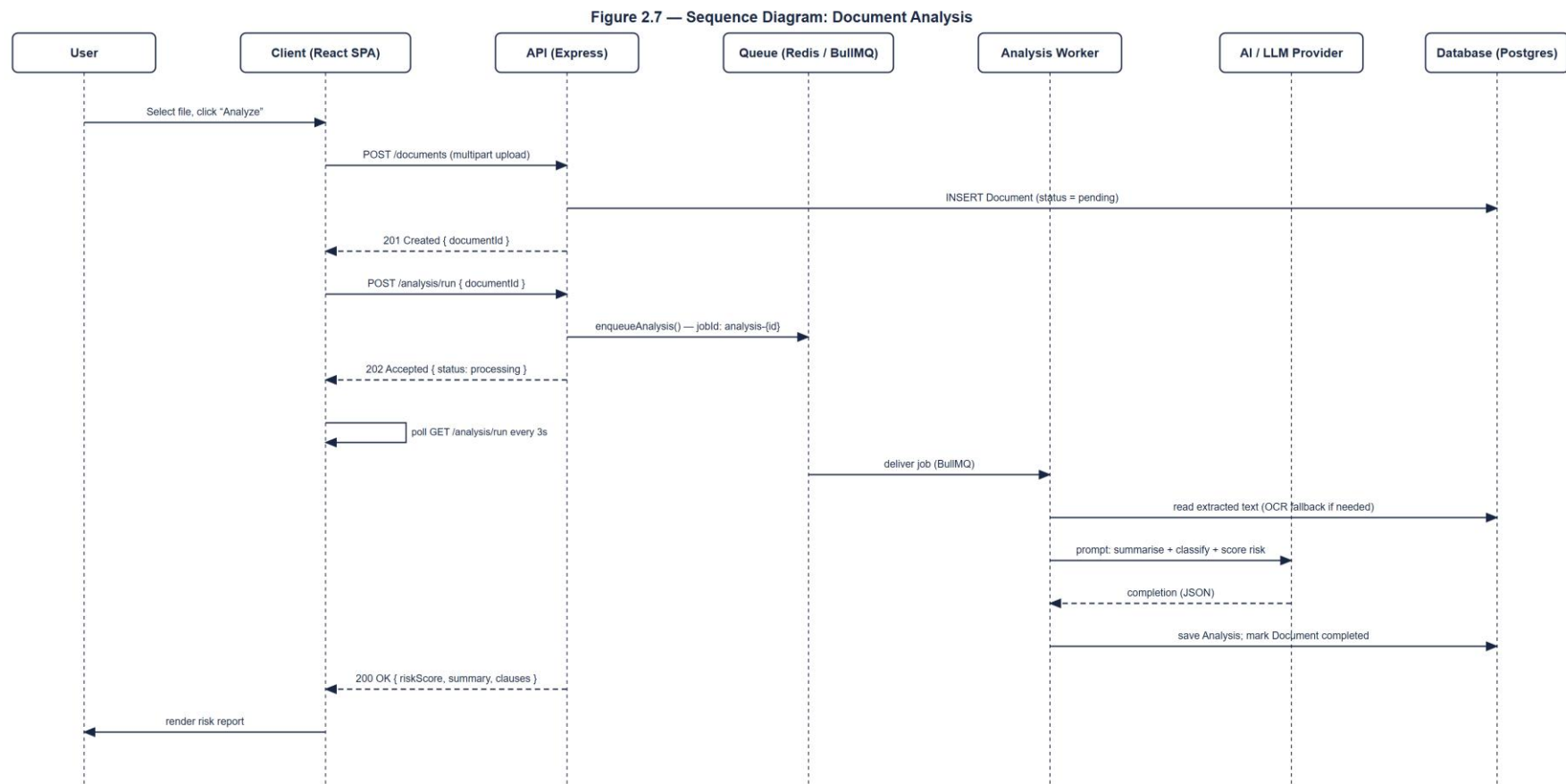


Figure 2.7 — Sequence Diagram: Document Analysis

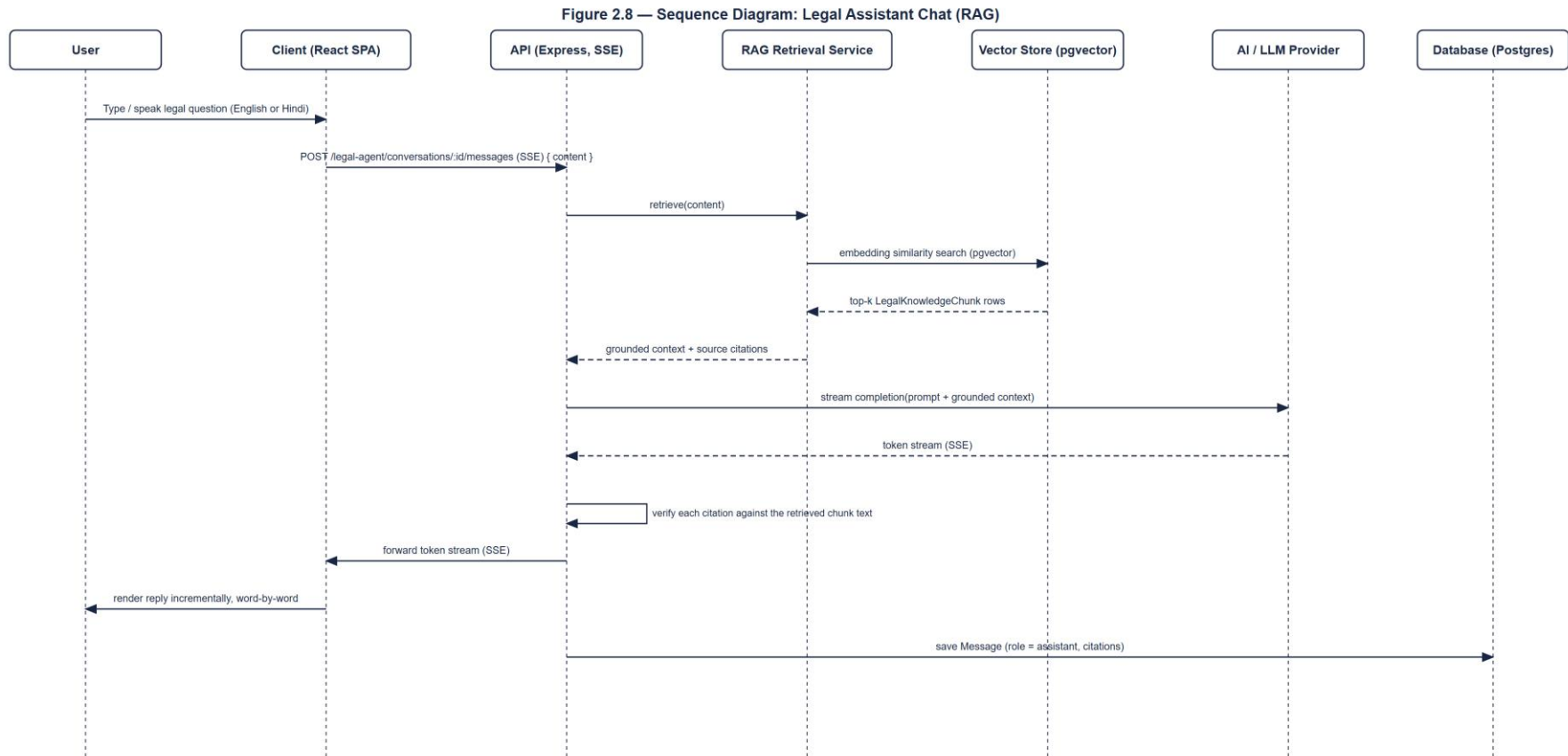


Figure 2.8 — Sequence Diagram: Legal Assistant Chat (RAG)

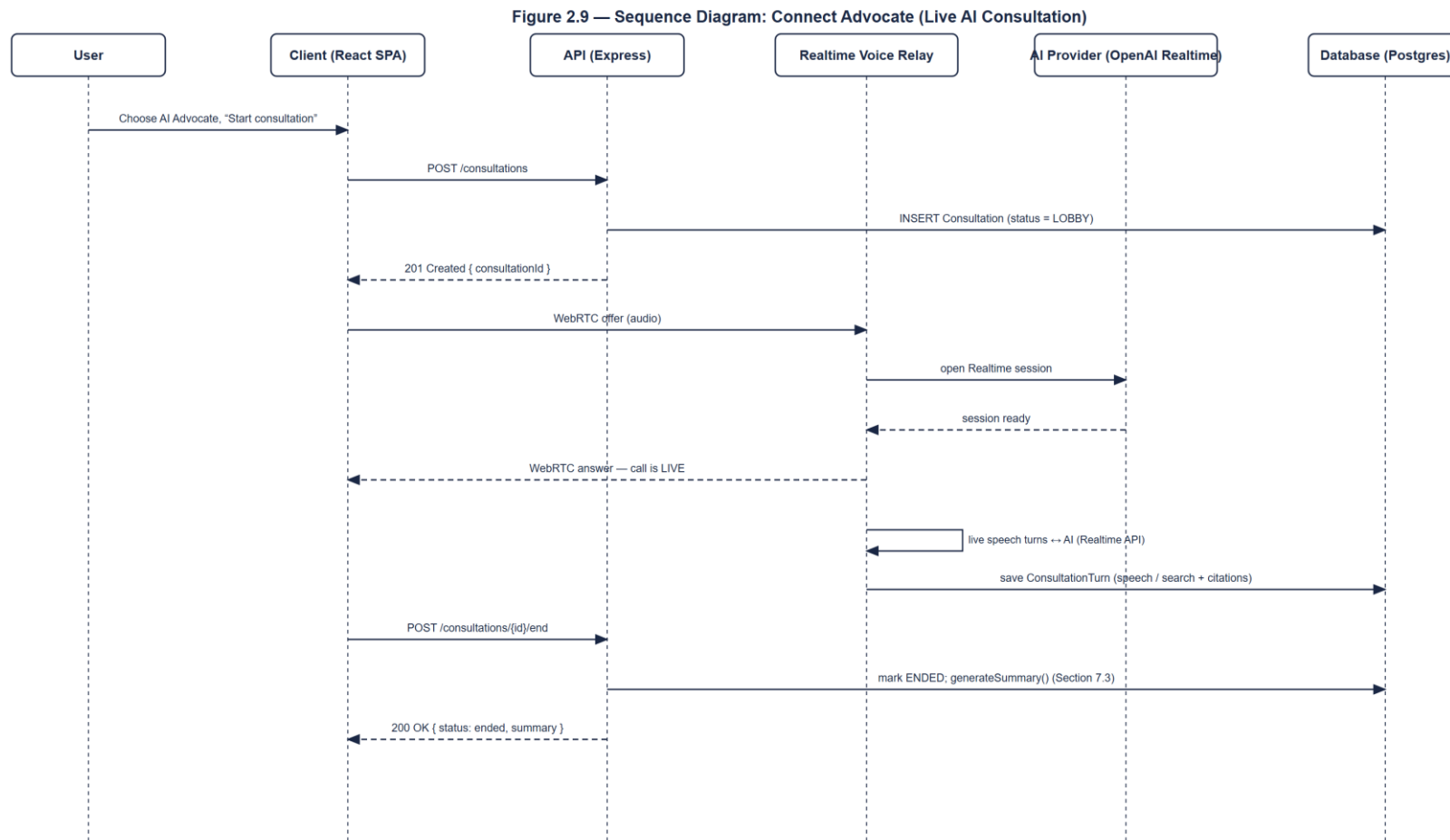


Figure 2.9 — Sequence Diagram: Connect Advocate (Live AI Consultation)

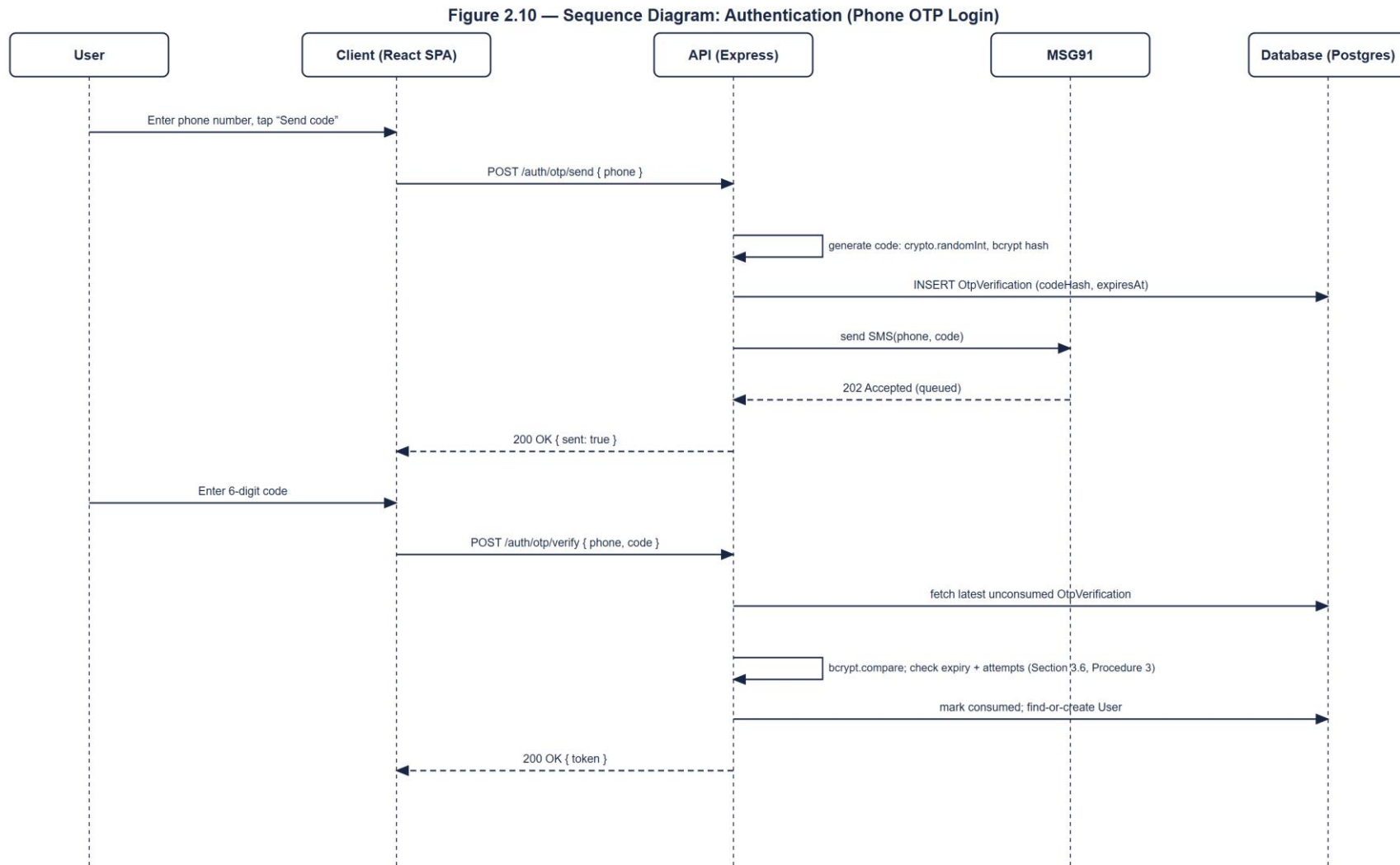
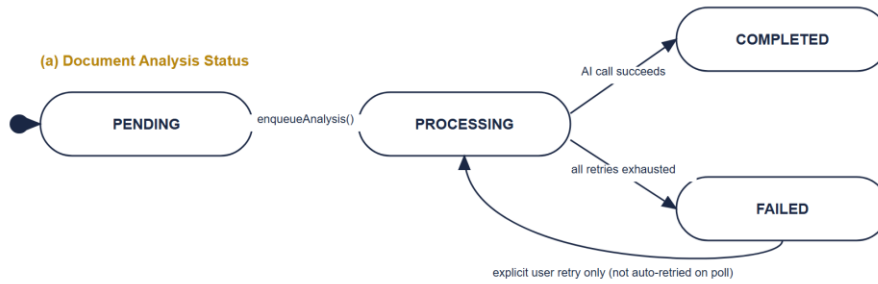


Figure 2.10 — Sequence Diagram: Authentication (Phone OTP Login)

2.5.5 State Transition Diagram

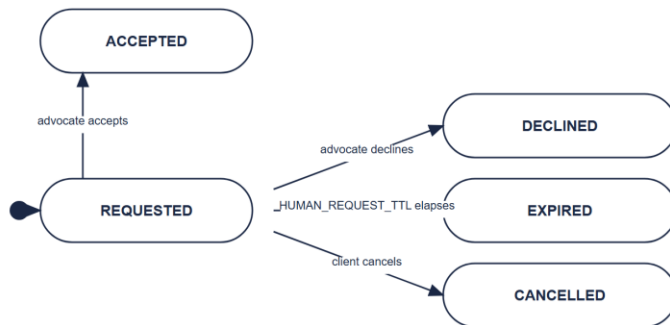
Two entities carry real, enforced state machines worth documenting explicitly: Document (Figure 2.11a), whose four-state lifecycle was exercised directly by the retry/failure system tests in Table 5.3 (TC-10, TC-11), and the human side of Consultation (Figure 2.11b), whose REQUESTED state can resolve four different ways depending on what the advocate — or the client, or the clock — does next. Both are derived directly from the status/enum fields in schema.prisma (Appendix A), not modelled independently of the implementation.

Figure 2.11 — State Transition Diagrams



Source: document.repository.ts / analysis.controller.ts — see Section 5.5, TC-10 / TC-11 for the system tests driving these transitions.

(b) Human Advocate Consultation Request



Source: ConsultationStatus enum, schema.prisma (Appendix A). The AI-advocate path (same entity) instead runs LOBBY → LIVE → ENDED / FAILED, shown implicitly in Figure 2.9.

Figure 2.11 — State Transition Diagrams

Consultation Status — Full Transition Reference

Figure 2.11b simplifies the human-advocate side for readability; the AI-advocate side shares the same ConsultationStatus enum (Table 2.4) but was left out of the diagram entirely since its states are fewer. Tables 2.5a and 2.5b list every transition actually implemented, read directly from consultation.service.ts, consultation.realtime.ts, and human.service.ts, rather than inferred from the enum's field order.

Table 2.5a — AI Advocate Path (LOBBY / LIVE / ENDED / FAILED)

From	To	Trigger
(new row)	LOBBY	Default value on creation (Consultation.status, Table 2.4) — createConsultation never sets it explicitly
LOBBY	LIVE	connect() atomically claims the lobby (updateMany WHERE status=LOBBY) — the same guard that stops two concurrent connect calls from both succeeding
LIVE	LOBBY	Realtime negotiation failed after the lobby was claimed — rolled back rather than left stuck LIVE
LOBBY / LIVE	FAILED	Starting a new call supersedes a stale open one; or endConsultation() is called while still in LOBBY
LIVE	ENDED	endConsultation(), an idle/time-limit timer, a closed WebSocket with no reconnect, or recoverStaleSessions() cleaning up a session orphaned by a server restart

Table 2.5b — Human Advocate Path (REQUESTED → ... → ENDED)

From	To	Trigger
(new row)	REQUESTED	requestConsultation() — explicit, not the schema default
REQUESTED	ACCEPTED	acceptRequest() — guarded: rejected if the row is no longer REQUESTED (prevents a double-accept race)
REQUESTED	DECLINED	declineRequest(), same guard
REQUESTED	CANCELLED	cancelRequest(), called by the requesting user before an advocate responds
REQUESTED	EXPIRED	The in-process sweeper (a 10-second setInterval, not a cron job) — no-response timeout
ACCEPTED	LIVE	markLive() — the client reports the call actually connected
ACCEPTED	CANCELLED	endCall(), reached if either party leaves before the call goes live
ACCEPTED	EXPIRED	The same sweeper — a no-show join timeout
LIVE	ENDED	endCall(), via either participant hanging up, the sweeper's time-limit, or its empty-room grace period

Every transition above is enforced by the same mechanism: repo.transition(id, fromStatuses[], data) performs an atomic updateMany scoped to WHERE status IN fromStatuses, and reports failure if the row had already moved on — the guard against a double-accept or double-cancel race is the database's own conditional update, not an application-level lock. DECLINED and EXPIRED are correctly terminal — no code transitions a consultation out of either state, by design, not by omission.

2.5.6 Activity Diagram

Figure 2.12 walks the single highest-value user journey — uploading a document through to seeing its risk report — as one flow, including both decision points that can end it early (a validation failure, or text that remains unusable even after the OCR fallback). It is deliberately the same journey already traced at the request/response level in Figure 2.7; the activity diagram shows the same logic from the perspective of what happens, step by step, rather than which component talks to which.

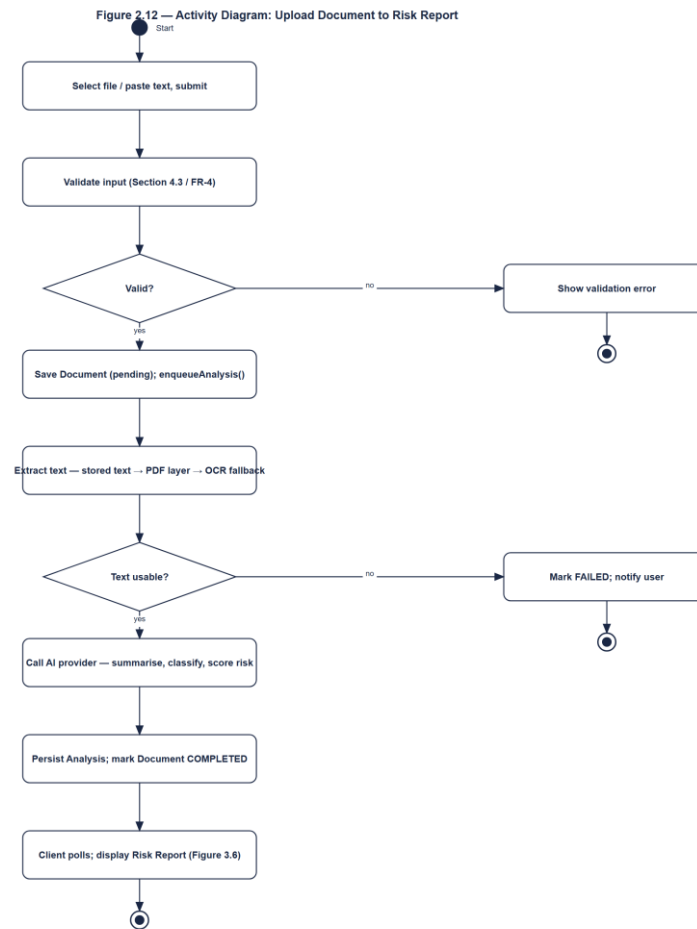


Figure 2.12 — Activity Diagram: Upload Document to Risk Report

2.5.7 Data Dictionary

Table 2.4 and Table 2.4a together list the key fields of all thirteen entities; the full, literal field list (with every Prisma attribute and relation) is given in the schema reproduced in Appendix A. The two tables were generated by reading that same `schema.prisma` directly rather than written from memory, which is also how an inconsistency between an earlier draft of this table and the actual schema — User/Document primary keys are plain auto-incrementing integers, not CUIDs, and Document.status/Message.role/Consultation.state are plain, convention-typed strings rather than Prisma enums — was caught and corrected here rather than left standing.

Table 2.4 — Data Dictionary (core entities)

Entity.Field	Type	Constraint	Description
User.id	Int	Primary Key, auto-increment	Unique identifier for the user
User.email	String	Unique, nullable	Email address, used for email/password and Google login
User.phone	String	Unique, nullable	Phone number, used for OTP login
User.password	String	Required (set to a bcrypt hash even for OAuth signup)	Never stored or logged in plaintext (Section 6.1)
User.role	Enum (UserRole)	Default: USER	USER ADMIN — gates admin-only endpoints
Document.id	Int	Primary Key, auto-increment	Unique identifier for the document
Document.userId	Int	Foreign Key → User, cascade delete	Owning user; enforces per-user data isolation
Document.status	String	Default: "pending"	App-level values: pending processing completed failed (not a DB enum — Section 3.3)
Document.filePath	String	Nullable	Absent for pasted-text documents
Analysis.documentId	Int	Foreign Key → Document, Unique	One analysis per document (cached, not re-run)
Analysis.riskScore	Int	Nullable, 0–100	Deterministic, server-computed risk score
Analysis.riskLevel	String	Derived from riskScore	Plain-string risk level, not a DB enum
Conversation.userId	Int	Foreign Key → User, cascade delete	Owning user of the chat conversation

Entity.Field	Type	Constraint	Description
Message.role	String	—	App-level values such as user/assistant (not a DB enum)
Message.citations	Json	Nullable	Verified Act/section citations backing an assistant reply
Advocate.kind	Enum (AdvocateKind)	—	AI HUMAN
Advocate.verificationStatus	Enum (VerificationStatus)	Default: UNVERIFIED	UNVERIFIED VERIFIED — gates whether a human advocate can accept consultations
Consultation.status	Enum (ConsultationStatus)	Default: LOBBY	LOBBY LIVE ENDED FAILED REQUESTED ACCEPTED DECLINED EXPIRED CANCELLED
LegalKnowledgeChunk.embedding	Unsupported("vector(1536)")	pgvector	Embedding used for similarity search (RAG retrieval)
LegalKnowledgeChunk.actShort	String	Nullable	Short name of the source Act, used in citations

Table 2.4a — Data Dictionary (remaining entities)

Entity.Field	Type	Constraint	Description
OtpVerification.phone	String	Indexed	Phone number the code was issued to (not unique — a number can request more than one OTP over time)
OtpVerification.codeHash	String	Required	bcrypt hash of the six-digit code; the plaintext code is never stored
OtpVerification.expiresAt	DateTime	Required	End of the code's validity window (Section 6.1)
OtpVerification.attempts	Int	Default: 0	Wrong-entry counter backing the attempt cap (Table 2.2a)
OtpVerification.consumed	Boolean	Default: false	Set true on first successful use — enforces single-use
ConversationDocument.conversationId / documentId	Int / Int	Composite unique, both cascade delete	Join row linking a document into a chat conversation's context
AdvocateCredential.advocateId	Int	Foreign Key → Advocate, cascade delete, indexed	Owning advocate profile
AdvocateCredential.type	Enum (CredentialType)	—	ENROLMENT DEGREE CERTIFICATION

Entity.Field	Type	Constraint	Description
			BAR_MEMBERSHIP (human) or KNOWLEDGE_SOURCE SCOPE LAST_VERIFIED (AI) — never a fabricated credential
AdvocateCredential.verified	Boolean	Default: false	Whether an admin has checked this credential (Section 6.2)
AdvocateAiConfig.advocateId	Int	Foreign Key → Advocate, Unique, cascade delete	One AI configuration per AI advocate profile
AdvocateAiConfig.temperature	Float	Default: 0.8	Sampling temperature passed to the Realtime voice API
AdvocateAiConfig.maxSessionMinutes	Int	Default: 20	Server-enforced cap on a single AI consultation's length
AdvocateAiConfig.version	Int	Default: 1	Bumped on every config change; a live call snapshots the version it started with (Consultation.aiConfigVersion)
ConsultationTurn.consultationId	Int	Foreign Key → Consultation, cascade delete, indexed	Owning consultation
ConsultationTurn.speaker	Enum (TurnSpeaker)	—	USER ADVOCATE SYSTEM
ConsultationTurn.kind	String	Default: "SPEECH"	SPEECH (transcribed talk) SEARCH (a law lookup the advocate ran) NOTICE (system message)
ConsultationTurn.citations	Json	Nullable	For a SPEECH turn: every Act/section the advocate mentioned, each with a verification status (Section 7.3); for a SEARCH turn: the retrieved passages

3. System Design

3.1 System Architecture

Figure 3.1 shows the system organised into four layers. The presentation layer is a single-page React application that talks to the backend over three distinct channels depending on what the interaction needs: ordinary REST/JSON requests for CRUD-style operations, Server-Sent Events for the streamed legal-assistant reply (so the client can render the answer token-by-token rather than waiting for the full response), and WebRTC for the audio/video path in Connect Advocate, signalled through a thin relay rather than proxying media through the application server itself.

The application layer deliberately separates the synchronous request path (the API) from the asynchronous one (the Analysis Worker): the API's job is to validate the request, persist or enqueue it, and respond immediately — it never itself calls the AI provider for document analysis. The Analysis Worker, a separate BullMQ consumer process, is what actually performs OCR fallback and the AI call, so that a slow or momentarily failing AI provider cannot make the HTTP API itself slow or unavailable. This split is the direct architectural consequence of the background-queue design covered in Figure 2.7 (Sequence Diagram: Document Analysis) in the previous section, and is what NFR "Performance" in Section 2.3.2 is referring to.

The data and queue layer uses PostgreSQL (with the pgvector extension for embedding similarity search) as the single system of record, and Redis for two distinct, namespaced purposes: as the BullMQ job broker, and as the backing store for the per-user rate limiters — the two uses are kept on separate key prefixes so an analysis-queue depth and a rate-limit counter can never collide. The external-services layer isolates the two third-party dependencies the system cannot function without — the OpenAI API (chat completion, embeddings, and the Realtime voice API) and the MSG91 SMS gateway (OTP delivery) — behind the application layer, so neither third-party outage nor API-shape change can reach the client directly; every external call is made from the server, never from the browser.

Figure 3.1 — System Architecture

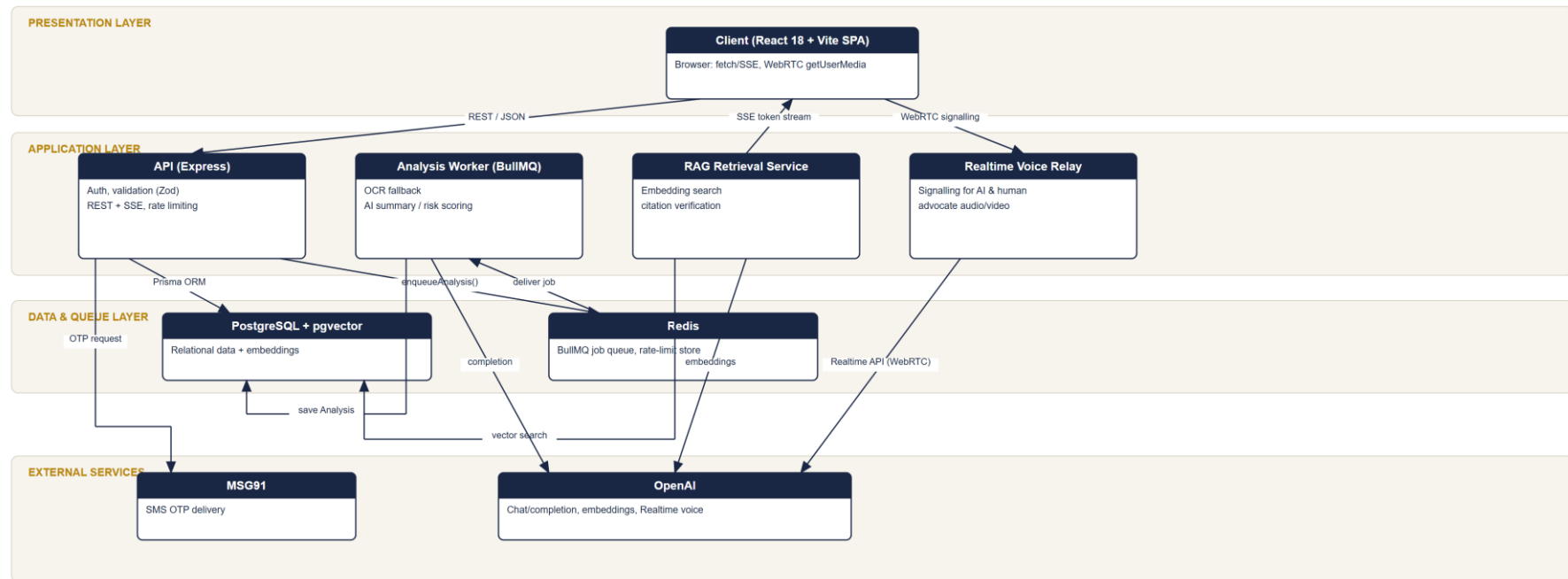


Figure 3.1 — System Architecture

3.2 Module Decomposition

The system is decomposed into seven feature-oriented modules (Figure 3.2), each implemented as its own directory of controller, service, and schema files under `server/src/modules` — a structure chosen so that a module's HTTP boundary (controller), business logic (service), and input-validation rules (schema) are co-located and can be understood, tested, and modified as a unit, rather than being scattered across type-based folders (all controllers together, all services together) that would force a developer to jump between unrelated features to understand one of them.

Auth & User, Document, and Analysis are the three modules that implement the core, originally-scoped document risk analysis feature; Legal Assistant and Connect Advocate are the two modules added once that core was stable, extending the same system to conversational and live-consultation legal help. Admin is deliberately small and narrow — it exists only to let a verified administrator manage advocate accounts, not as a general back-office. Platform / Shared is not a feature at all but the cross-cutting concerns every other module depends on: the Zod validation middleware, the structured Pino logger, the Helmet security headers, and the OpenAPI documentation generator — implemented once, centrally, and reused rather than duplicated per module.

Each module's process logic, briefly, is as follows.

- Auth & User — registers and authenticates a user through three interchangeable paths (email/password, phone OTP, Google OAuth) that all converge on the same outcome: a signed JWT and a User row. The module owns password hashing, token issuance, and the OAuth callback exchange, and is the only module that ever reads or writes `User.password`.
- Document — accepts a file upload or pasted text, decides which extraction path applies (stored text, PDF text layer, or OCR, per Figure 2.5), and owns the Document row's lifecycle fields (status, favourite, soft metadata). It never calls the AI provider itself — that is deliberately Analysis's job, not Document's — so a slow AI call can never block a simple rename or delete.
- Analysis — the background-job half of the system (Figure 2.7): dequeues a job, extracts text, calls the AI provider, derives the risk score deterministically (Section 3.3), and persists the result. Idempotent by design — a second run for an already-analysed document is a cache read, not a re-computation.
- Legal Assistant — owns the RAG retrieval + streaming chat loop (Figure 2.8): embeds the query, searches `LegalKnowledgeChunk`, builds a grounded prompt, streams the reply over SSE, and verifies every citation against what was actually retrieved before the message is persisted.
- Connect Advocate — the live-channel module, split into three sub-concerns that share one Consultation entity: the AI voice path (WebRTC + OpenAI Realtime, Figure 2.9), the human-request path (Figure 2.11b's state machine), and the advocate desk (accept/decline/notes) a verified human advocate uses to run their side of a call.
- Admin — narrow by design: advocate verification, credential management, and AI-config updates, all gated by `requireAdmin` (Section 6.2). It has no user-management, analytics, or billing surface, because none of those exist yet (Section 8).

- Platform / Shared — the cross-cutting concerns described above, plus the OTP and Speech modules, which are shared infrastructure (phone verification, audio transcription) rather than being owned by any single feature module.

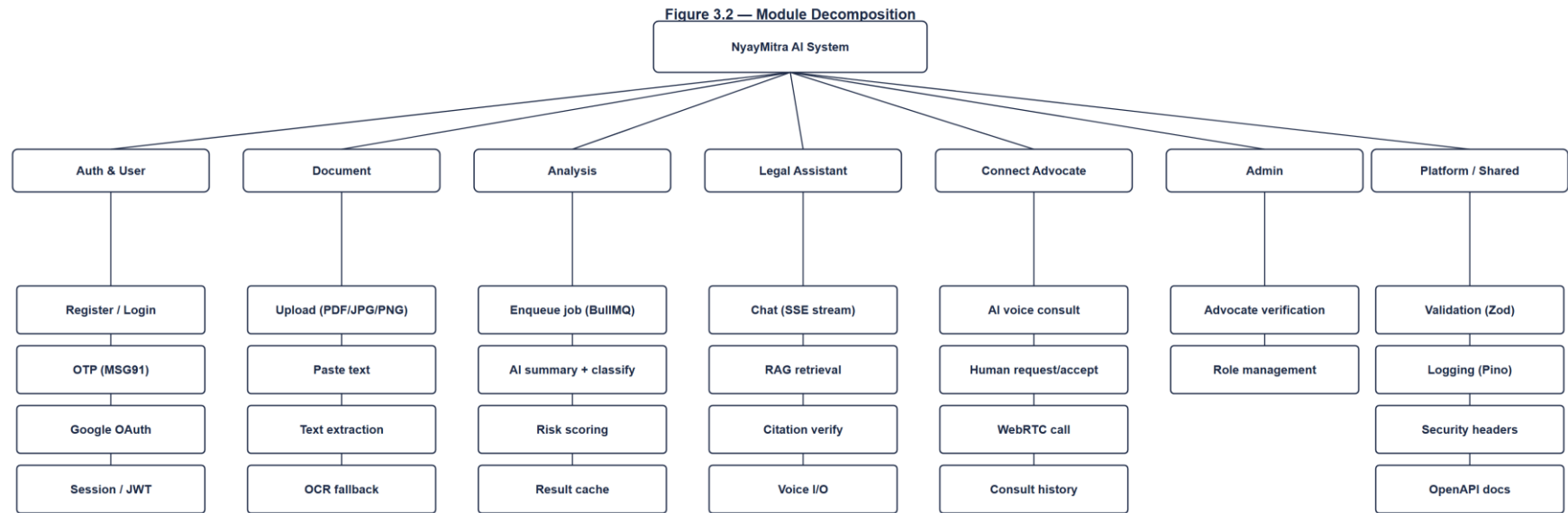


Figure 3.2 — Module Decomposition

3.2.1 Client-Side Architecture

Figure 3.2 and Section 3.2 describe the server's technical-layer decomposition (routes → middleware → controller → service → repository, Figure 3.9). The client is organised differently, by deliberate choice: a feature-sliced structure under client/src, where each domain owns its own components, API calls, and hooks together, rather than being split across generic "components", "services", and "state" folders.

Folder	Purpose
app/	App-wide setup — the Redux store and the theme context provider
components/	Shared, generic UI with no feature-specific logic — layout chrome (Sidebar, PageIntro) and common dialogs (FormError, ConfirmDialog, RenameDialog)
features/<domain>/	One folder per domain (auth, document, dashboard, legal-agent, consultation, human, admin) — each bundles its own components, a thin *Api.ts wrapper, and custom hooks together
pages/	Route-level screen components (Home, Login, Dashboard, LegalAssistant, ...), including pages/admin/ for admin screens
routes/	Route-guard wrapper components — ProtectedRoute, AdminRoute, AdvocateRoute
services/	Cross-cutting API infrastructure — the shared axios instance (api.ts) and error normalisation (apiError.ts, Appendix F.4)

State management is deliberately mixed, not uniform: Redux Toolkit (configureStore, a single auth slice) holds session state consumed across unrelated parts of the tree (the Sidebar, every ProtectedRoute check), while every other feature's data — documents, chats, dashboard counts, a live call — is local state inside a custom hook (useDashboardData, useLegalChat, useWebRtcCall) rather than pushed into the global store. No React Query/TanStack Query or Zustand is used; React Context is reserved for one cross-cutting UI concern that genuinely has nothing to do with server data — the light/dark theme, persisted to localStorage and synced to the prefers-color-scheme media query on first load.

Routing uses react-router-dom with a single, central route table (App.tsx) rather than per-feature route files or lazy-loaded route chunks — every page component is a plain top-level import. Access control is composed, not centralised in one generic guard: ProtectedRoute, AdminRoute, and AdvocateRoute nest around a route element (e.g. ProtectedRoute > AdvocateRoute > AdvocateDeskPage), and AdvocateRoute's own comment states plainly that it is a client-side convenience only — the server re-checks every authorisation decision independently (Section 6.2), consistent with this report's repeated point that the client is never trusted as a security boundary.

Every feature's *Api.ts wrapper calls through one shared axios instance rather than constructing requests ad hoc per screen, specifically — per that file's own comment — "so every screen performs these actions identically." A response interceptor on that same instance is what actually detects a session-expiry (Appendix F.4) and raises it as a plain DOM CustomEvent, which App.tsx listens for and turns into a Redux action — a deliberate decoupling point, since the axios layer has no access to React context or the store directly. One gap is worth naming rather than silently leaving implicit: no ErrorBoundary component exists anywhere in the client, so an uncaught render-time exception in any one screen is not currently contained to that screen — a concrete, scoped candidate for Future Scope rather than the broader items already listed in Section 8.

Table 3.2 — Custom Hooks Reference (Client)

Hook	Purpose
useDashboardData	Fetches the caller's documents and chats in parallel for the Dashboard (Figure 3.5), exposing loading/failed state plus update/remove helpers so a list can be patched in place after an action
useDocumentAnalysis	Polls POST /analysis/run every 2 seconds (Figure 2.7) until the result or a failure is ready — a deliberate client-side poll against a background job, not a blocking wait on the original request
useDocumentActions	Wraps rename/favourite/delete so every screen that can act on a document does so identically, surfacing the same friendly error (Appendix F.4) on failure
useLegalChat	Owns conversation list/active-conversation/message state and reads the token-by-token SSE stream (Figure 2.8) into the Legal Assistant screen (Figure 3.7)
useConsultationCall	Drives the AI voice consultation's call-phase state machine on the client (idle → requesting-mic → connecting → live → ended/error) and turns a getUserMedia failure into a specific, actionable message rather than a generic permission error
useLocalCamera	Manages the optional camera stream for a consultation — explicitly optional, with its own error copy noting the call continues fine without it
useHub	One realtime connection for the page's lifetime (human-advocate presence/signalling), created only once a session token exists and torn down on unmount
useWebRtcCall	Drives the human-advocate call's own phase state machine (idle → starting → waiting-peer → connecting → live → ended/error), parameterised by role (user vs advocate) since both sides of the call share this hook
useAdvocateAccount	Reads whether the signed-in account is linked to an advocate profile, backing the client-side AdvocateRoute convenience check (Section 3.2.1)

3.3 Database Design

The physical schema (implemented in Prisma ORM and introduced as the Entity-Relationship model in Figure 2.2) follows normalised, third-normal-form design for the relational tables, with two deliberate exceptions made for read performance rather than by oversight. First, `Analysis.riskScore` is stored as a derived column rather than computed on every read — the AI provider returns only a three-level `riskLevel` (Low/Medium/High, never a number), and `riskScore` is written once, deterministically, from that `riskLevel` at analysis time via a fixed lookup table (`riskScoreForLevel`: Low→30, Medium→60, High→90, Section 4.2) — so that sorting or filtering a user's documents by numeric score does not need to recompute a level-to-score mapping on every request. Second, `Analysis` itself is a cache, not a transient result: `Analysis.documentId` is unique, enforcing at most one analysis per document, and a repeat request for an already-analysed document is served from this stored row rather than re-invoking the AI provider — a deliberate cost and latency decision, not an accident of the schema.

Every foreign key that scopes a row to its owning user (`Document.userId`, `Conversation.userId`, `Consultation.userId`, and equivalently `Analysis` through its `Document`) is indexed, because every one of them is used on the hot path of the per-user data-isolation check described as FR-3 in Section 2.3.1 — the server's authorisation middleware filters by exactly these columns on every request, so an unindexed foreign key here would have turned a security control into a full table scan. `LegalKnowledgeChunk.embedding` carries a pgvector ANN (approximate-nearest-neighbour) index, since the Legal Assistant's retrieval step (Figure 2.8) runs a similarity search against this column on every chat turn; without the index, retrieval latency would grow linearly with the size of the ingested legal-knowledge corpus instead of staying near-constant.

Schema evolution is managed through Prisma Migrate, with each migration checked into the repository and applied in order — the schema itself grew incrementally alongside the phases in Table 2.1 (notably, the Phase D migration that introduced user profiles, analysis caching, and the structured risk-data columns described above), rather than being designed complete and final before any code was written, consistent with the incremental paradigm described in Section 2.4.

3.4 Data Integrity and Constraints

Referential integrity is enforced at the database level by Postgres foreign-key constraints (the real generated SQL for two of them is shown in Section 4.3), and the ON DELETE behaviour for each was a deliberate per-relationship choice, not the Prisma default left unexamined. Two patterns cover the whole schema: CASCADE for data genuinely owned by its parent — deleting a `Document` deletes its `Analysis`, deleting a `User` deletes their `Documents`, `Conversations`, and `Consultations`, since none of that data has meaning without its owner; and SET NULL for optional or historical references — deleting an `Advocate` sets `Consultation.advocateId` to null rather than deleting the consultation itself, because the advocate's name and kind are snapshotted onto the `Consultation` row precisely so a client's consultation history survives the advocate account being removed.

Value-level integrity is enforced the same way, not left to application code alone: every fixed-vocabulary field — `UserRole`, `AdvocateKind`, `AdvocateStatus`, `VerificationStatus`, `CredentialType`, `ConsultationStatus`, `TurnSpeaker`, `KnowledgeSourceType` — is a native Postgres enum type (Appendix A), so a direct SQL statement attempting to write `Consultation.status = 'bogus'` is rejected by the database itself, independent of whether the application's own validation layer (Section 6.3) is ever bypassed.

Table 3.1 — Key Uniqueness Constraints

Entity	Constraint	Enforces
User	UNIQUE(email), UNIQUE(phone)	One account per email and per phone number
Analysis	UNIQUE(documentId)	At most one analysis per document (Section 3.3 — cache, not re-run)
Advocate	UNIQUE(slug), UNIQUE(userId)	A stable public URL; one advocate profile per login account
AdvocateAiConfig	UNIQUE(advocateId)	Exactly one AI configuration per advocate
LegalKnowledgeChunk	UNIQUE(contentHash)	Re-ingesting the same statute text does not duplicate it

The relational tables are normalised to third normal form: no table stores a value that is derivable from another table's own key rather than from its row's identity (Document does not duplicate User.email; Analysis does not duplicate Document.title), and repeating groups are pulled into their own child tables rather than repeated as columns — AdvocateCredential exists specifically so an advocate can have an arbitrary number of credentials without widening the Advocate table once per credential type. The one intentional departure from strict normal form — Analysis.riskScore stored as a derived column — is disclosed and justified in Section 3.3, not introduced silently here.

3.5 User Interface Design

The interface follows a single consistent design language across every screen: a dark-navy fixed sidebar for primary navigation, the same navy/gold/cream colour pair used throughout this report's own diagrams, plain-language copy instead of technical or HTTP-status-coded messages (a direct product decision, not a gap in error handling — see the Friendly-Errors convention used throughout the client), and full bilingual support (English and Hindi) in both the static interface text and the Legal Assistant's own replies. The six screens below illustrate the main user-facing surfaces; each is a real screenshot of the running application, not a mock-up.

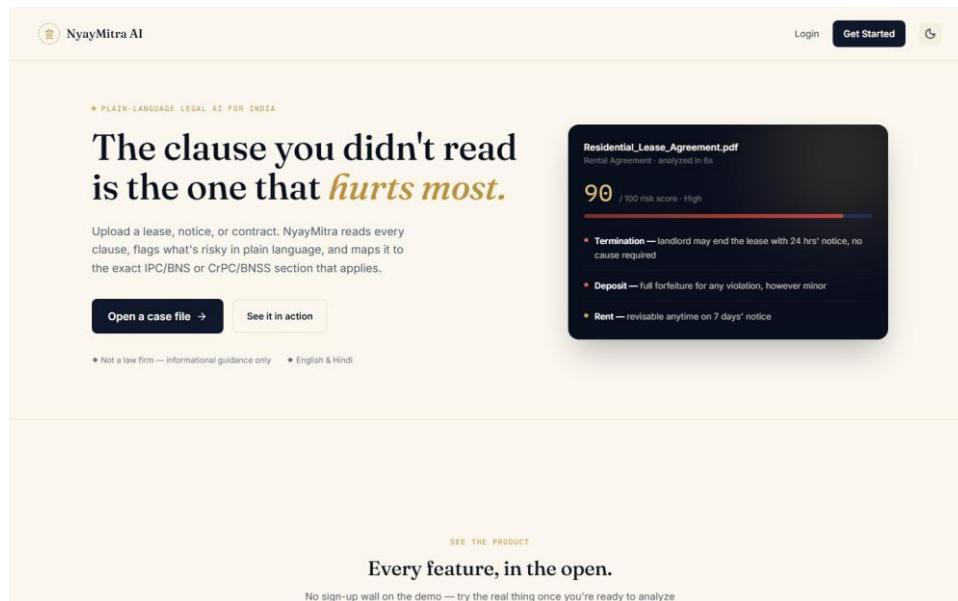


Figure 3.3 — Screen: Home (Landing Page)

The landing page leads with the product's actual core output — a real risk-score card — rather than generic marketing copy, and states the AI/not-a-law-firm boundary from Section 2.1 above the fold rather than in a footnote.

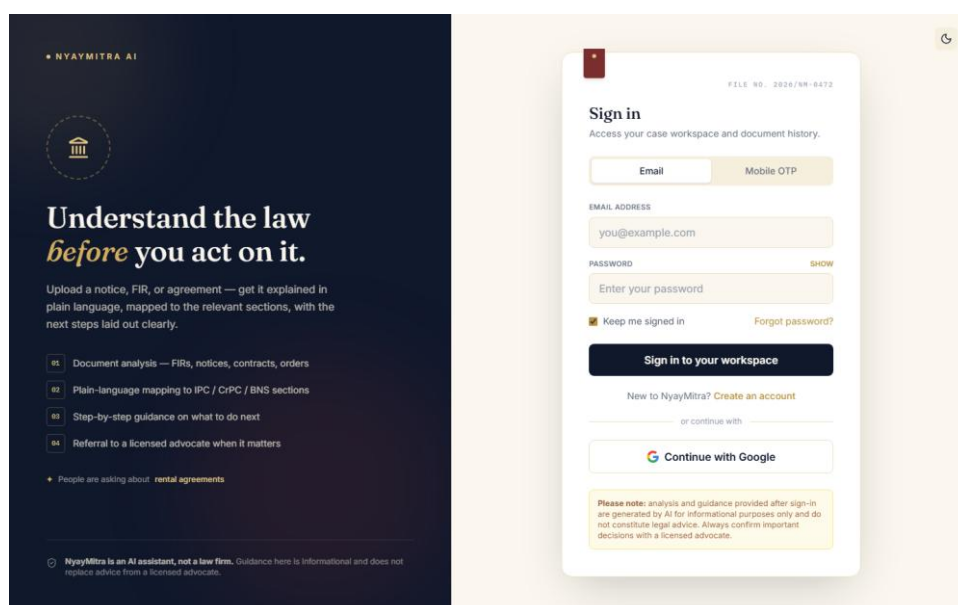


Figure 3.4 — Screen: Login / Register

Authentication offers all three paths from FR-1/FR-2 on one screen — email/password, phone OTP, and Google OAuth — rather than forcing a choice before the user can see what is available, with the same AI-generated-information disclaimer repeated at the point of sign-in.

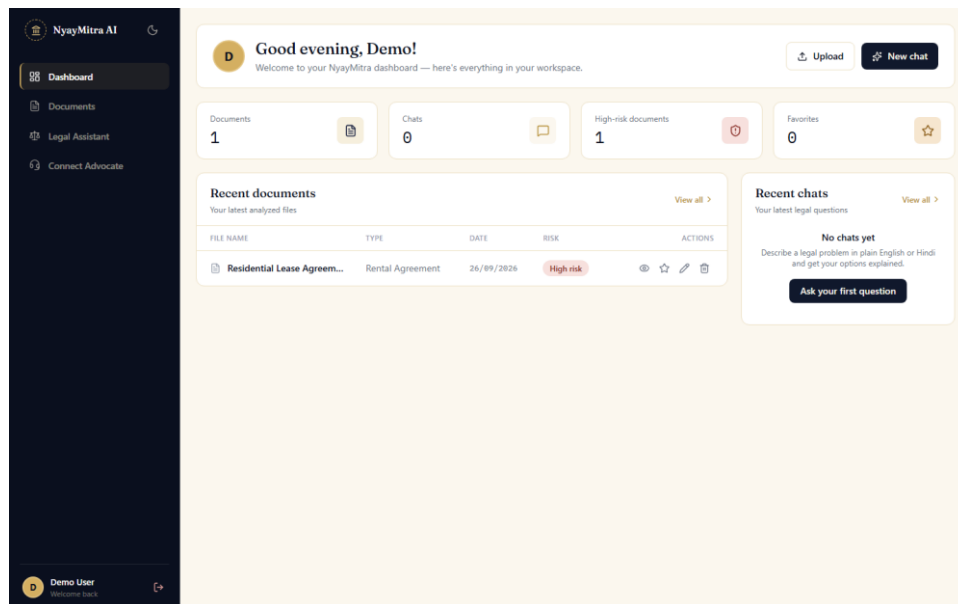


Figure 3.5 — Screen: Dashboard

The dashboard is the authenticated landing screen: document and chat counts, a highlighted count of high-risk documents specifically (surfacing the system's core value — risk, not just storage — at a glance), and recent-activity lists for both documents and chats, each with a direct “View all” escape hatch to its full listing screen.

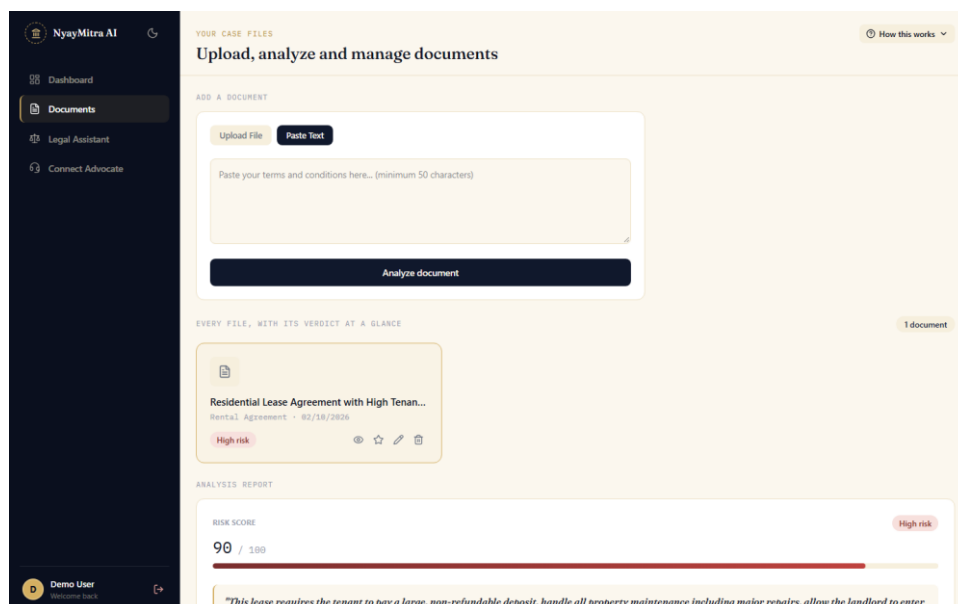


Figure 3.6 — Screen: Document Risk Report

The risk report is the output of the Document Analysis flow traced in Figure 2.7: a quantified risk score, the plain-language summary, and the structured, per-clause breakdown (FR-6) are all rendered from the same Analysis row the worker wrote — nothing on this screen is computed client-side, which is what keeps the number shown here consistent with whatever the server would return to any other client reading the same document.

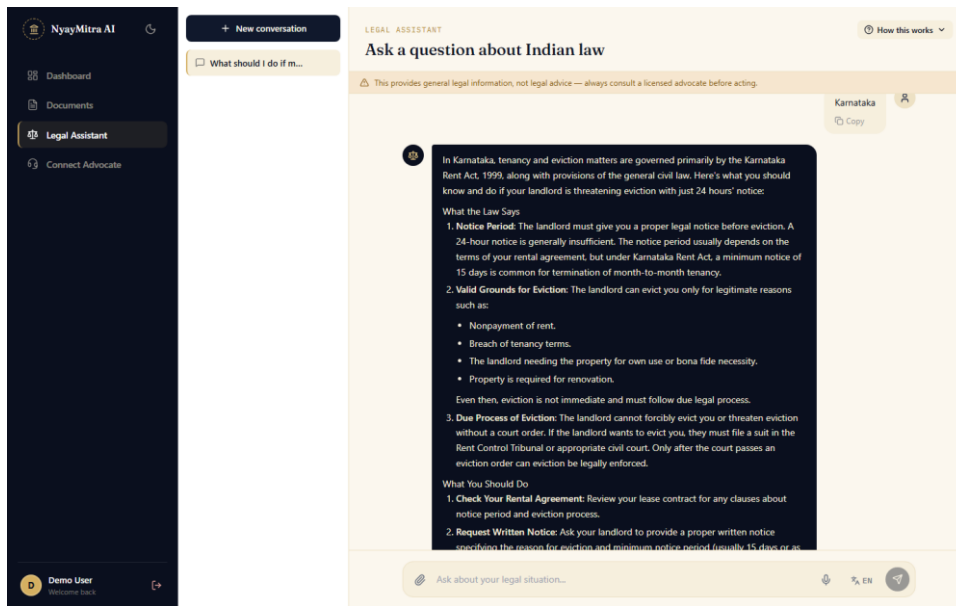


Figure 3.7 — Screen: Legal Assistant

The Legal Assistant screen renders the streamed reply described in Figure 2.8 incrementally as tokens arrive over SSE, with each citation rendered as a distinct, inspectable reference back to the source Act and section rather than as plain inline text — a deliberate UI decision to keep “where this came from” visually separate from “what the model said”.

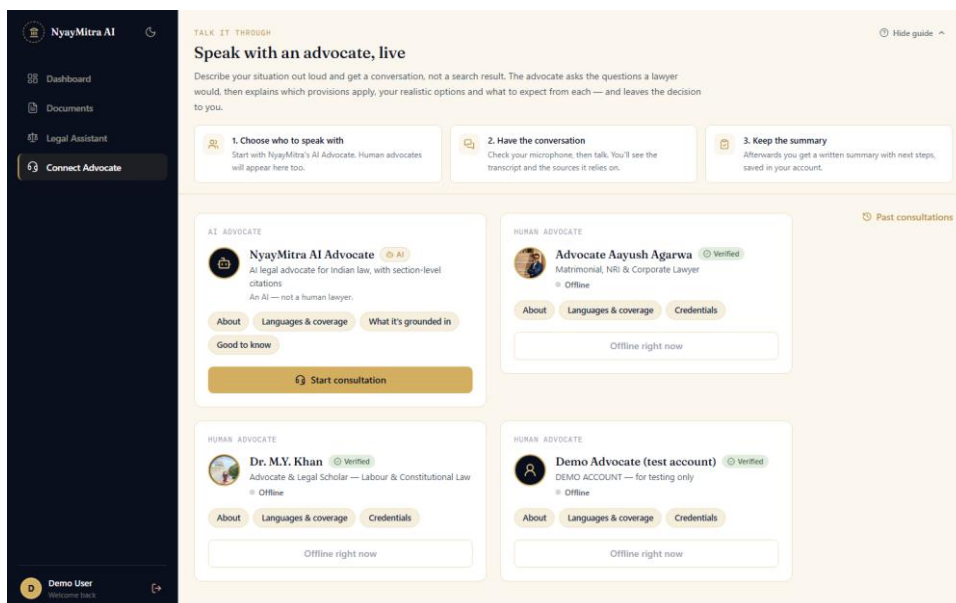


Figure 3.8 — Screen: Connect Advocate

Connect Advocate surfaces both escalation paths side by side — an immediate AI voice consultation (the live call traced in Figure 2.9) and a request to a verified human advocate — rather than hiding the human option behind the AI one, consistent with this report's own framing in Section 2.1 that the AI paths are meant to sit alongside, not replace, a qualified advocate for anything consequential.

3.5.1 UI Design Principles and Consistency

Four conventions are enforced across every screen above, not just the ones they happen to be most visible on:

- Friendly errors, never raw HTTP text: `apiErrorMessage()` maps every status to a fixed, human message (Appendix F.4), shown inline on a form via a shared `FormError` component, or as a toast for a quick, non-form action — the same rule applied consistently rather than left to whichever screen a bug happened to surface on.
- Bilingual by construction, not by translation layer bolted on after: the language field threads through the same request/response schemas as everything else (`createConversationBodySchema`, Section F.3 pattern), so an unsupported language code is a 400, not a silent fallback that masks a typo.
- One colour vocabulary for risk, everywhere: the same Low/Medium/High palette (Section 7.1.1) is used for a document's own badge, its clause-level risk items, and the Dashboard's high-risk count — a clause flagged High never reads as a different colour than the document-level badge showing the same word.
- No client-side computation of anything the server already computed: the risk score, risk level, and citation-verification status shown on screen are always the exact values the server returned (Figure 3.6's own description above), not re-derived in the browser — so two different clients reading the same `Analysis` or `ConsultationSummary` row can never disagree.

3.6 Procedural Design

The three procedures below give the step-by-step logic behind the system's most consequential flows, at the level of pseudocode rather than the actual TypeScript (which is reproduced separately in Section 4.2). Each corresponds directly to a sequence diagram already presented in Section 2.5.4.

Procedure 1 — `AnalyzeDocument(documentId)`: the background job behind Figure 2.7.

```
PROCEDURE AnalyzeDocument(documentId)
  doc ← FETCH Document WHERE id = documentId
  IF doc NOT FOUND OR doc.analysis EXISTS THEN
    RETURN // nothing to do – not an error
  END IF

  TRY
    text ← ExtractText(doc) // stored text → PDF text layer → OCR fallback
  CATCH extractionError
    RAISE UnrecoverableError // never worth retrying
  END TRY

  IF LENGTH(text) < MIN_USABLE_TEXT_LENGTH THEN
    RAISE UnrecoverableError
  END IF

  result ← CALL AiProvider(text) // summary, riskLevel, clauses – may throw (retryable)
  riskScore ← RiskScoreForLevel(result.riskLevel)

  SAVE Analysis(documentId, result.summary, result.riskLevel, riskScore, result.clauses)
  MARK doc AS analyzed(result.title, result.documentType)
  RETURN Analysis
```

Procedure 2 — `AskLegalQuestion(query, language)`: the RAG flow behind Figure 2.8.

```
PROCEDURE AskLegalQuestion(query, language)
  embedding ← Embed(query)
```

```

chunks ← VectorSearch(embedding, topK = 4)           // pgvector similarity search
context ← FormatAsCitableContext(chunks)

reply ← STREAM CompletionFromAiProvider(query, context, language)
                                         // token-by-token, over SSE
citations ← ExtractCitations(reply)

FOR EACH citation IN citations DO
  IF citation.act/section NOT IN chunks' sources THEN
    MARK citation AS unverified           // Section 6 – never silently trusted
  END IF
END FOR
citations ← Dedupe(citations)             // cap at 5, drop repeats (Section 4.2)

reply ← AppendDisclaimer(reply)
SAVE Message(conversationId, role = "assistant", reply, citations)
RETURN reply                             // already streamed to the client

```

Procedure 3 — VerifyOtp(phone, submittedCode): the single-use, rate-limited check behind FR-2 / Figure 2.10 / Section 6.1.

```

PROCEDURE VerifyOtp(phone, submittedCode)
  record ← FETCH latest OtpVerification WHERE phone = phone AND consumed = false
  IF record NOT FOUND THEN
    RETURN 401 "invalid code"
  END IF
  IF NOW() > record.expiresAt THEN
    RETURN 401 "code expired"
  END IF
  IF record.attempts >= MAX_ATTEMPTS THEN
    RETURN 429 "too many attempts"
  END IF

  IF NOT BcryptCompare(submittedCode, record.codeHash) THEN
    INCREMENT record.attempts
    RETURN 401 "invalid code"
  END IF

  MARK record AS consumed                // single-use – Table 6.1
  user ← FindOrCreateUserByPhone(phone)
  RETURN JWT(user)

```

Procedure 4 — VerifyCitations(advocateText, retrieved): the grounding check behind Figure 2.8/2.9's citation-verification step.

Not fuzzy or AI-based matching — a plain regex extraction followed by an exact key lookup, deliberately, so "verified" is a reproducible fact about what was retrieved, not another AI judgement call stacked on top of the first one.

```

PROCEDURE VerifyCitations(advocateText, retrieved)
  refs ← ExtractSectionRefs(advocateText) // regex match "Section N of <Act>" and "<Act>, s.N",
                                         // longest-alias-first so "BNSS" wins over
  "BNS";                                // normalises "103(1)" -> "103", dedupes by
actShort+section
  retrievedKeys ← SET of (chunk.actShort + "|" + chunk.section) FOR chunk IN retrieved

  results ← []
  FOR EACH ref IN refs
    IF ref.ingested = false THEN
      status ← "unverified_act_not_loaded" // Act was never ingested at all (e.g.
repealed IPC/CrPC)
    ELSE IF (ref.actShort + "|" + ref.section) IN retrievedKeys THEN
      status ← "verified"
    ELSE
      status ← "unverified_not_retrieved" // ingested, but not surfaced to the model
this turn

```

```

    END IF
    APPEND { actShort: ref.actShort, section: ref.section, status, raw: ref.raw } TO results
  END FOR
  RETURN results

```

Called on every chunk of the AI advocate's live speech (`consultation.realtime.ts`) so an unverified citation can be fed back to the model to self-correct mid-call, and again on the full transcript when the Consultation Summary Report is generated (Section 7.3) — the same function, not a second, looser check.

Procedure 5 — ExtractDocumentText(doc): the stored-text / PDF-layer / OCR decision behind FR-5, Figure 2.5, and Figure 2.12.

```

PROCEDURE ExtractDocumentText(doc)
  IF doc.content IS NOT EMPTY THEN           // pasted/stored text – nothing to extract
    RETURN { text: doc.content, usedOcr: false }
  END IF
  IF doc.filePath IS EMPTY THEN
    RETURN { text: "", usedOcr: false }
  END IF

  IF Extension(doc.filePath) IN {png, jpg, jpeg} THEN
    text ← OcrImage(doc.filePath)           // tesseract.js directly – no PDF layer to
  try first
    RETURN { text, usedOcr: true }
  END IF

  pdfText ← ExtractTextFromPDF(doc.filePath) // pdf-parse
  IF Trim(pdfText).length >= MIN_USABLE_TEXT_LENGTH THEN // 50 characters
    RETURN { text: pdfText, usedOcr: false }
  END IF

  // Fallback: rasterize up to MAX_OCR_PAGES (5) pages and OCR each – a
  // deliberate cost/latency cap, the same pattern as the AI prompt's own
  // character cap (Section 3.6's AnalyzeDocument, Procedure 1).
  ocrText ← OcrPdf(doc.filePath, maxPages: 5)
  RETURN { text: (ocrText OR pdfText), usedOcr: true }

```

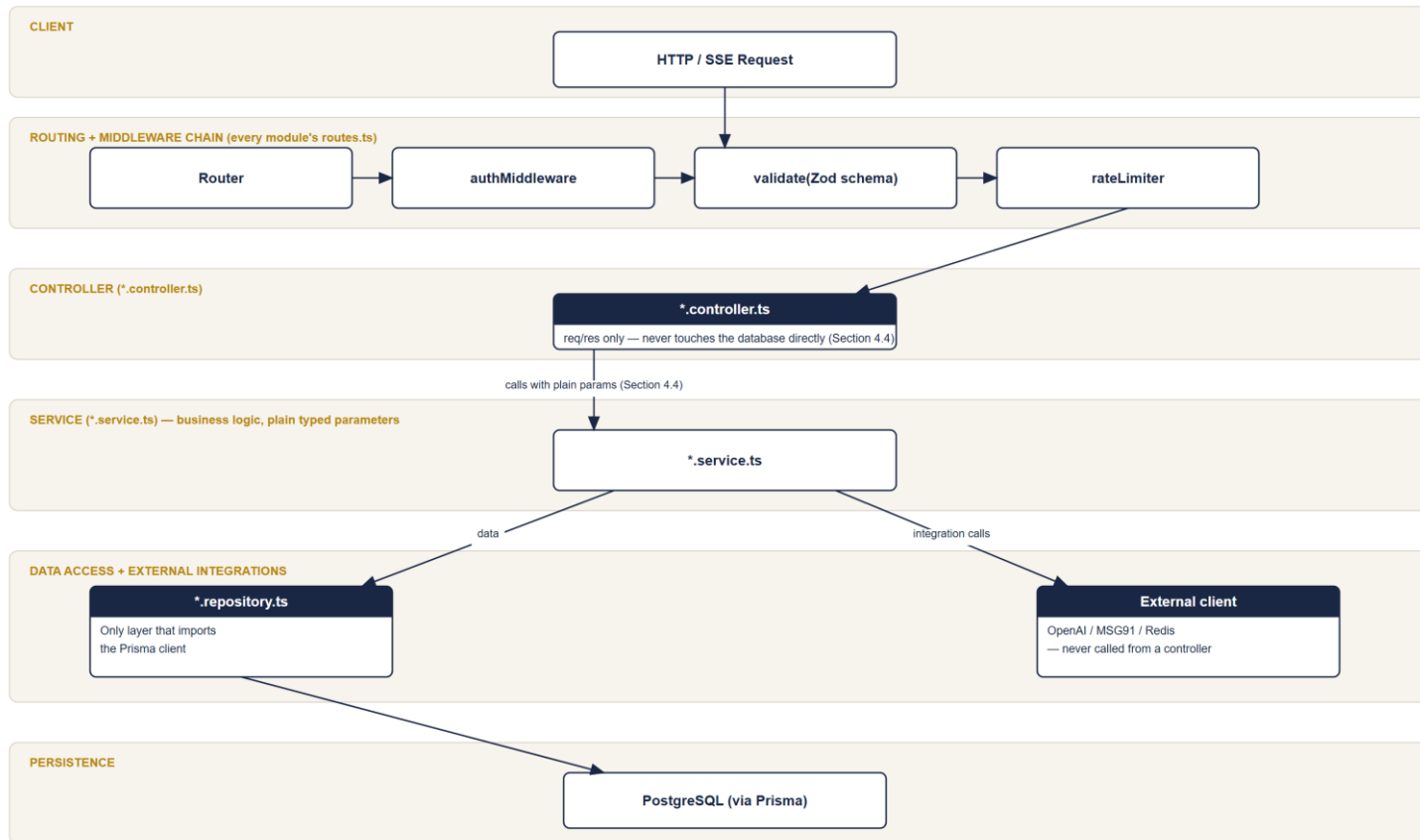
The worker (`analysis.worker.ts`) applies the same 50-character threshold a second time, after extraction returns: if the final text is still too short even after OCR, it throws an `UnrecoverableError` — skipping BullMQ's retry entirely, reasoned explicitly in the code's own comment as “a file we genuinely can't read ... will never succeed on retry — fail it once, not three times” — and the document is marked failed (Section 2.5.6's Activity Diagram, the “Mark FAILED” branch).

This function's existence is itself a fix for a real defect, not an original design: `uploadDocument` used to call `pdf-parse` unconditionally at upload time, which crashed on any non-PDF image upload (the `pdf-parse` crash defect in Table 5.4). The fix was to remove extraction from the upload path entirely and centralise it here, where it is extension-aware and runs only later, inside the worker's own try/catch guard — the same debugging discipline described in Section 5.7, applied to the one defect this report's procedural design section can show in full rather than only summarise in a table.

3.7 Component Diagram

Figure 3.9 makes explicit the one internal layering pattern every feature module in Figure 3.2 follows, shown once generically rather than drawn seven times identically. A request passes through routing and the middleware chain (Section 6.3) before it ever reaches a controller; the controller only ever touches req/res and calls into its service with plain parameters (Section 4.4); the service holds the actual business logic and is the only layer allowed to reach either the repository or an external integration client; and the repository is, deliberately, the only file in a module permitted to import the Prisma client at all — a rule that is not enforced by a linter, but is consistently true across the whole codebase, which is exactly what let Section 5.3's unit tests call service functions directly with no database connection in the first place.

Figure 3.9 — Component Diagram: Module Layering Pattern



The same five-layer pattern repeats identically across all seven feature modules (Figure 3.2) — shown here once, generically, rather than seven times.

Figure 3.9 — Component Diagram: Module Layering Pattern

3.8 Test Case Design

Designed at the same time as the modules themselves — a test case for a function was written against its specification (Section 2.3.1's FR, or the pseudocode in Section 3.6), before being relied on to prove the implementation correct, rather than reverse-engineered from the code afterwards. Table 5.3 (Section 5.5) lists the full, executed set with its Pass/Fail result; this section documents the design approach behind it, which Section 5 then reports the outcome of.

Unit Test Case Design

Unit test cases target one dependency-free function (Section 4.1) at a time, designed using two conventional black-box techniques applied consistently across the suite:

- Equivalence partitioning — grouping inputs into classes expected to behave identically, and writing one representative case per class rather than one per possible value: a phone number is either well-formed or not (TC-15), a pasted-text document is either at least 50 characters or not (Appendix Zod schema, Section 4.2), an OTP code either matches the stored hash or does not (Table 5.3).
- Boundary value analysis — testing directly at a partition's edge, where an off-by-one defect actually lives: `riskScoreForLevel`'s unit tests (Section 4.2) check the three exact defined inputs (Low/Medium/High) plus the undefined-input fallback, rather than only a single "happy path" value; the OTP attempt counter is tested at the resend-cooldown boundary (TC-04) rather than only well inside or well outside the window.

Each unit test case is designed against the function's own contract — its declared parameters and return shape — with no database, queue, or network dependency (Section 5.1), which is what makes Table 5.1's 155 server unit test cases able to run in milliseconds rather than seconds.

System Test Case Design

System test cases are designed around a complete request/response cycle through the real HTTP server (Section 5.1, Section 5.8's worked example) rather than a single function, using two further techniques:

- Scenario-based design — each case follows one realistic user action end to end (register, request an OTP, upload a document, run analysis), matching the flows already specified as Use Cases in Section 2.5.3.1, so a system test case and a Use Case Specification describe the same behaviour from two complementary angles: one as a user-facing contract, the other as an executable check of it.
- Negative/error-path design — for every positive case, a deliberate matching negative one: not just "a valid upload succeeds" but "an upload under the 50-character minimum is rejected with 400, before it reaches the service" (TC-07); not just "the correct OTP logs the user in" but "the same code rejected on reuse" (Table 5.3) and "a wrong code increments the attempt counter rather than being silently ignored".

Table 5.5 (Section 5.9) is the honest counterpart to this subsection: it states which of the seven feature modules this system-test-case design approach was actually applied to, and which five were verified manually instead — the same design techniques above, not extended to every module, named explicitly rather than left to be inferred from Table 5.1's totals.

4. Coding

This chapter shows representative extracts chosen to illustrate specific engineering decisions already discussed in Sections 2 and 3, not a listing of the full implementation. The complete, unabridged source for this project's four most substantial modules — Authentication/OTP, Document ingestion, the Analysis pipeline, and RAG grounding — is reproduced in full at the end of this report (“Complete Source Code — Core Modules”), as additional pages outside the 100–125 page project-documentation count, consistent with the MCSP-232 guideline's own framing of that page range as “(excluding coding)” (page 12). The remaining modules follow the identical layering pattern shown in Figure 3.9 and are documented through the Use Case Specifications, sequence diagrams, and procedural-design pseudocode already presented; the complete, runnable repository is additionally provided on the CD attached to this report's last page.

4.1 Coding Standards and Practices

- Language: TypeScript in strict mode on both client and server; ESLint and Prettier enforced on every file, not opt-in per module.
- Structure: one directory per feature module (Figure 3.2), each owning its own controller, service, schema, and repository file — never a type-based split (“all controllers together”) across the whole codebase.
- Naming: camelCase for functions and variables, PascalCase for React components and TypeScript types, and a consistent module.concern.ts file-naming pattern (e.g. analysis.worker.ts, analysis.schema.ts) so a file's role is clear from its name alone.
- Pure-function extraction: logic that needed independent unit testing — risk-score derivation, citation de-duplication — was deliberately kept as a small, dependency-free exported function rather than inlined, specifically so Section 5 (Testing) could call it directly without standing up a database or a queue.
- Version control: one Git branch per gap/phase (Table 2.1), merged only after manual verification against the running application — the workflow this whole report's Gantt chart is reconstructed from.

4.2 Representative Code Extracts

Deterministic risk scoring

Kept as its own tiny, dependency-free module (Section 3.3) so both the controller and the background worker can use it without the worker having to import the controller's own queue/Redis wiring just for a lookup.

```
export const riskScoreForLevel = (level: string): number =>
  RISK_SCORE_BY_LEVEL[level] ?? 60;
```

server/src/modules/analysis/analysis.riskScore.ts

Background job failure bookkeeping

The worker only marks a document “failed” once nothing more will be retried — not on every transient attempt in between. BullMQ's job.attemptsMade is 0-indexed (attempts completed

before this one), which a real system test caught being off-by-one in an earlier version of this function.

```
export const analysisJobProcessor = async (job: Job<AnalysisJobData>) => {
  try {
    return await processAnalysisJob(job.data);
  } catch (err) {
    const maxAttempts = job.opts.attempts ?? 1;
    const isFinalAttempt =
      err instanceof UnrecoverableError || job.attemptsMade + 1 >= maxAttempts;
    if (isFinalAttempt) {
      await markDocumentFailed(job.data.documentId);
    }
    throw err;
  }
};
```

server/src/modules/analysis/analysis.worker.ts

Request validation (FR-4)

Every request body is checked against an explicit Zod schema before it reaches a service or the database.

```
export const createTextBodySchema = z.object({
  content: z
    .string({ error: "Pasted text is required" })
    .trim()
    .min(50, "Pasted text is too short. Please paste at least 50 characters."),
});
```

server/src/modules/document/document.schema.ts

Citation de-duplication (RAG grounding, Figure 2.8)

The model occasionally repeats the same source across citations; this caps and de-duplicates by URL before a reply is shown to the user.

```
export const dedupeCitations = (rawCitations: Citation[]): Citation[] => {
  const seenUrls = new Set<string>();
  return rawCitations
    .filter((c) => (seenUrls.has(c.url) ? false : (seenUrls.add(c.url), true)))
    .slice(0, 5);
};
```

server/src/modules/legal-agent/legal-agent.service.ts

4.3 Database Creation, Data Insertion, and Access Rights

The schema is never hand-written as SQL; Prisma Migrate generates it from schema.prisma (Appendix A) and applies it as versioned migration files checked into the repository (Section 3.3). The excerpt below is the actual generated SQL from two real migrations — the initial schema, and the later migration that added phone/OTP login — shown here specifically because it demonstrates genuine schema evolution rather than a schema designed complete and final on day one (Section 2.4).

```
-- From migrations/20260808215010_init/migration.sql
CREATE TABLE "users" (
  "id"          SERIAL          NOT NULL,
  "name"        TEXT            NOT NULL,
  "email"       TEXT            NOT NULL,
  "password"    TEXT            NOT NULL,
  "created_at"  TIMESTAMP(3)    NOT NULL DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT "users_pkey" PRIMARY KEY ("id")
```

```

);

CREATE TABLE "documents" (
  "id"          SERIAL NOT NULL,
  "user_id"     INTEGER NOT NULL,
  "status"      TEXT   NOT NULL DEFAULT 'pending',
  "created_at"  TIMESTAMP(3) NOT NULL DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT "documents_pkey" PRIMARY KEY ("id")
);

CREATE UNIQUE INDEX "users_email_key" ON "users"("email");

ALTER TABLE "documents" ADD CONSTRAINT "documents_user_id_fkey"
  FOREIGN KEY ("user_id") REFERENCES "users"("id") ON DELETE CASCADE ON UPDATE CASCADE;

-- From migrations/20260919043000_add_phone_and_otp/migration.sql
-- Real schema evolution: email becomes optional once phone/OTP login is added
ALTER TABLE "users" ALTER COLUMN "email" DROP NOT NULL;
ALTER TABLE "users" ADD COLUMN "phone" TEXT;
CREATE UNIQUE INDEX "users_phone_key" ON "users"("phone");

CREATE TABLE "otp_verifications" (
  "id"          SERIAL NOT NULL,
  "phone"       TEXT   NOT NULL,
  "code_hash"   TEXT   NOT NULL,
  "expires_at"  TIMESTAMP(3) NOT NULL,
  "attempts"    INTEGER NOT NULL DEFAULT 0,
  "consumed"    BOOLEAN NOT NULL DEFAULT false,
  CONSTRAINT "otp_verifications_pkey" PRIMARY KEY ("id")
);
CREATE INDEX "otp_verifications_phone_idx" ON "otp_verifications"("phone");

```

Generated by Prisma Migrate — not hand-written

Data insertion goes exclusively through Prisma's query builder (Section 6.3) — every row is created from application code, never by a hand-run INSERT. The OTP write path is a representative example: the code (the value actually shown to the user over SMS) never touches the database — only its bcrypt hash does, the same discipline described for passwords in Section 6.1.

```

export const createOtp = async (phone: string, codeHash: string, expiresAt: Date) => {
  return prisma.otpVerification.create({
    data: { phone, codeHash, expiresAt },
  });
};

```

server/src/modules/otp/otp.repository.ts

Access rights are enforced in the application layer, not through separate PostgreSQL database roles: the server connects with a single database credential, and every query is scoped by application logic instead (Section 6.2). Table 4.1 states this explicitly, since it is a deliberate architectural choice rather than an omission — appropriate for a single-service backend where the application is the only thing that ever talks to the database directly.

Table 4.1 — Access Rights by Role

Role	Database-Level Access	Application-Level Access
Unauthenticated	None — no connection is ever made on the client's behalf	Register, log in, request OTP only
USER	None directly; all access is through the single application DB credential	Own documents, conversations, consultations only — enforced by the ownership check in Section 6.2 (FR-3)

Role	Database-Level Access	Application-Level Access
ADMIN	Same single application DB credential — no elevated database role	Everything a USER can do, plus advocate verification and account management — gated by requireAdmin (Section 6.2), re-checked from the database on every request

4.4 Standardization of the Coding

Four checkpoints were applied consistently across the codebase rather than case by case; each is illustrated by code already shown earlier in this section rather than repeated here.

- **Code efficiency:** expensive work is never repeated when a cached or derived answer already exists — `riskScoreForLevel` (Section 4.2) is an $O(1)$ lookup computed once at analysis time, not recomputed on every read; a repeat analysis request is served from the cached `Analysis` row (Section 3.3) instead of re-invoking the AI provider; every foreign key on the per-user data-isolation hot path is indexed (Section 3.4), and `LegalKnowledgeChunk.embedding` carries a `pgvector` ANN index specifically because RAG retrieval runs on every chat turn.
- **Error handling:** the background worker (Section 4.2) draws an explicit, deliberate line between two error classes — `UnrecoverableError` for input that will never succeed on retry (a corrupt file, unreadable even after OCR), versus a plain `Error` for a transient failure (an AI-provider blip) that BullMQ's backoff should retry — rather than treating every failure identically. At the HTTP layer, a single central error-handling middleware converts every thrown error into a clean, consistent JSON response (verified directly by TC-13, Table 5.3) instead of leaking a raw stack trace.
- **Parameter calling and passing:** service and repository functions take plain, typed parameters (e.g. `createOtp(phone, codeHash, expiresAt)`, Section 4.3) rather than the raw Express `req/res` objects — only the controller layer ever touches `req/res`. This is not a style preference: it is what let Section 5.3's unit tests call business logic directly, with no Express app, no HTTP layer, and no mocking, at the millisecond speeds Table 5.1 reports.
- **Validation checks:** every request body, path parameter, and query string is checked against an explicit Zod schema (Section 4.2, 4.3) before it reaches a service or the database — malformed input is rejected with a 400 at the boundary, consistently, rather than each handler re-implementing its own ad hoc checks.

5. Testing

5.1 Testing Strategy

Testing follows two tiers, run with Vitest across both the server and the client. Unit tests target the small, dependency-free functions deliberately extracted for this purpose (Section 4.1) — pure logic with no database, queue, or network call, so each one runs in milliseconds with no setup. System tests go the opposite way on purpose: they run the real Express app against a real, disposable PostgreSQL and Redis instance (a dedicated `*_test` database, not the development database) through Supertest, issuing actual HTTP requests rather than calling service functions directly or mocking the database. This was a deliberate choice, not an oversight — a mocked integration test can pass while the real integration is broken, which is exactly the failure mode Section 5.6 documents a real instance of.

Two safeguards keep system testing itself safe to run. First, `src/test/setupEnv.ts` refuses to start the suite at all unless `DATABASE_URL` points at a database whose name contains `_test`, so a misconfigured environment cannot run tests against real data. Second, the same file asserts that the MSG91 SMS credentials are empty in the test environment before any test runs — a guard that exists directly because of a defect this project's own testing found (Section 5.6).

Table 5.1 — Test Suite Summary

Layer	Framework	Test Files	Test Cases	Result
Server — Unit	Vitest	20	155	155 passed
Server — System / Integration	Vitest + Supertest	6	67	67 passed
Client — Unit	Vitest + React Testing Library	4	31	31 passed
Total	—	30	253	253 passed

5.2 Testing Plan

Rather than a time-boxed test phase bolted on at the end, testing ran continuously from Phase A onward — every phase in Table 2.1 that touched server logic shipped with its own unit tests in the same branch, and the system-test suite (Section 5.4) was itself the deliverable of Phase G (Table 2.1), applied retroactively across every module already built. Table 5.2 states this as an explicit plan rather than leaving it implicit.

Table 5.2 — Testing Plan

Stage	Scope	Entry Criterion	Exit Criterion	Tooling
Unit (per phase)	New pure functions/schemas added in that phase	Function is dependency-free and exported	All cases pass; edge/invalid input covered	Vitest
System (Phase G)	Full request/response cycles, all modules	A real <code>*_test</code> Postgres + Redis instance is reachable	All cases pass against the real stack, not mocks	Vitest + Supertest
Regression (ongoing)	Entire suite	Any change to server or client source	253/253 still passing (Table 5.1) before merge	Vitest, run locally pre-merge

Stage	Scope	Entry Criterion	Exit Criterion	Tooling
Manual verification	UI flows, voice/WebRTC paths not covered by Supertest	A new client-facing feature is implemented	Golden path + edge cases exercised in a real browser	Manual, against the running dev app

5.3 Unit Testing Coverage

Unit tests cover, among others: the deterministic risk-score derivation and citation deduplication shown in Section 4.2; every Zod validation schema (user, document, analysis, OTP, legal-agent) against both valid and deliberately malformed input; the JWT helper; the structured logger's redaction rules, asserted against real serialized JSON output rather than the redaction config alone; the per-user rate-limiter key generator; and the background-worker job processor's retry and failure-marking logic described in Section 4.2, tested directly against a hand-built fake BullMQ job object.

5.4 System / Integration Testing Coverage

System tests exercise complete request/response cycles for: registration and login, including every validation-error path distinguished from genuine 401/404 business errors; phone OTP send/verify, including the resend cooldown and single-use consumption; document CRUD with per-user data isolation (FR-3) asserted as an explicit 403, not a leaked 404 or silent empty result; the full background analysis flow traced in Figure 2.7 — enqueue, cache-on-repeat-request, no-double-run under concurrent requests, and retry-then-succeed/retry-then-fail against the real queue; legal-assistant conversation validation and its own IDOR guard; and baseline security headers on both API and static routes.

5.5 Representative Test Cases

Table 5.3 — Representative Test Cases

ID	Module	Test Case	Expected Result	Result
TC-01	Auth	Register with an already-registered email	Rejected; no duplicate account created	Pass
TC-02	Auth	Login with correct credentials	200 OK with a usable JWT	Pass
TC-03	Auth	Access a protected route with a garbage token	401, route not reached	Pass
TC-04	OTP	Second OTP /send within 30s of the first	429, resend cooldown enforced	Pass
TC-05	OTP	Verify an already-consumed code a second time	401, code not reusable	Pass
TC-06	Documents	User A requests User B's document by id	403 (IDOR guard) — not a 404 that would leak existence	Pass
TC-07	Documents	Paste text under the 50-character minimum	400 at the validation layer, before the service runs	Pass
TC-08	Analysis	Run analysis twice on the same document	Second call returns the cached Analysis; AI not called again	Pass
TC-09	Analysis	Two concurrent run requests on one fresh document	Only one analysis job actually executes	Pass

ID	Module	Test Case	Expected Result	Result
TC-10	Analysis	AI call fails transiently, then succeeds on retry	Job completes; document ends up Completed, not Failed	Pass
TC-11	Analysis	AI call fails on every configured attempt	Document marked Failed only after the final attempt	Pass
TC-12	Legal Assistant	Conversation rename with neither title nor language	400, refused.refine() validation triggered	Pass
TC-13	Platform	GET an unknown API route	Clean 404 JSON body, not an HTML stack trace	Pass
TC-14	Platform	Any normal API response	Helmet baseline security headers present	Pass
TC-15	Validation	Phone number starting with 0 or otherwise malformed	400, rejected before reaching the service	Pass
TC-16	Validation	Numeric id param that is zero, negative, or non-integer	400, rejected by the shared idParamSchema	Pass
TC-17	Auth	JWT round-trips an arbitrary payload; wrong secret is rejected	Verifies correctly with the right secret only	Pass
TC-18	Platform	Log a payload with a top-level password field	Redacted in the real serialized output	Pass
TC-19	Platform	Log a payload with a password nested one level deep	Redacted in the real serialized output	Pass
TC-20	Platform	Log a request carrying an Authorization header	Header value redacted in the output	Pass
TC-21	Documents	Update a document with neither title nor isFavorite	400, refine() validation triggered	Pass
TC-22	Documents	Toggle isFavorite with a non-boolean value	400, rejected at the schema	Pass
TC-23	Documents	Delete a document, then GET it again	404 — confirmed gone, not soft-hidden	Pass
TC-24	Documents	List documents as two different authenticated users	Each sees only their own documents (FR-3)	Pass
TC-25	Legal Assistant	Ask a question with language omitted	Defaults to English rather than rejecting	Pass
TC-26	Legal Assistant	Ask a question with an unrecognised language code	400, rejected rather than silently defaulted	Pass

5.6 Defects Identified During Testing

The following defects were found by the testing process itself — not by manual use — and are included because each is a concrete instance of testing doing its actual job: catching a real bug before it reached users, not merely confirming code that already worked.

Table 5.4 — Defects Found and Resolved

#	Defect	Found By	Resolution
1	BullMQ's job.attemptsMade is 0-indexed (attempts completed before this one), not the current attempt number. A permanently-	A real system test driving genuine BullMQ backoff (TC-11) — not the unit test,	Corrected the final-attempt condition in analysisJobProcessor

#	Defect	Found By	Resolution
	failing job never reached a Failed state, because the off-by-one meant the final-attempt check never evaluated true.	whose hand-mocked fake job object had encoded the same wrong assumption.	(Section 4.2) and the unit test's fake values to match verified-real behaviour.
2	Uploading an image for OCR triggered an unconditional pdf-parse call left over from an unused preview feature, crashing the upload with a 500.	System testing of the OCR-fallback upload path added when image support was introduced.	Removed the dead preview code path entirely rather than special-casing it for images.
3	Omitting MSG91_AUTH_KEY from .env.test let the server's config loader fall back to the real .env file, so the OTP system test sent a live SMS through the production MSG91 account during a test run.	Noticed from an unexpected MSG91 dashboard send while running the OTP system test.	Set explicit empty values in .env.test and added the startup guard described in Section 5.1 (setupEnv.ts) that refuses to run tests if production credentials are detected.
4	Pino's redact wildcard pattern (e.g. "/*.password") only matches one level of nesting; a top-level { password: ... } field was logged completely unredacted.	A unit test asserting on the logger's real serialized JSON output, not just its redaction configuration.	Listed every sensitive field at both the top level and one level of nesting in the redaction rules.

5.7 Debugging and Code Improvement

Each defect in Table 5.4 followed the same debugging discipline: reproduce it under a real, non-mocked test first (never fix from reading the code alone), confirm the fix against that same test, then check whether the same mistake could exist elsewhere before moving on — which is precisely how defect #4 was found: fixing the top-level password-redaction gap prompted re-checking every other sensitive field for the same one-level-of-nesting blind spot, not just the field the original test happened to catch.

Two improvements came directly out of debugging rather than from a plan: the .env.test credential guard (Section 5.1) did not exist until defect #3 showed it was needed, and analysisJobProcessor's final-attempt condition (Section 4.2) was rewritten, with its unit test's fake values corrected to match, only after defect #1 showed the original assumption about job.attemptsMade was wrong. Both changes are still in the codebase specifically because the system test suite (Section 5.4) exercises real BullMQ backoff and a real environment loader rather than a simplified stand-in for either — the class of defect each one caught is, by construction, invisible to a unit test that mocks the thing that was actually wrong.

5.8 Worked Test Case Walkthrough

One real unit test file, validate.middleware.unit.test.ts, reproduced here almost in full rather than summarised, to show what “unit testing” concretely means in this codebase rather than only asserting a count. This is the test suite that specifies the exact 400 error format documented in Appendix F.3.

```
it("passes a 400 AppError to next() when the body fails validation", () => {
  const schema = z.object({ email: z.string().email("Enter a valid email address") });
  const { req, res, next } = makeReqRes({ body: { email: "not-an-email" } });

  validate({ body: schema })(req, res, next);

  expect(next).toHaveBeenCalledTimes(1);
  const err = next.mock.calls[0][0];
```

```

    expect(err.statusCode).toBe(400);
    expect(err.message).toContain("Enter a valid email address");
  });

  it("names the failing field in the error message", () => {
    const schema = z.object({ password: z.string().min(8, "Password must be at least 8 characters") });
    const { req, res, next } = makeReqRes({ body: { password: "short" } });

    validate({ body: schema })(req, res, next);

    const err = next.mock.calls[0][0];
    expect(err.message).toBe("password: Password must be at least 8 characters");
  });

```

server/src/common/middleware/validate.middleware.unit.test.ts (two of eight test cases in this file)

The file's other six cases (not reproduced here) check that a passing schema calls `next()` with no error, that `req.body` is replaced with the parsed — not the raw — value (so a `z.coerce.number()` or `.trim()` actually takes effect downstream), that `body/params/query` validate independently of each other, and that the middleware stops at the first failing target and never mutates a later one. Together the eight cases specify the exact contract Appendix F.1/F.3 describes in prose: what the response looks like, and precisely which field gets named when more than one is invalid — a contract a future change to `validate.middleware.ts` cannot silently break without this test file failing first.

A system test, for contrast.

Where the unit test above calls a function directly with hand-built fake objects, a system test drives the real HTTP server with Supertest against the real test database — `otp.system.test.ts` is reproduced in full below because it is short and demonstrates something a unit test structurally cannot: a full, stateful, ordered flow across six separate requests sharing one phone number.

```

describe("OTP login/registration flow", () => {
  const phone = uniquePhone();
  let realCode: string;

  it("400s a malformed phone number on send", async () => {
    const res = await request(app).post("/api/v1/auth/otp/send").send({ phone: "123" });
    expect(res.status).toBe(400);
  });

  it("sends a code for a valid phone number", async () => {
    const res = await request(app).post("/api/v1/auth/otp/send").send({ phone });
    expect(res.status).toBe(200);
    expect(typeof res.body.data.devCode).toBe("string");
    realCode = res.body.data.devCode;
  });

  it("30s resend cooldown: a second /send right after the first is rejected", async () => {
    const res = await request(app).post("/api/v1/auth/otp/send").send({ phone });
    expect(res.status).toBe(429);
  });

  it("verifies the real code and creates/logs in the account", async () => {
    const verify = await request(app).post("/api/v1/auth/otp/verify").send({ phone, code: realCode });
    expect(verify.status).toBe(200);
    expect(verify.body.data.user).not.toHaveProperty("password");
  });

  it("rejects the same code a second time (already consumed)", async () => {

```

```

    const res = await request(app).post("/api/v1/auth/otp/verify").send({ phone, code: realCode
  });
  expect(res.status).toBe(400);
});
});
});

```

server/src/test/system/otp.system.test.ts (5 of 7 cases; two pure-validation 400 cases omitted for space)

The devCode field only appears because server/.env.test deliberately configures no MSG91 credentials — otp.service.ts's dev-mode fallback (Section 6.1) logs the code instead of sending a real SMS and returns it directly in the response, which is exactly what lets this test assert against the real code without needing a live SMS gateway or a mocked one. This is also why the test can run entirely offline: it exercises actual bcrypt hashing, actual Postgres rows, and the actual 30-second cooldown and single-use rules (Table 2.2a) — the only thing not real is the SMS delivery itself, isolated behind one well-named branch.

5.9 Test Coverage by Module

Stated plainly rather than left to be inferred from Table 5.1's totals: automated test coverage is not even across the seven feature modules in Figure 3.2. Table 5.5 names exactly which modules the 253 passing tests actually exercise, and which are verified only manually (Section 5.2's fourth row) — the same honesty this report applies to Section 2.1's scope boundary and Table 6.2's OWASP gaps, extended to testing specifically.

Table 5.5 — Automated Test Coverage by Module

Module	Unit Tests	System Tests	Verification Method
Auth / User	Yes	Yes	Automated (Table 5.3, TC-01/02/03/17)
OTP	Yes	Yes	Automated (Table 5.3, TC-04/05)
Documents	Yes	Yes	Automated (Table 5.3, TC-06/07/21-24)
Analysis (background queue)	Yes	Yes	Automated (Table 5.3, TC-08-11)
Legal Assistant (RAG chat)	Yes	Yes	Automated (Table 5.3, TC-12/25/26)
Platform (logging, headers, error envelope)	Yes	Yes	Automated (Table 5.3, TC-13/14/18-20)
Consultations (AI voice advocate)	No	No	Manual only, against the running app (Section 5.2; Section 8, item 6)
Human Consultations (advocate desk)	No	No	Manual only, against the running app (Section 5.2; Section 8, item 6)
Advocates (admin CRUD, credentials)	No	No	Manual only, against the running app (Section 5.2)
RAG ingestion/retrieval pipeline	No	No	Verified via rag.eval.ts (Appendix D.3) — a retrieval-quality script, not a pass/fail Vitest suite
Speech (transcription)	No	No	Manual only, against the running app (Section 5.2)

The five rows with no automated coverage are exactly the modules built most recently (Phases E/F, Table 2.1) — consistent with the Lessons Learned observation (Section 8.1) that system testing should have started earlier in the schedule rather than being concentrated at the end. It

is a real gap, not a hidden one: it is the specific, concrete content behind Future Scope item 6 (Section 8) and the regression-coverage caveat already stated in Section 5.2's testing plan.

6. System Security Measures

Security is enforced at every layer the system has — authentication, authorisation, input, transport, storage, and logging — rather than relegated to a single gatekeeping middleware. Table 6.1 summarises the controls actually implemented, each mapped to the specific threat it addresses; the sections below give the reasoning behind the less obvious choices.

6.1 Authentication and Session Security

Passwords are hashed with bcrypt (10 salt rounds) before storage; the plaintext password is never persisted or logged. Phone-based OTP login follows the same discipline: the six-digit code is generated with `crypto.randomInt` (a cryptographically secure generator, not `Math.random`), stored only as a bcrypt hash, and enforced as single-use, time-limited, and rate-limited on resend — all four properties verified directly by the system tests in Table 5.3 (TC-04, TC-05), not merely asserted here. Authenticated sessions are JSON Web Tokens signed with a server-held secret and a 7-day expiry; Google OAuth is offered as an alternative login path for users who prefer not to set a password at all.

6.2 Authorisation and Access Control

Two separate mechanisms govern what an authenticated user may do. First, role-based access control gates admin-only routes (advocate verification, account management): the `requireAdmin` middleware re-reads the user's role from the database on every request rather than trusting the role embedded in the JWT, so revoking admin access takes effect immediately rather than waiting up to seven days for the token to expire. Second, and independently, every document, conversation, and consultation route checks row ownership explicitly (the pattern asserted in Table 5.3, TC-06: `doc.userId !== req.user.id` returns 403, not a 404 that would also leak whether the id exists) — this is the IDOR defence named as FR-3 in Section 2.3.1, and it is enforced in application code on every request, not assumed from the database schema alone.

6.3 Input Validation and Transport Security

Every request body, path parameter, and query string is checked against an explicit Zod schema before it reaches a service or the database (Section 4.2); malformed input is rejected with a 400 at the boundary, never allowed to reach business logic. All database access goes through Prisma's parameterised query builder — no route constructs SQL by string concatenation — which rules out classic SQL injection by construction rather than by developer discipline. HTTP responses carry Helmet's security headers (CSP, HSTS, X-Content-Type-Options, and related protections), with two narrow, explicitly documented exceptions: the interactive API-docs page is mounted before Helmet so its bundled inline script still renders (Section 5), and the resource policy is relaxed specifically for the one directory of public advocate-profile photos the client loads cross-origin. CORS is an explicit origin allowlist (the configured client and server URLs) with credentials enabled, not a wildcard.

6.4 Data and Infrastructure Security

Only one upload directory — advocate-profile photos — is served as static files; uploaded documents and voice recordings, which may contain sensitive personal or legal content, are never reachable by a direct URL. All credentials (database URL, Redis URL, OpenAI and

MSG91 API keys, JWT secret) are supplied through environment variables and are never hard-coded or committed — the single occasion this discipline slipped (Section 5.6, defect #3) is precisely why the test environment now has an automated startup guard refusing to run if production credentials are detected. The structured logger redacts sensitive fields (passwords, tokens, OTP codes) at the point of logging, at both the top level and one level of nesting (Section 5.6, defect #4) — asserted against real serialized log output, not just the redaction configuration. Cost-bearing and abuse-prone endpoints (AI analysis, chat, OTP send) sit behind a per-user, Redis-backed rate limiter, on a separate key prefix from the job queue so the two cannot collide.

Table 6.1 — Security Controls Summary

Control	Implementation	Threat Mitigated
Password storage	bcrypt hash, 10 salt rounds	Credential theft from a database dump
OTP codes	bcrypt-hashed, single-use, time-limited, resend cooldown	OTP brute-force and replay
Session tokens	JWT, 7-day expiry, server-signed	Session forgery
Admin authorisation	Role re-read from DB on every request, not trusted from the JWT	Stale-privilege access after revocation
Per-user data isolation	Explicit ownership check on every document/conversation/consultation route	IDOR (FR-3)
Input validation	Zod schema on every request body, param, and query	Injection, malformed-input crashes
SQL injection	Prisma parameterised queries exclusively; no raw string-built SQL	SQL injection
HTTP security headers	Helmet, with two narrow, documented exceptions	XSS, clickjacking, MIME-sniffing
CORS	Explicit origin allowlist with credentials, not a wildcard	Cross-origin credential theft
File exposure	Only advocate-photos served statically; documents/recordings never public	Direct-URL data leakage
Secrets management	Environment variables only; test environment has a startup guard (Section 5.6)	Credential leakage into logs or version control
Log redaction	Sensitive fields redacted at two nesting levels, verified against real output	Sensitive data exposure via logs
Rate limiting	Per-user, Redis-backed, separate key prefix from the job queue	Abuse, cost exhaustion, brute force

6.5 OWASP Top 10 Mapping

Table 6.2 maps the system's controls against the OWASP Top 10 (2021). It is included for completeness, and deliberately says so plainly where a category is not yet formally addressed, rather than overstating coverage the project does not actually have.

Table 6.2 — OWASP Top 10 (2021) Mapping

OWASP Category	This System's Position
A01 — Broken Access Control	Per-request IDOR ownership checks (Table 6.1); admin role re-read from the database on every request, never trusted from the JWT
A02 — Cryptographic Failures	Passwords and OTP codes hashed with bcrypt, never stored or logged in plaintext; secrets supplied via environment variables
A03 — Injection	Prisma's parameterised query builder exclusively — no raw string-built SQL anywhere in the codebase; Zod validation at every request boundary
A04 — Insecure Design	IDOR scoping and deterministic risk scoring were design decisions from Phase A/D (Table 2.1), not retrofits; the background-job split (Section 3.1) is itself a deliberate availability-by-design choice
A05 — Security Misconfiguration	Helmet security headers with two narrow, documented exceptions; explicit CORS allowlist, not a wildcard; environment-driven configuration with no default production secrets
A06 — Vulnerable and Outdated Components	Not formally addressed by an automated dependency-scanning process in this submission — an honest gap, not a claimed control; a reasonable Future Scope item (Section 8) rather than left unstated
A07 — Identification and Authentication Failures	OTP is single-use, time-limited, and rate-limited on resend (Table 5.3, TC-04/TC-05); JWT sessions have a bounded 7-day expiry
A08 — Software and Data Integrity Failures	Schema changes are versioned Prisma migrations checked into the repository (Section 4.3), not applied ad hoc against a live database
A09 — Security Logging and Monitoring Failures	Structured, redacted request logging (Pino + pino-http) on every request; redaction itself is unit-tested against real serialized output, not merely configured
A10 — Server-Side Request Forgery	The only server-side outbound fetches of external URLs (statute ingestion, Table 9.1) use a fixed, hard-coded source registry — never a user-supplied URL — so there is no attacker-controllable fetch target

6.6 Security Verification Checklist

A short, consolidated sign-off list drawing together the specific evidence already presented in this chapter and in Section 5, rather than a fresh set of claims — every row below points back to a table, section, or test case already in this report, not a new, unverified assertion.

- Passwords and OTP codes are bcrypt-hashed, never logged or stored in plaintext — verified directly by inspection of `user.service.ts/otp.service.ts` (Section 6.1) and by the redaction unit tests (TC-18/19/20, Table 5.3).
- Cross-user data access returns 403, never a leaking 404 or the resource itself — verified by TC-06 and TC-24 (Table 5.3), not merely asserted as a design intention.
- Every request body, path parameter, and query string is validated against an explicit Zod schema before reaching a service — verified by the eight-case `validate.middleware` test suite walked through in full in Section 5.8.
- Security headers (Helmet baseline) are present on both API and static routes — verified by TC-14 (Table 5.3).
- No secret is hard-coded in the repository — every credential is environment-supplied (Appendix B), and the one tracked environment file (Appendix B.1) deliberately keeps its own secrets as placeholders or explicit blanks.

- Rate limiting is applied to every cost-bearing route, keyed by user id where available (Appendix F.5) — not an afterthought bolted onto only the routes that happened to need it during testing.
- Nine of ten OWASP Top 10 (2021) categories are mapped to an implemented, verifiable control (Table 6.2); the tenth (A06, dependency scanning) is named as an open gap (Section 8, item 9), not silently left unmentioned.

This checklist is deliberately not a claim of completeness beyond what the rest of the report supports — Table 5.5's five modules with no automated coverage, and the dead-code duplicate error handler named in Appendix F.1, are exactly the kind of finding this list does not paper over.

Taken together, Sections 6.1 through 6.6 describe security as this project actually practised it: enforced at every layer the system has, verified by a real, named test or a real, named code path rather than asserted in prose alone, and — where a gap genuinely remains — named as a gap rather than quietly omitted. That last property is itself a deliberate security practice, not merely a documentation style choice: a report that only lists controls that work invites exactly the false confidence Section 2.1 opened by warning against in the context of a legal document's own one-sided clauses.

7. Reports

The system's primary outputs are not printed reports but structured, on-screen documents the user can revisit at any time from their account. This section lays out the field structure of each generated report for reference, alongside the screen where it was already shown rendered end to end.

7.1 Document Risk Analysis Report

Produced by the analysis pipeline traced in Figure 2.7 and shown rendered in Figure 3.6. Cached per document (Section 3.3) — the same report is returned on every subsequent view rather than regenerated.

The AI provider itself returns only a document-type label and a three-level riskLevel (Low/Medium/High) — both constrained server-side to a fixed set, any other value the model returns is coerced to "Other" rather than stored as-is (analysis.service.ts). The numeric Risk Score is not an AI output at all: it is the deterministic lookup already described in Section 3.3/4.2 (Low→30, Medium→60, High→90).

Field	Description
Document Title / Type	Auto-classified by the AI into one of eight fixed types (Rental Agreement, Employment Contract, NDA, Loan Agreement, Privacy Policy, Terms of Service, Business Contract, Other); any other value is coerced to "Other"
Risk Score	30, 60, or 90 — a fixed lookup from Risk Level (Section 3.3), not a continuous AI-produced number
Risk Level	Low / Medium / High — exactly three levels; the AI never outputs a fourth, higher tier
Summary	Plain-language AI summary of the document's effect
Risk Items	Per-clause list: the risky text, its category (one of Financial, Termination, Liability, Privacy, Other), severity (Low/Medium/High), and an explanation of why it is flagged

```
{
  "documentId": 482,
  "title": "Residential Rental Agreement",
  "documentType": "Rental Agreement",
  "riskScore": 60,
  "riskLevel": "Medium",
  "summary": "An 11-month leave-and-licence agreement that renews automatically unless either party gives 30 days' written notice. The security-deposit clause and the maintenance clause are the two highest-risk terms.",
  "riskItems": [
    {
      "clause": "The Licensee shall forfeit the entire security deposit in the event of early termination for any reason whatsoever.",
      "category": "Financial",
      "severity": "High",
      "explanation": "A full, unconditional forfeiture clause is disproportionate and likely unenforceable as a penalty rather than genuine pre-estimated loss."
    },
    {
      "clause": "All structural and non-structural repairs shall be borne solely by the licensee.",
      "category": "Liability",
      "severity": "Medium",
      "explanation": "Structural repairs are conventionally the licensor's responsibility; shifting them entirely to the licensee is an unusual allocation of risk."
    }
  ]
}
```

Field	Description
<pre>} } }</pre>	

Example — POST /analysis/run response body once status is completed (abridged, field names and value sets exactly as returned by analysis.service.ts)

7.1.1 Risk Category and Severity Taxonomy

Both the category and severity a clause is classified under come from a closed set the AI is prompted with and the server independently enforces — a value outside the set is coerced rather than trusted, so the taxonomy below is the complete, exhaustive set actually ever stored, not merely the common cases.

Category	Typical clause this covers
Financial	Security deposits, fees, penalties, payment schedules, and forfeiture terms
Termination	Notice periods, renewal/auto-renewal, and conditions under which either party can end the agreement
Liability	Indemnity, limitation-of-liability, and which party bears responsibility for loss or damage
Privacy	Data collection, sharing, and retention terms, in documents such as a Privacy Policy or Terms of Service
Other	Anything that does not fit the four categories above, or a value the AI returned that the server did not recognise

Severity, separately, is always exactly one of Low, Medium, or High — there is no fourth, more severe tier (Section 7.1); the overall document-level Risk Score and Risk Level use this same three-point scale, not a clause-level aggregate computed from the individual riskItems.

The client enforces the same three-value set independently, on the rendering side: RISK_LEVEL_BADGE (client/src/features/document/riskStyles.ts) is a fixed lookup of exactly High/Medium/Low to a badge class string, unit-tested (riskStyles.unit.test.ts) to confirm it has no more and no fewer than those three keys, that every value is visually distinct, and that every level defines both a light- and a dark-mode class — so a server-side change that ever introduced a fourth level would fail this test before it could reach a user as an unstyled badge.

Table 7.1a — Risk Badge Colour Tokens (Light / Dark)

Level	Light-mode classes	Dark-mode classes
High	bg-risk-high-bg, text-risk-high-fg	bg-risk-high-bg-dark, text-risk-high-fg-dark
Medium	bg-risk-med-bg, text-risk-med-fg	bg-risk-med-bg-dark, text-risk-med-fg-dark
Low	bg-risk-low-bg, text-risk-low-fg	bg-risk-low-bg-dark, text-risk-low-fg-dark

7.2 Legal Assistant Grounded Answer

Produced by the RAG flow traced in Figure 2.8 and shown rendered in Figure 3.7.

Field	Description
Query	The user's question, typed or transcribed from speech, in English or Hindi
Answer	Grounded reply, streamed to the client token-by-token over SSE

Field	Description
Citations	Act short name + section for each provision cited, verified against the retrieved source text
Disclaimer	Appended server-side unconditionally (Section 4.2) — not left to the model's discretion
<pre>{ "query": "My landlord is refusing to return my security deposit two months after I vacated. What can I do?", "answer": "Under most state Rent Control Acts and the general principles codified for tenancy disputes, a landlord withholding a deposit without a lawful deduction (unpaid rent, documented damage) is liable to refund it with interest in some states...", "citations": [{ "act": "Model Tenancy Act, 2021", "section": "Section 10(3)" }, { "act": "Consumer Protection Act, 2019", "section": "Section 2(7) – deficiency in service" }], "disclaimer": "This is general legal information, not legal advice for your specific situation. Consult a licensed advocate before taking action." }</pre>	

Example — POST /legal-agent/conversations/:id/messages final SSE payload (abridged)

7.3 Consultation Summary Report

Generated once a live consultation (Figure 3.8) ends, from the full speech transcript plus every passage the advocate actually retrieved during the call — not regenerated from scratch, so it cannot introduce a legal provision that was never looked up. The summary's own citations are then re-checked a second time against those retrieved passages before being stored, the same verification the live call itself applies, so a provision the model adds while synthesising the write-up (rather than during the conversation) is still caught and flagged rather than silently trusted.

Field	Description
Situation	2–4 sentence account of what happened, when, where, and who was involved
Key Facts	Dates, amounts, parties, and documents mentioned in the call
Questions Asked	What the advocate asked the client during the consultation
Provisions	Each cited Act/section with its plain-language meaning
Options	Possible courses of action, each with steps, forum, timeline, likely reaction, and risks
Recommendation	The suggested path forward and the reasoning behind it
Next Steps / Deadlines	Concrete actions for the client to take, and any time-bound items mentioned
Open Questions	Things the client should still confirm with a human advocate
Provisions Checked	Verification status of every citation in the summary, not only the ones retrieved live
<pre>{ "situation": "Client purchased a used car from a dealer who verbally promised a refund if defects appeared within 30 days; the engine failed on day 12 and the dealer is now refusing the refund.", "keyFacts": ["Purchase date: within the last 30 days", "Defect: engine failure, day 12", "Amount in dispute: vehicle purchase price", "No written warranty was signed"], "questionsAsked": ["Was anything in writing?", "Was the defect reported immediately?", "Was the car sold as used/as-is?"], "provisions": [</pre>	

Field	Description
	<pre> { "act": "Consumer Protection Act, 2019", "section": "Section 2(11)", "meaning": "A defect in goods sold is an actionable deficiency even absent a written warranty, if a representation induced the sale." }], "options": [{ "path": "Consumer complaint", "forum": "District Consumer Disputes Redressal Commission", "timeline": "Weeks to a few months", "likelyReaction": "Dealer may settle to avoid a formal order", "risks": "Requires evidence of the verbal promise (witnesses, messages)" }], "recommendation": "File a written complaint to the dealer first, referencing the verbal promise, before escalating to the Consumer Commission.", "nextSteps": ["Send a dated written demand to the dealer", "Gather any witnesses or messages confirming the verbal promise"], "openQuestions": ["Whether any communication exists corroborating the verbal promise"], "provisionsChecked": [{ "section": "Consumer Protection Act, 2019 – Section 2(11)", "verified": true }] } </pre>

Example — stored ConsultationSummary record (abridged)

8. Future Scope and Further Enhancement

The system's scope was deliberately bounded (Section 2.1) to keep it achievable within the project timeline. The following are concrete, honest extensions of that scope — gaps the current implementation is aware of, not speculative feature wishes.

1. State-specific statute coverage

The Legal Knowledge Base currently ingests ten central Acts only (Table 9.1); a question resting on a state amendment — a state Rent Control Act's specific notice period, for instance — is exactly the kind of gap named honestly in Section 2.1 rather than silently mis-answered. Extending `rag.sources.ts` (Appendix A's sibling file) with a state-jurisdiction dimension, alongside the existing "IN" central jurisdiction, is an additive change to the ingestion pipeline already built (Figure 2.3), not a redesign of it.

2. Downloadable report export

The Document Risk Analysis Report and the Consultation Summary (Section 7) are currently on-screen only. A PDF export — rendering the same Analysis/summary JSON the screen already reads, through a server-side template rather than a client-side screenshot — would let a user hand a report to a landlord, employer, or advocate directly, which is a realistic and frequently-requested use of exactly this kind of output.

3. Multi-document comparison

Every analysis today is of one document in isolation. Redlining two versions of the same contract, or comparing a signed document against its original draft, would reuse the same extraction and AI-summary pipeline (Figure 2.5) twice and add a diff step — a natural extension of Analysis rather than a new module.

4. Broader language support

The interface and the Legal Assistant currently support English and Hindi only. Nothing in the architecture (Section 3.1) assumes English specifically — the AI provider, the prompt templates, and the UI copy are all already parameterised by a language field — so adding further Indian languages is primarily a content and prompt-localisation effort, not an architectural one.

5. An admin analytics dashboard

Aggregate risk-level trends, document-type distribution, and usage metrics would build directly on the Admin module already in place (Figure 3.2) and the data already being written on every analysis (Section 2.5.7) — the gap is a reporting view over existing rows, not new data collection.

6. Automated regression coverage for voice/WebRTC

Section 5.1 scopes system testing to HTTP request/response cycles through Supertest; the live-audio and WebRTC signalling paths (Figure 2.9) are currently verified manually against the running application (Section 5.2) rather than by an automated browser-level suite. Dedicated WebRTC test tooling (e.g. Playwright with real media streams) is a known, named gap rather than an unstated one.

7. A deeper human-advocate marketplace

Specialisation search, ratings and reviews, and scheduled — not only immediately-live — consultations would extend the Connect Advocate module (Figure 3.8) and the existing AdvocateCredential/verification model (Appendix A) rather than requiring a new data model.

8. Cost reduction at scale

Fine-tuning or self-hosting a smaller model for the highest-volume, lowest-complexity calls — document-type classification, risk-level scoring — while keeping the general-purpose OpenAI model for open-ended chat and voice, would reduce per-request cost without touching the RAG-grounding guarantees (Figure 2.8) that matter most for the Legal Assistant's accuracy.

9. Automated dependency vulnerability scanning

Named explicitly in Table 6.2 (OWASP A06) as not yet formally addressed: wiring npm audit, or an equivalent scanner, into the same pre-merge step the regression suite already runs on (Table 5.2) would close this gap with infrastructure the project already has, rather than requiring a new CI system.

8.1 Lessons Learned

A few honest observations from actually building this system solo, over seven phases (Table 2.1), that are more useful recorded here than left as tacit knowledge.

Decisions that paid off.

- Grounding the Legal Assistant in retrieved statute text with server-side citation verification (Section 2.5.4) from the start, rather than trusting the model's own claims and adding verification later, avoided an entire class of “plausible-sounding wrong answer” defects this report would otherwise have had to document as found-and-fixed rather than designed-in (Section 7.2).
- The repository-pattern rule that only *.repository.ts files touch Prisma (Figure 3.9, Section 4.4) made the four real defects in Table 5.4 straightforward to isolate — each one lived in exactly one layer, not smeared across a controller that also queried the database directly.
- Treating the background analysis queue (Figure 2.7) as a first-class module from Phase D onward, instead of a synchronous call bolted on later, meant a slow OCR fallback or a flaky AI provider response never blocked the HTTP request/response cycle for an unrelated route.

What would be done differently.

- Automated dependency scanning (Section 8, item 9) and a WebRTC-level automated test harness (Section 8, item 6) should have been scheduled as their own small phases from the start, the way the background queue and RAG grounding were — instead they ended up as end-of-project gaps named honestly rather than quietly fixed, simply because nothing in the original seven-phase plan (Table 2.1) made space for them.
- Several defects in Table 5.4 (the attemptsMade indexing bug, the Pino redact-wildcard nesting limit) were specifically the kind that only a real system test, not a unit test of the function in isolation, would have caught — system testing (Section 5.1) was correctly scoped, but it should have started earlier in the schedule than Phase G, rather than being concentrated at the end.

- The single-part-time-developer constraint (Table 2.2a) was managed honestly rather than denied, but it remains the project's single largest risk; a two-or-more-contributor team would have let Phase B (Legal Assistant/RAG) and Phase C (UI) run in parallel instead of sequentially, which Figure 2.1's own critical path shows as the dominant cost driver.

None of this changes the conclusion in Section 2.1 that the system meets the need it set out to address; it is recorded because an honest account of what was learned is worth more, to a future maintainer or examiner, than a report that only describes the finished system as though it had been obvious from the start.

8.2 Conclusion

Section 2.1 identified a specific, concrete need: a fast, low-cost way to understand what is risky in a legal document before signing it, and a way to get a question about Indian law answered in a way that is grounded in, and verifiably traceable to, the actual statutory text. NyayMitra AI addresses both halves of that need directly — the Document Risk Analysis pipeline (Figure 2.7, Section 7.1) and the RAG-grounded Legal Assistant (Figure 2.8, Section 7.2) — with a third, escalation path to a real AI or human advocate (Connect Advocate, Figure 2.9) for anything that genuinely exceeds what an automated system should answer alone. All fifteen functional requirements in Section 2.3.1 are implemented and exercised by at least one of the 253 passing automated tests (Table 5.1) or a documented manual verification step (Table 5.5).

The system was built, tested, and secured to a standard this report has tried to describe exactly as it is, not as a finished product narrative written after the fact: Table 5.4 names four real defects found and fixed rather than omitted, Table 5.5 names five modules with no automated coverage rather than implying uniform rigor, Table 6.2 names one OWASP control (A06, dependency scanning) not yet addressed, and Section 2.1's own Scope and Limitations names what the system deliberately does not attempt — state-specific statutes, legal advice in the professional sense, and multi-document comparison among them. None of these are failures of the project; they are the honest boundary of what a single part-time developer, working across seven incremental phases alongside full-time study and employment (Table 2.1), could deliver to a genuinely working, tested, and secured standard within the available time.

Taken together, the system demonstrates that grounding an AI's legal output in retrieved, verifiable source text — rather than trusting a model's own claim — is both achievable and testable with ordinary, freely-available tools (Section 2.1.1's economic feasibility finding), and that the resulting system measurably narrows the specific information-asymmetry gap Section 2.1 opened with. The Future Scope items above are the next, concrete steps toward closing the remaining gaps — not evidence that the current system falls short of what it set out, specifically and modestly, to do.

8.3 Maintenance and Support Plan

Not a hypothetical plan for a future maintainer: every mechanism below is one this project already used on itself during development (Table 5.4's four real defects were all found, fixed, and verified this way), restated here as the ongoing maintenance process rather than a one-off debugging history.

Activity	Mechanism already in place
Applying a schema change	A new, version-controlled Prisma migration (Section 4.3); never an ad hoc ALTER TABLE against a live database

Activity	Mechanism already in place
Rolling out a server change	npm run build, then npm run prisma:deploy (applies only committed migrations, never prompts), then restart under a process supervisor (Appendix D.5) — no manual, undocumented step
Diagnosing a production issue	Structured Pino logs with request correlation and secret redaction (Section 6.4); every log is JSON, so a log aggregator can be pointed at it without a server-side code change
Verifying a fix before it ships	The regression suite (Table 5.1) must still pass 253/253 before merge (Table 5.2) — the same gate every change was held to during development, not relaxed for "maintenance" changes
Tracking a defect to resolution	Reproduce under a real, non-mocked test first, fix, confirm against that test, then check for the same mistake elsewhere (Section 5.7) — the exact discipline that caught all four defects in Table 5.4
Rotating or updating a secret	Every credential is environment-supplied (Appendix B), never hard-coded (Section 6.4) — rotation is a deployment-environment change, not a code change or redeploy
Extending the Legal Knowledge Base	POST /rag/ingest-statutes (Appendix C) re-runs ingestion against the fixed source registry (Table 9.1); adding a new Act is a registry entry plus a re-ingest, not a schema change

The gaps this maintenance process does not yet cover are the same ones named honestly earlier rather than repeated here: automated dependency-vulnerability scanning is not yet wired into the pre-merge gate (Table 6.2, OWASP A06), and five modules have no automated regression coverage to protect against during a future change (Table 5.5) — both are Future Scope items (Section 8, items 6 and 9), not silently assumed solved.

9. Bibliography

9.1 Primary Legal Sources

The Legal Assistant (Section 2.5.4, Figure 2.8) answers only from statute text actually ingested into the LegalKnowledgeChunk store (Section 2.5.7), retrieved from the following official government sources — not from the AI model's own training-time knowledge of Indian law.

Table 9.1 — Statute Sources Ingested into the Legal Knowledge Base

Act	Official Source
Bharatiya Nyaya Sanhita, 2023 (BNS)	mha.gov.in — Ministry of Home Affairs
Bharatiya Nagarik Suraksha Sanhita, 2023 (BNSS)	mha.gov.in — Ministry of Home Affairs
Bharatiya Sakshya Adhinyam, 2023 (BSA)	mha.gov.in — Ministry of Home Affairs
Code of Criminal Procedure, 1973 (CrPC)	indiacode.nic.in — Government of India
Indian Evidence Act, 1872 (IEA)	indiacode.nic.in — Government of India
Protection of Women from Domestic Violence Act, 2005	indiacode.nic.in — Government of India
Consumer Protection Act, 2019	ncdrc.nic.in — National Consumer Disputes Redressal Commission
Transfer of Property Act, 1882	indiacode.nic.in — Government of India
Right to Information Act, 2005	cic.gov.in — Central Information Commission
Protection of Children from Sexual Offences Act, 2012 (POCSO)	indiacode.nic.in — Government of India

9.2 Technical References

1. IGNOU School of Computer and Information Sciences. Project Guidelines for Master of Computer Applications (MCSP-232), July 2024 & January 2025 — the source of the report structure and page-count requirements followed throughout this document.
2. React documentation — react.dev — consulted for component lifecycle and hooks used throughout the client (Section 3.1).
3. Express.js documentation — expressjs.com — consulted for routing and middleware chaining (Figure 3.9).
4. Prisma ORM documentation — prisma.io/docs — consulted for schema modelling, migrations (Section 4.3) and the query API used in every *.repository.ts file.
5. PostgreSQL documentation — postgresql.org/docs, and the pgvector extension — github.com/pgvector/pgvector — consulted for the vector-similarity index behind RAG retrieval (Section 2.5.1).
6. Redis documentation — redis.io/docs — consulted for the append-only persistence mode used in docker-compose.yml (Appendix D.1).
7. BullMQ documentation — docs.bullmq.io — consulted for job-retry semantics, including the attemptsMade indexing defect noted in Section 5.4.
8. OpenAI API reference (Chat Completions, Embeddings, Realtime API) — platform.openai.com/docs — consulted for the streaming completion API (Figure 2.8) and the Realtime voice API (Figure 2.9).

9. Zod documentation — zod.dev — consulted for the request-validation schemas described in Section 4.2.
10. Helmet.js documentation — helmetjs.github.io — consulted for the HTTP security headers in Section 6.3.
11. Pino logger documentation — getpino.io — consulted for structured logging and the redact-path behaviour noted in Section 5.4.
12. Vitest documentation — vitest.dev — the test runner used for every unit and system test in Section 5.1.
13. Playwright documentation — playwright.dev — used both for browser-based system tests (Section 5.1) and for rendering this report's own SVG diagrams.
14. Jones, M. et al. RFC 7519 — JSON Web Token (JWT). IETF, 2015 — the standard the session-token format in Section 6.1 follows.
15. Hardt, D. (ed.) RFC 6749 — The OAuth 2.0 Authorization Framework. IETF, 2012 — the standard behind the Google OAuth login option (Appendix B).
16. W3C. WebRTC 1.0: Real-Time Communication Between Browsers — w3.org/TR/webrtc — the standard behind the Connect Advocate human consultation call (Section 3.1, Appendix C).
17. OWASP Foundation. OWASP Top 10 — owasp.org/www-project-top-ten — the checklist the security mapping in Table 6.2 is built against.

10. Appendices

Appendix A — Complete Prisma Schema

The full physical schema referenced throughout Section 2.5.7 (Data Dictionary) and Section 3.3 (Database Design), reproduced here in full rather than the core-entity subset shown earlier. All thirteen entities, every field, and every relation are exactly as implemented — this is the real schema.prisma file, not a simplified reconstruction.

Appendix A.1 — Migration History

The schema was never written once; it grew through nine real, version-controlled migrations (server/prisma/migrations), each applied — never hand-edited against a live database (Section 4.3) — as the corresponding phase in Table 2.1 needed it. The dates below are each migration folder's own timestamp prefix, not a reconstruction.

Date	Migration	What it added
2026-08-08	init	The original schema: User, Document, Analysis, Conversation, Message (Phase A/B, Table 2.1)
2026-08-09	add_document_classification_and_structured_risk_items	Document.documentType and Analysis.riskItems (the structured, per-clause risk list behind Section 7.1)
2026-08-22	add_legal_agent_and_rag	ConversationDocument and LegalKnowledgeChunk (Phase B's RAG grounding, Figure 2.8)
2026-09-19	add_phone_and_otp	User.phone and OtpVerification (FR-2's phone-OTP login, Section 6.1)
2026-09-19	add_document_favorite	Document.isFavorite
2026-09-20	add_roles_and_advocates	User.role, Advocate, AdvocateCredential (Phase D's admin/advocate foundation)
2026-09-20	add_knowledge_grounding_metadata	LegalKnowledgeChunk's sourceType/jurisdiction/actShort columns — the grounding metadata Table 9.1 and Procedure 4 (VerifyCitations) both depend on
2026-09-20	add_consultations	AdvocateAiConfig, Consultation (Phase E's AI voice advocate, Figure 2.9)
2026-09-21	add_human_consultations	ConsultationTurn and the REQUESTED/ACCEPTED/... status values (Phase E's human-advocate side, Table 2.5b)

Reading the sequence top to bottom is, in effect, a second, independent trace of Table 2.1's phase order — RAG grounding arrives before advocates, advocates before consultations, and the human-consultation side arrives last — not asserted in Section 2.2, but checkable directly against these nine folders' own names and timestamps.

```

generator client {
  provider = "prisma-client"
  output   = "../src/generated/prisma"
}

datasource db {
  provider = "postgresql"
}

model User {
  id          Int          @id @default(autoincrement())
  name        String
  email       String?      @unique
  phone       String?      @unique
  password    String
  role        UserRole     @default(USER)
  createdAt   DateTime     @default(now()) @map("created_at")

  documents    Document[]
  conversations Conversation[]
  consultations Consultation[]
  advocateProfile Advocate?

  @@map("users")
}

model OtpVerification {
  id          Int          @id @default(autoincrement())
  phone       String
  codeHash    String       @map("code_hash")
  expiresAt   DateTime     @map("expires_at")
  attempts    Int          @default(0)
  consumed    Boolean       @default(false)
  lastSentAt  DateTime     @default(now()) @map("last_sent_at")
  createdAt   DateTime     @default(now()) @map("created_at")

  @@index([phone])
  @@map("otp_verifications")
}

model Document {
  id          Int          @id @default(autoincrement())
  userId      Int          @map("user_id")
  filePath    String?      @map("file_path")
  content     String?
}

```

```

status      String    @default("pending")
title       String?
documentType String?   @map("document_type")
isFavorite  Boolean    @default(false) @map("is_favorite")
createdAt   DateTime   @default(now()) @map("created_at")

user        User      @relation(fields: [userId], references: [id],
onDelete: Cascade)
analysis    Analysis?
conversationDocuments ConversationDocument[]

  @@map("documents")
}

model Analysis {
  id          Int      @id @default(autoincrement())
  documentId  Int      @unique @map("document_id")
  summary     String
  riskLevel   String    @map("risk_level")
  riskScore   Int?      @map("risk_score")
  clauses     String[]  @default([])
  riskItems   Json      @default("[]") @map("risk_items")
  createdAt   DateTime  @default(now()) @map("created_at")

  document    Document @relation(fields: [documentId], references: [id], onDelete: Cascade)

  @@map("analyses")
}

model Conversation {
  id          Int      @id @default(autoincrement())
  userId      Int      @map("user_id")
  title       String?
  language    String    @default("en")
  createdAt   DateTime  @default(now()) @map("created_at")
  updatedAt   DateTime  @updatedAt @map("updated_at")

  user        User      @relation(fields: [userId], references: [id], onDelete:
Cascade)
  messages    Message[]
  documents    ConversationDocument[]

  @@map("conversations")
}

```

```

model Message {
  id          Int          @id @default(autoincrement())
  conversationId Int        @map("conversation_id")
  role        String
  content      String
  kind         String      @default("text")
  clarifyOptions Json?     @map("clarify_options")
  citations    Json?
  audioUrl     String?     @map("audio_url")
  createdAt    DateTime    @default(now()) @map("created_at")

  conversation Conversation @relation(fields: [conversationId], references: [id], onDelete: Cascade)

  @@map("messages")
}

model ConversationDocument {
  id          Int          @id @default(autoincrement())
  conversationId Int        @map("conversation_id")
  documentId   Int         @map("document_id")
  createdAt    DateTime    @default(now()) @map("created_at")

  conversation Conversation @relation(fields: [conversationId], references: [id], onDelete: Cascade)
  document      Document    @relation(fields: [documentId], references: [id], onDelete: Cascade)

  @@unique([conversationId, documentId])
  @@map("conversation_documents")
}

// Where a knowledge chunk came from. Advocate retrieval only uses the GOV_*
// types; AGGREGATOR (e.g. Indian Kanoon) is kept for the document chat only.
enum KnowledgeSourceType {
  GOV_STATUTE
  GOV_JUDGMENT
  GOV_GAZETTE
  AGGREGATOR
}

model LegalKnowledgeChunk {
  id          Int          @id @default(autoincrement())
  sourceUrl   String       @map("source_url")

```

```

sourceTitle    String          @map("source_title")
act            String?
section        String?
content        String
embedding      Unsupported("vector(1536)")
publishedAt    DateTime?       @map("published_at")
crawledAt      DateTime        @default(now()) @map("crawled_at")
contentHash    String          @unique @map("content_hash")
// Grounding metadata
sourceType     KnowledgeSourceType @default(AGGREGATOR) @map("source_type")
sourceDomain   String?         @map("source_domain")
jurisdiction   String          @default("IN")
actShort       String?         @map("act_short")
sectionHeading String?         @map("section_heading")
language       String          @default("en")
effectiveFrom  DateTime?       @map("effective_from")
effectiveTo    DateTime?       @map("effective_to")

@@index([sourceType, jurisdiction])
@@index([actShort, section])
@@map("legal_knowledge_chunks")
}

// — Advocates (AI and human share one profile structure) —————

enum UserRole {
  USER
  ADMIN
}

enum AdvocateKind {
  AI
  HUMAN
}

enum AdvocateStatus {
  DRAFT
  ACTIVE
  DISABLED
}

enum VerificationStatus {
  UNVERIFIED
  VERIFIED
}

```

```

// HUMAN advocates use ENROLMENT/DEGREE/CERTIFICATION/BAR_MEMBERSHIP.
// The AI advocate uses KNOWLEDGE_SOURCE/SCOPE/LAST_VERIFIED – never invented
// degrees or bar numbers.
enum CredentialType {
    ENROLMENT
    DEGREE
    CERTIFICATION
    BAR_MEMBERSHIP
    KNOWLEDGE_SOURCE
    SCOPE
    LAST_VERIFIED
}

model Advocate {
    id Int @id @default(autoincrement())
    kind AdvocateKind
    displayName String @map("display_name")
    slug String @unique
    photoUrl String? @map("photo_url")
    headline String?
    bio String?
    languages String[] @default([])
    practiceAreas String[] @default([]) @map("practice_areas")
    courts String[] @default([])
    statesCovered String[] @default([]) @map("states_covered")
    yearsExperience Int? @map("years_experience")
    status AdvocateStatus @default(DRAFT)
    verificationStatus VerificationStatus @default(UNVERIFIED) @map("verification_status")
    verifiedAt DateTime? @map("verified_at")
    verifiedById Int? @map("verified_by_id")
    // The login account a human advocate uses to reach their Advocate Desk (optional, one
    // advocate per account).
    userId Int? @unique @map("user_id")
    acceptingConsultations Boolean @default(false) @map("accepting_consultations")
    sortOrder Int @default(0) @map("sort_order")
    createdAt DateTime @default(now()) @map("created_at")
    updatedAt DateTime @updatedAt @map("updated_at")

    credentials AdvocateCredential[]
    aiConfig AdvocateAiConfig?
    consultations Consultation[]
    user User? @relation(fields: [userId], references: [id], onDelete:
    SetNull)

    @@map("advocates")

```

```

}

model AdvocateCredential {
  id          Int          @id @default(autoincrement())
  advocateId Int          @map("advocate_id")
  type        CredentialType
  title       String
  issuer      String?
  year        Int?
  identifier   String?
  documentUrl String?      @map("document_url")
  verified     Boolean      @default(false)
  sortOrder    Int         @default(0) @map("sort_order")
  createdAt    DateTime     @default(now()) @map("created_at")

  advocate Advocate @relation(fields: [advocateId], references: [id], onDelete: Cascade)

  @@index([advocateId])
  @@map("advocate_credentials")
}

model AdvocateAiConfig {
  id          Int          @id @default(autoincrement())
  advocateId Int          @unique @map("advocate_id")
  provider     String      @default("openai")
  model        String
  voice        String      @default("alloy")
  temperature  Float       @default(0.8)
  personaPrompt String     @default("") @map("persona_prompt")
  maxSessionMinutes Int    @default(20) @map("max_session_minutes")
  ragConfig    Json        @default("{}") @map("rag_config")
  version      Int         @default(1)
  updatedAt    DateTime    @updatedAt @map("updated_at")

  advocate Advocate @relation(fields: [advocateId], references: [id], onDelete: Cascade)

  @@map("advocate_ai_configs")
}

// — Consultations (live sessions with an advocate) —————

enum ConsultationStatus {
  LOBBY
  LIVE

```

```

ENDED
FAILED
// Requests to a human advocate
REQUESTED
ACCEPTED
DECLINED
EXPIRED
CANCELLED
}

enum TurnSpeaker {
    USER
    ADVOCATE
    SYSTEM
}

model Consultation {
    id                Int                @id @default(autoincrement())
    userId            Int                @map("user_id")
    // Kept (SET NULL) if the advocate is later deleted, so a saved consultation
    // never disappears; the advocate name is snapshotted for display.
    advocateId       Int?               @map("advocate_id")
    advocateName     String              @map("advocate_name")
    // Snapshot, so history still knows whether it was an AI or a human call if the advocate is
    // deleted.
    advocateKind     AdvocateKind       @default(AI) @map("advocate_kind")
    state             String
    language          String
    status            ConsultationStatus @default(LOBBY)
    consentAt         DateTime            @map("consent_at")
    model             String?
    aiConfigVersion   Int?                @map("ai_config_version")
    callId            String?             @map("call_id")
    startedAt         DateTime?           @map("started_at")
    endedAt           DateTime?           @map("ended_at")
    durationSec       Int                 @default(0) @map("duration_sec")
    endReason         String?             @map("end_reason")
    usage             Json?
    summary           Json?
    summaryStatus     String              @default("NONE") @map("summary_status")
    // Human consultations: what the client wrote, the request lifecycle, and the advocate's
    // notes.
    subject           String?
    requestedAt       DateTime?           @map("requested_at")
    respondedAt       DateTime?           @map("responded_at")
    declineReason     String?             @map("decline_reason")
    privateNotes      String?             @map("private_notes")
}

```

```

    sharedNotes    String?      @map("shared_notes")
    createdAt      DateTime      @default(now()) @map("created_at")

    user           User          @relation(fields: [userId], references: [id], onDelete: Cascade)
    advocate       Advocate?     @relation(fields: [advocateId], references: [id], onDelete:
SetNull)
    turns          ConsultationTurn[]

    @@index([userId, createdAt])
    @@index([status])
    @@map("consultations")
}

model ConsultationTurn {
  id              Int            @id @default(autoincrement())
  consultationId  Int            @map("consultation_id")
  speaker         TurnSpeaker
  // SPEECH = transcribed talk, SEARCH = a law lookup the advocate ran, NOTICE = system message
  kind           String          @default("SPEECH")
  text           String
  // ADVOCATE speech: every "Section N of Act" mentioned, with its verification status.
  // SEARCH: the passages retrieved (citation, source URL, similarity).
  citations       Json?
  atMs           Int             @default(0) @map("at_ms")
  createdAt      DateTime        @default(now()) @map("created_at")

  consultation    Consultation @relation(fields: [consultationId], references: [id], onDelete:
Cascade)

  @@index([consultationId, id])
  @@map("consultation_turns")
}

```

server/prisma/schema.prisma

Appendix B — Environment Configuration Reference

Every piece of environment-specific configuration (Section 3.1) is supplied through variables, never hard-coded (Section 6.4).

Variable	Purpose
PORT	Server listen port (default 5000)
DATABASE_URL	PostgreSQL connection string
REDIS_URL	Redis connection string — BullMQ job queue and rate limiting
JWT_SECRET	Signing secret for session tokens
OPENAI_API_KEY	OpenAI API key — chat completion, embeddings, Realtime voice
MSG91_AUTH_KEY / MSG91_TEMPLATE_ID	OTP SMS gateway credentials (Section 6.1)
GOOGLE_CLIENT_ID / GOOGLE_CLIENT_SECRET	Google OAuth login
CLIENT_URL / SERVER_URL	CORS allowlist (Section 6.3) and absolute links
RAG_INGEST_SECRET	Authenticates the statute-ingestion admin endpoint (Table 9.1)
ADMIN_EMAILS	Bootstrap list of administrator accounts
NODE_ENV	development / production / test — gates the test-database and credential guards (Section 5.1)
DB_USER / DB_PASSWORD / DB_NAME	Consumed by docker-compose.yml to configure the local Postgres container (Appendix D.1) — not read anywhere in server/src itself, since the Node process connects via DATABASE_URL instead
FIRECRAWL_API_KEY	Web-scraping provider used when ingesting a statute source into the Legal Knowledge Base (Table 9.1)
LOG_LEVEL	Pino log verbosity; defaults to silent under NODE_ENV=test (Section 5.1), info otherwise
DAILY_CONSULT_MINUTES	Per-user rolling 24-hour cap on AI voice-consultation minutes (Figure 2.9)
HUMAN_CALL_MAX_MINUTES	Maximum duration of a live human-advocate call before the sweeper ends it (Table 2.5b)
HUMAN_REQUEST_TTL_SECONDS / HUMAN_JOIN_TTL_SECONDS	How long an advocate has to respond to, then join, a human-consultation request before the sweeper marks it EXPIRED (Table 2.5b)
STUN_URLS / TURN_URLS	WebRTC STUN/TURN server lists for Connect Advocate call setup (Section 2.3.4's constraint on TURN budget)
TURN_SECRET / TURN_USERNAME / TURN_CREDENTIAL	TURN relay authentication — either a shared secret (time-limited credentials) or a fixed username/credential pair, depending on the TURN provider

Appendix B.1 — The Test Environment Template

No generic .env.example is tracked in the repository — server/.env itself is gitignored and created by hand from the table above. The one environment template that is tracked is server/.env.test.example, reproduced here in full because its comments explain two real decisions this report discusses elsewhere rather than merely listing variable names: the

credential guard that fixed defect #3 in Table 5.4, and why running the test suite can never silently send a real OTP SMS or hit a real database.

```
# Template for server/.env.test – copy to server/.env.test and fill in real
# local values. server/.env.test is gitignored (it holds a real DB password),
# this example file is the one tracked in git.
#
# System tests run the real Express app against a SEPARATE Postgres database
# (never your dev database) and mock the OpenAI client, so the API keys below
# can stay as placeholders.

PORT=3001
SERVER_URL=http://localhost:3001
CLIENT_URL=http://localhost:5173

# Point this at a database name that is NOT your dev database, e.g. suffix
# it with _test. `npm run test:db:setup` creates it and runs migrations.
DB_USER=postgres
DB_PASSWORD=postgres
DB_NAME=riskassessmentdb_test
DATABASE_URL="postgresql://postgres:postgres@localhost:5432/riskassessmentdb_test?schema=public"

# Separate logical Redis DB (index 1) so test jobs/keys never mix with dev's
# queue (index 0), even though both point at the same local Redis server.
REDIS_URL=redis://localhost:6379/1

JWT_SECRET=test_jwt_secret_for_automated_tests_only
OPENAI_API_KEY=test-openai-key-not-used-mocked-in-tests

# Deliberately blank, not omitted: env.ts's own dotenv.config() call falls
# back to server/.env for any variable *not already set* – omitting these
# here would let real MSG91 prod credentials (if your .env has them) leak
# into test runs, and otp.system.test.ts would call the live MSG91 API.
# Blank here keeps otp.service.ts's dev-mode path active in tests.
MSG91_AUTH_KEY=
MSG91_TEMPLATE_ID=
```

server/.env.test.example (verbatim, tracked in git)

This file is the credential guard named in Section 5.1 and Table 5.4 made concrete: before it existed, `server/.env.test` simply omitted the MSG91 keys, which let Node's config loader silently fall back to the real `server/.env` — including a real MSG91 key, if the developer happened to have one configured locally — and the OTP system test (Section 5.8) sent one live SMS during a test run before this was caught. Setting the keys explicitly to blank, rather than leaving them unset, is what keeps `otp.service.ts`'s dev-mode fallback active even when the developer's own `.env` is fully configured for production use.

Appendix C — API Endpoint Reference

Every route actually registered under `/api/v1`, read directly from the `ten.routes.ts` files rather than reconstructed from memory — the same source FR-14's auto-generated OpenAPI document (`/api-docs`) is built from. “Auth” of — means no authentication is required; “User” means any authenticated user; “Admin” means `requireAdmin` (Section 6.2) gates the route.

Table C.1 — Complete API Endpoint Reference

Module	Method	Path	Auth	Description
Users	POST	<code>/users/register</code>	—	Register a new account (email/password)

Module	Method	Path	Auth	Description
Users	POST	/users/login	—	Log in with email/password
Users	GET	/users/me	User	Get the authenticated user's profile
Auth	GET	/auth/google	—	Start the Google OAuth flow
Auth	GET	/auth/google/callback	—	Google OAuth callback
OTP	POST	/auth/otp/send	—	Send a phone OTP code
OTP	POST	/auth/otp/verify	—	Verify an OTP code and issue a JWT
Documents	POST	/documents/upload	User	Upload a PDF/JPG/PNG document
Documents	POST	/documents/text	User	Create a document from pasted text
Documents	GET	/documents/get-documents	User	List the caller's own documents
Documents	GET	/documents/:id/file	User	Download the original file
Documents	GET	/documents/:id	User	Get one document's details
Documents	PATCH	/documents/:id	User	Rename or toggle favourite
Documents	DELETE	/documents/:id	User	Delete a document
Analysis	POST	/analysis/run	User	Run or poll analysis for a document (Figure 2.7)
Legal Assistant	POST	/legal-agent/conversations	User	Create a conversation
Legal Assistant	GET	/legal-agent/conversations	User	List conversations
Legal Assistant	GET	/legal-agent/conversations/:id	User	Get one conversation
Legal Assistant	PATCH	/legal-agent/conversations/:id	User	Rename or change language
Legal Assistant	DELETE	/legal-agent/conversations/:id	User	Delete a conversation
Legal Assistant	POST	/legal-agent/conversations/:id/messages	User	Send a chat message (SSE stream, Figure 2.8)
Legal Assistant	POST	/legal-agent/conversations/:id/voice-messages	User	Send a voice message
Legal Assistant	GET	/legal-agent/messages/:messageId/audio	User	Get a reply's spoken audio
Legal Assistant	POST	/legal-agent/conversations/:id/documents	User	Attach a document to a conversation

Module	Method	Path	Auth	Description
RAG	POST	/rag/ingest	Admin	Ingest/re-ingest the legal knowledge base
RAG	GET	/rag/status	User	Get ingestion status
RAG	POST	/rag/ingest-statutes	Admin	Ingest official statute sources (Table 9.1)
RAG	GET	/rag/statutes	Admin	Get statute-ingestion status
Speech	POST	/speech/transcribe	User	Transcribe an audio clip
Advocates	GET	/advocates	User	List active advocates
Advocates	GET	/advocates/:slug	User	Get one advocate's public profile
Admin Advocates	GET	/admin/advocates	Admin	List all advocates
Admin Advocates	POST	/admin/advocates	Admin	Create an advocate
Admin Advocates	GET	/admin/advocates/:id	Admin	Get one advocate
Admin Advocates	PATCH	/admin/advocates/:id	Admin	Update an advocate
Admin Advocates	DELETE	/admin/advocates/:id	Admin	Delete an advocate
Admin Advocates	POST	/admin/advocates/:id/credentials	Admin	Add a credential
Admin Advocates	PATCH	/admin/advocates/:id/credentials/:credentialId	Admin	Update a credential
Admin Advocates	DELETE	/admin/advocates/:id/credentials/:credentialId	Admin	Delete a credential
Admin Advocates	PUT	/admin/advocates/:id/ai-config	Admin	Update the AI advocate's configuration
Admin Advocates	POST	/admin/advocates/:id/photo	Admin	Upload an advocate photo
Admin Advocates	DELETE	/admin/advocates/:id/photo	Admin	Remove an advocate photo
Admin Advocates	PUT	/admin/advocates/:id/account	Admin	Link a login account to an advocate
Admin Advocates	DELETE	/admin/advocates/:id/account	Admin	Unlink a login account
Consultations	GET	/consultations/options	User	Get available AI advocate options
Consultations	GET	/consultations	User	List the caller's consultations
Consultations	POST	/consultations	User	Create a consultation (AI advocate)
Consultations	GET	/consultations/:id	User	Get one consultation
Consultations	GET	/consultations/:id/turns	User	Poll consultation turns during a live call
Consultations	POST	/consultations/:id/connect	User	Start the AI voice call (Figure 2.9)

Module	Method	Path	Auth	Description
Consultations	POST	/consultations/:id/end	User	End a consultation
Consultations	DELETE	/consultations/:id	User	Delete a consultation
Human Consultations	POST	/human-consultations	User	Request a human-advocate consultation
Human Consultations	POST	/human-consultations/:id/cancel	User	Cancel a request
Human Consultations	POST	/human-consultations/:id/end	User	End a human consultation
Human Consultations	GET	/human-consultations/:id/ice	User	Get WebRTC ICE servers
Advocate Desk	GET	/advocate-desk/whoami	User	Check whether the account is a linked advocate
Advocate Desk	GET	/advocate-desk/me	User	Get the advocate's desk summary
Advocate Desk	POST	/advocate-desk/available	User	Toggle availability
Advocate Desk	GET	/advocate-desk/consultations/:id	User	Get one consultation (desk view)
Advocate Desk	POST	/advocate-desk/consultations/:id/accept	User	Accept a human-consultation request
Advocate Desk	POST	/advocate-desk/consultations/:id/decline	User	Decline a request
Advocate Desk	PUT	/advocate-desk/consultations/:id/notes	User	Update private/shared notes

Appendix C.1 — Endpoint-to-Use-Case Cross-Reference

A short index back from Table C.1's raw routes to the Use Case Specifications (Section 2.5.3.1) they implement, for a reader moving in the other direction — from “what does this endpoint do” to “what user-facing flow is it part of.”

Use Case	Primary endpoints (Table C.1)
UC-1 Register & Login	POST /users/register, POST /users/login, GET /auth/google, GET /auth/google/callback
UC-2 Upload / Paste Document	POST /documents/upload, POST /documents/text, POST /analysis/run
UC-3 View AI Risk Report	POST /analysis/run (poll), GET /documents/:id
UC-4 Ask Legal Question (Chat, RAG)	POST /legal-agent/conversations, POST /legal-agent/conversations/:id/messages
UC-5 Voice Consultation — AI Advocate	POST /consultations, POST /consultations/:id/connect, GET /consultations/:id/turns, POST /consultations/:id/end
UC-6 Request Human Advocate	POST /human-consultations, POST /human-consultations/:id/cancel, GET /human-consultations/:id/ice
UC-7 Accept / Decline Consultation	GET /advocate-desk/consultations/:id, POST .../accept, POST .../decline, PUT .../notes
UC-8 Manage Advocate Accounts	POST/PATCH/DELETE /admin/advocates, POST /admin/advocates/:id/credentials

Use Case	Primary endpoints (Table C.1)
UC-9 Verify Phone (OTP)	POST /auth/otp/send, POST /auth/otp/verify

Appendix D — Deployment and Installation Guide

Step-by-step instructions to set up a development copy of NyayMitra AI from a clean checkout, and the additional steps for a production build. Every command and script name below is copied from the real server/package.json and client/package.json files and from server/docker-compose.yml, not reconstructed.

D.1 Prerequisites

Requirement	Notes
Node.js	Used to run both the server (ts-node / compiled JS) and the client (Vite) toolchains
npm	Package manager; both server and client ship an npm-based package.json
PostgreSQL with the pgvector extension	Required for the RAG vector-similarity search (Section 2.5.1); the bundled docker-compose.yml uses the pgvector/pgvector:pg17 image
Redis	Backs the BullMQ analysis queue and the shared rate-limiter store (Section 3.1); docker-compose.yml uses redis:7-alpine with --appendonly yes
Docker and Docker Compose	Optional, but the simplest way to bring up PostgreSQL and Redis locally without installing either natively
OpenAI API key	Chat completion, embeddings and Realtime voice (Section 2.3, FR-7, FR-11)
MSG91 account and auth key	SMS OTP delivery for phone-based login (Section 6.1)

D.2 Local Development Setup

1. Clone the repository and change into the project root.
2. Create server/.env (gitignored — not tracked, since it holds real secrets) and fill in every variable listed in Appendix B. There is no generic .env.example to copy; the tracked template that exists, server/.env.test.example (Appendix B.1), is specifically for the test environment, not local development.
3. From the server directory, run “docker compose up -d” to start the db and redis containers defined in docker-compose.yml (PostgreSQL on port 5432, Redis on port 6379).
4. Install server dependencies: “cd server && npm install”. The postinstall script runs prisma generate automatically, so the Prisma client is ready immediately after install.
5. Apply the database schema: “npm run prisma:migrate”. This runs prisma migrate dev, which also creates the database on first run (Section 4.3).
6. Start the server: “npm run dev” (nodemon, restarts on file change) or “npm start” (ts-node, single run).
7. In a second terminal, install and start the client: “cd client && npm install && npm run dev”. Vite prints a local URL (typically http://localhost:5173).
8. Open the printed client URL in a browser. The client calls the API at the address in VITE_API_BASE_URL, falling back to http://localhost:3000/api/v1 if that variable is not set.

D.3 Available npm Scripts

Server (server/package.json):

Script	Purpose
start	Run the server once with ts-node (no file-watch restart)
dev	Run the server under nodemon for local development
build	Compile TypeScript to JavaScript with tsc
postinstall	Runs automatically after npm install; regenerates the Prisma client
prisma:generate / prisma:studio	Regenerate the Prisma client / open Prisma's local data browser
prisma:migrate / prisma:deploy	Create and apply a new migration (dev) / apply pending migrations only (production, Section 4.3)
rag:eval	Run the RAG retrieval-quality evaluation script
test / test:unit / test:system / test:coverage	Run the Vitest suite — all tests, unit only, system only, or with a coverage report (Section 5.1)
test:db:setup / test:redis:flush	Prepare an isolated test database / clear the test Redis instance before a test run

Client (client/package.json):

Script	Purpose
dev	Start the Vite development server with hot module reload
build	Type-check (tsc -b) then produce a static production bundle (vite build)
lint	Run ESLint over the client source
preview	Serve the production build locally for a final check before deployment
test / test:watch / test:coverage	Run the client's Vitest suite (Section 5.1)

D.4 Running the Test Suite

`npm test` (server or client directory) runs the full Vitest suite described in Section 5.1. The server's `test:db:setup` and `test:redis:flush` scripts point the suite at an isolated database and Redis instance first, so a test run never reads or writes development data.

D.5 Production Build and Deployment

1. Build both projects: “`npm run build`” in server (tsc, output under `dist/`) and in client (tsc -b && vite build, output under `dist/`).
2. Apply migrations with “`npm run prisma:deploy`”, not `prisma:migrate` — `prisma migrate deploy` applies only already-committed migrations and never prompts interactively, which matters for a non-interactive deployment pipeline (Section 4.3).
3. Start the compiled server (“`node dist/server.js`”) under a process supervisor such as `pm2` or a `systemd` unit, so the process restarts automatically after a crash or host reboot. Multi-instance clustering and a reverse-proxy/load-balancer tier are not yet implemented (Section 8 — Future Scope).
4. Serve the client's `dist/` folder as static files from any static host or CDN, with `VITE_API_BASE_URL` set at build time to the deployed server's public URL.

5. Set `NODE_ENV=production` on the server so the test-database and development-only credential guards described in Section 5.1 are disabled.

Appendix E — User Manual

A walkthrough of the application from a first-time user's point of view, cross-referenced to the UI screens already shown in Section 3.4 (Figures 3.3-3.8) rather than repeating the same screenshots here.

E.1 Registering and Signing In

A new user opens the Home screen (Figure 3.3) and selects Register. Registration accepts either an email and password or a phone number; a phone number must be verified with a one-time password sent by SMS (FR-1, Section 6.1) before the account can sign in. Returning users sign in from the Login screen (Figure 3.4) with whichever credential they registered with. A successful sign-in issues a JWT session token (Section 6.1) that the client attaches to every subsequent API call.

E.2 Uploading a Document and Reading the Risk Report

1. From the Dashboard (Figure 3.5), select Upload Document and choose a PDF, DOCX, or image file (FR-5).
2. The document is queued for background analysis (Figure 2.7); the Dashboard shows its status as Pending, Processing, or Completed.
3. If the file is a scanned image with no extractable text, the OCR fallback (Section 2.3.1, FR-5) runs automatically before analysis — no separate action is needed.
4. Once Completed, open the document to view the Risk Report (Figure 3.6): an overall risk score, a plain-language summary, and a clause-by-clause breakdown with each clause's own risk level and explanation (FR-6, FR-8).
5. Reports can be revisited at any time from the Dashboard's document list; no re-upload is needed to read a previously completed report.

E.3 Asking the Legal Assistant

The Legal Assistant screen (Figure 3.7) accepts a free-text legal question in English or Hindi (FR-9, FR-10) and streams back an answer token-by-token (Figure 2.8). Every answer is grounded in retrieved statute text (RAG, Section 2.5.4) and shows the source citations it was built from, so a user can open the underlying statute (Table 9.1) rather than trust the summary alone. A conversation's earlier turns stay visible so follow-up questions can refer back to them.

E.4 Requesting a Consultation (Connect Advocate)

The Connect Advocate screen (Figure 3.8) offers two paths. The first is an immediate AI voice consultation (FR-11, FR-12): the user selects an AI advocate persona and starts a live voice call (Figure 2.9) answered by the configured AI model in real time. The second is a request routed to a human advocate (FR-13): the user describes the matter, the request appears on the matching advocate's Advocate Desk, and the advocate accepts or declines it from there; an accepted request becomes a live audio/video call over WebRTC.

E.5 Troubleshooting

Symptom	Likely cause / what to do
OTP SMS never arrives	Check the phone number was entered in the correct format; request a new OTP after the first one's validity window expires (Section 6.1)
Document stays “Processing” for a long time	Large scanned files take longer because of the OCR fallback step (Section 2.3.1); if it exceeds a few minutes, re-check the file is a readable PDF/image rather than corrupted
Legal Assistant answer has no citations	The retrieved context did not contain a matching statute passage; the answer is still grounded-first by design (Section 2.5.4), so a missing citation is a signal to rephrase the question rather than trust an uncited claim
Voice call does not connect	WebRTC consultations require microphone permission in the browser and a working network path for ICE negotiation (Appendix C, GET /human-consultations/:id/ice)

Appendix F — Error Handling and Status Code Reference

How the server actually reports failure to the client, read directly from the real middleware (server/src/common/middleware/error.middleware.ts, validate.middleware.ts, rateLimit.middleware.ts) rather than reconstructed from convention.

F.1 Error Response Envelope

Every error response, from any route, shares one JSON shape: { success: false, message: string }. A known, deliberately-thrown `AppError` (message + statusCode set by the code that threw it) is returned as-is; any other, unexpected error is logged server-side via Pino (Section 6.4) and collapsed to a generic 500 “Internal Server Error” — the client is never shown a raw stack trace or exception message for an unanticipated failure, in any environment, since the stack is simply never placed in the response body to begin with (not a `NODE_ENV`-gated redaction, Section 6.4’s redact-path logger behaviour is the closer analogue).

One defect was found while documenting this path: `app.ts` registers a second, inline 500 handler after the central `errorHandler` and the 404 catch-all. Because the central handler already responds to every route error first, this second handler is unreachable dead code in practice — but unlike the central handler it would include the raw `err.message` in its response if it were ever reached, and it is not `NODE_ENV`-gated either. It is flagged here rather than silently left out, consistent with this report’s practice of naming defects found rather than only describing the system as designed (Section 5.4).

F.2 HTTP Status Code Reference

Status	Meaning in this system	Representative source
400 Bad Request	Zod schema validation failure, or a business-rule check (invalid id, missing file, bad consent)	validate.middleware.ts; document.service.ts
401 Unauthorized	Missing/invalid session token, or a wrong password/OTP	auth.middleware.ts; user.service.ts; otp.service.ts
403 Forbidden	Authenticated but not permitted (non-admin on an admin route; not a call participant)	admin.middleware.ts; human.service.ts
404 Not Found	Referenced entity does not exist, or the route itself is unmatched	user.service.ts; document.controller.ts; app.ts catch-all

Status	Meaning in this system	Representative source
409 Conflict	Duplicate email/mobile on registration, or a resource in the wrong state for the action (advocate already busy)	user.service.ts; human.service.ts; advocate.service.ts
413 Payload Too Large	Uploaded file exceeds the configured size limit	document upload route
429 Too Many Requests	Redis-backed rate limiter triggered, or an OTP-specific cooldown/attempt cap	rateLimit.middleware.ts; otp.service.ts
502 Bad Gateway	The AI voice provider could not start a live call	consultation.service.ts
500 Internal Server Error	Any error not explicitly thrown as an AppError — logged, never detailed to the client	error.middleware.ts (default branch)

F.3 Validation Error Format

A Zod schema failure is converted to a 400 AppError whose message names only the first failing field, not the full issue list — simpler for a form to show inline (Section F.4) at the cost of reporting one problem at a time rather than every invalid field in a single response.

```
{
  "success": false,
  "message": "password: Password must be at least 8 characters"
}
```

Example — 400 response from the validate() middleware (actual envelope shape)

F.4 Client-Side Friendly Error Mapping

apiErrorMessage() (client/src/services/apiError.ts) maps a response's HTTP status to a fixed, friendly message via a BY_STATUS table, preferring the server's own message field when present except for any 5xx status, which is always generalised so an internal error never reaches the UI verbatim (the standing friendly-errors rule this project follows). A network failure with no response at all is distinguished from a server error and shown its own message; a 401 on an already-authenticated request additionally clears the stored session and fires a session-expired event rather than just showing an error. FormError.tsx renders the result inline on forms; quick, non-form actions use a toast instead (Section E).

Table F.4a quotes BY_STATUS verbatim — the exact strings a user sees, not a paraphrase — because the source file's own opening comment states the rule this table exists to enforce: “Screens must show this instead of err.message, which for axios is text like 'Request failed with status code 404'.”

Table F.4a — Exact Friendly-Error Text by Status

Status	Exact text shown to the user
400	Some of the details don't look right. Please check them and try again.
401 (session-expired case)	Your session has expired. Please sign in again.
403	You don't have permission to do that.
404	We couldn't find what you were looking for.
409	That can't be done right now. Please refresh and try again.
413	That file is too large. Please choose a smaller one.
429	You're going a little fast. Please wait a moment and try again.
502	A service we depend on is unavailable. Please try again in a moment.

Status	Exact text shown to the user
503	NyayMitra is temporarily unavailable. Please try again in a moment.
504	This is taking longer than expected. Please try again.
Any other 5xx	Something went wrong on our side. Please try again in a moment.
No response — offline/DNS/CORS	We can't reach NyayMitra right now. Please check your internet connection and try again.
No response — timeout (ECONNABORTED/ETIMEDOUT)	This is taking longer than expected. Please try again.

Two exceptions to the status-only lookup, both in `messageForStatus()` (`apiError.ts`): a 400/403/404/409 prefers the server's own message field over the table above when one is present — the server-side messages are already written to be human-readable (Appendix F.1's `AppError` convention), so there is nothing to improve on; and 401/403/404/409 responses from the three credential routes (login, register, OTP) are never treated as a session-expiry even though 401 normally means one, since a wrong password is not an expired session.

F.5 Rate Limiting

Three Redis-backed limiters apply depending on the route: a 30-per-15-minutes limiter on AI-calling routes, a 100-per-15-minutes general limiter, and a 300-per-15-minutes limiter on the analysis-status poll endpoint (Section 2.3.4 — polling by design, so it needs a looser cap than a normal write route). All three key on the authenticated user's id where available, falling back to IP address for unauthenticated requests, and all send the standard `RateLimit-*` response headers rather than the older non-standard `X-RateLimit-*` headers. Exceeding any of them returns 429 with the same `{ success:false, message }` envelope as every other error (Section F.1), not a bare empty response.

Appendix G — Sample API Requests

Five complete request/response examples, end to end, for the core golden path (register → upload → analyse → ask a question). Every path, field name, and status code below is copied from the real route and schema files (`user.routes.ts/user.schema.ts`, `document.routes.ts`, `analysis.route.ts/analysis.schema.ts`, `legal-agent.routes.ts/legal-agent.schema.ts`) rather than written from memory — the base path for every example is `/api/v1` (Appendix C).

G.1 Register

```
curl -X POST https://<host>/api/v1/users/register \
  -H "Content-Type: application/json" \
  -d '{"name":"Asha Rao","email":"asha@example.com","password":"correcthorse1"}'

# 201 Created
{ "success": true, "data": { "id": 501, "name": "Asha Rao", "email": "asha@example.com", "role": "USER" } }
```

G.2 Login

```
curl -X POST https://<host>/api/v1/users/login \
  -H "Content-Type: application/json" \
  -d '{"email":"asha@example.com","password":"correcthorse1"}'

# 200 OK
{ "success": true, "data": { "token": "<JWT>", "user": { "id": 501, "name": "Asha Rao", "role": "USER" } } }
```

G.3 Upload a Document

```
curl -X POST https://<host>/api/v1/documents/upload \
  -H "Authorization: Bearer <JWT>" \
  -F "file=@rental-agreement.pdf"

# 201 Created
{ "success": true, "data": { "id": 482, "status": "pending", "filePath": "uploads/482-rental-
agreement.pdf" } }
```

G.4 Run Analysis

```
curl -X POST https://<host>/api/v1/analysis/run \
  -H "Authorization: Bearer <JWT>" -H "Content-Type: application/json" \
  -d '{"documentId": 482}'

# First call (status was pending) – 202 Accepted, job enqueued (Figure 2.7)
{ "success": true, "data": { "status": "processing" } }

# A later call, once the worker has finished – 200 OK (Section 7.1's full shape)
{ "success": true, "data": { "status": "completed", "riskScore": 60, "riskLevel": "Medium",
"...": "..." } }
```

G.5 Ask the Legal Assistant

A conversation must exist first (POST /legal-agent/conversations); the example below sends into an existing one and receives a Server-Sent Events stream rather than a single JSON body.

```
curl -N -X POST https://<host>/api/v1/legal-agent/conversations/77/messages \
  -H "Authorization: Bearer <JWT>" -H "Content-Type: application/json" \
  -d '{"content": "My landlord is refusing to return my security deposit. What can I do?"}'

# 200 OK, Content-Type: text/event-stream – one SSE "data:" frame per token, e.g.
data: {"token": "Under"}
data: {"token": " most"}
data: {"token": " state"}
...
data: {"done": true, "citations": [{"act": "Model Tenancy Act, 2021", "section": "Section 10(3)"}]}
```

Appendix H — List of Figures and Tables

LIST OF FIGURES

Figure	Caption
2.1	PERT Chart (Network Diagram)
2.2	Entity-Relationship Diagram
2.3	DFD: Context Diagram (Level 0)
2.4	DFD: Level 1
2.5	DFD: Level 2 (Decomposition of Process 3.0)
2.6	Use Case Diagram
2.7	Sequence Diagram: Document Analysis
2.8	Sequence Diagram: Legal Assistant Chat (RAG)
2.9	Sequence Diagram: Connect Advocate (Live AI Consultation)
2.10	Sequence Diagram: Authentication (Phone OTP Login)
2.11	State Transition Diagrams
2.12	Activity Diagram: Upload Document to Risk Report
3.1	System Architecture
3.2	Module Decomposition
3.3	Screen: Home (Landing Page)
3.4	Screen: Login / Register
3.5	Screen: Dashboard
3.6	Screen: Document Risk Report
3.7	Screen: Legal Assistant
3.8	Screen: Connect Advocate
3.9	Component Diagram: Module Layering Pattern

LIST OF TABLES

Table	Caption
1.1	Objective-to-Functional-Requirement Traceability
2.1	Development Phases and Schedule
2.1a	Functional Requirement Acceptance Criteria
2.1b	Non-Functional Requirement Targets
2.2	Cost and Effort Estimation
2.2a	Risk Register
2.3	Hardware and Software Requirements
2.4 / 2.4a	Data Dictionary (core / remaining entities)
2.5a / 2.5b	Consultation Status Transition Reference (AI / Human advocate path)
3.1	Key Uniqueness Constraints

Table	Caption
3.2	Custom Hooks Reference (Client)
4.1	Access Rights by Role
5.1	Test Suite Summary
5.2	Testing Plan
5.3	Representative Test Cases
5.4	Defects Found and Resolved
5.5	Automated Test Coverage by Module
6.1	Security Controls Summary
6.2	OWASP Top 10 (2021) Mapping
7.1a	Risk Badge Colour Tokens (Light / Dark)
9.1	Statute Sources Ingested into the Legal Knowledge Base
C.1	Complete API Endpoint Reference
F.4a	Exact Friendly-Error Text by Status

11. Glossary

Term	Definition
API	Application Programming Interface — the set of endpoints a client uses to talk to the server
BullMQ	A Redis-backed job queue library for Node.js, used for the background analysis worker (Section 3.1)
CORS	Cross-Origin Resource Sharing — the browser rule controlling which origins may call the API
CSP	Content Security Policy — an HTTP header restricting which scripts/resources a page may load
DFD	Data Flow Diagram — a model of how data moves between processes and stores (Section 2.5.2)
ER Diagram	Entity-Relationship Diagram — a model of the database's tables and their relationships (Section 2.5.1)
FR / NFR	Functional / Non-Functional Requirement (Section 2.3)
IDOR	Insecure Direct Object Reference — a vulnerability class where one user can access another user's data by guessing an id (Sections 2.3.1, 6.2)
JWT	JSON Web Token — a signed token used to represent an authenticated session (Section 6.1)
LLM	Large Language Model — the class of AI model (e.g. OpenAI's GPT models) used for summarisation, chat, and voice
OCR	Optical Character Recognition — extracting text from a scanned or photographed image (Section 2.3.1, FR-5)
OTP	One-Time Password — a short-lived code used for phone-based login (Section 6.1)
OWASP	Open Web Application Security Project — publisher of the Top 10 web security risks list
pgvector	A PostgreSQL extension adding vector similarity search, used for RAG retrieval (Section 2.5.1)
PERT	Program Evaluation and Review Technique — a project-scheduling network diagram (Section 2.2, Figure 2.1)
RAG	Retrieval-Augmented Generation — grounding an AI's answer in retrieved source text rather than its own recollection (Section 2.5.4, Figure 2.8)
Redis	An in-memory data store used here as both the BullMQ job broker and the rate-limiter backing store (Section 3.1)
REST	Representational State Transfer — the conventional HTTP API style used by this system's synchronous endpoints
SDLC	Software Development Life Cycle
SRS	Software Requirement Specification (Section 2.3)
SSE	Server-Sent Events — a one-way streaming HTTP response, used for the Legal Assistant's token-by-token reply (Figure 2.8)
TLS	Transport Layer Security — the encryption protocol underlying HTTPS
UML	Unified Modeling Language — the notation used for the Use Case and Sequence diagrams (Section 2.5.3, 2.5.4)

Term	Definition
Vitest	The JavaScript/TypeScript test framework used for both unit and system testing (Section 5.1)
WebRTC	Web Real-Time Communication — the browser API/protocol used for the Connect Advocate audio/video call (Section 3.1)
Zod	A TypeScript-first schema validation library used for every request-input check (Section 4.2, 6.3)
BCNF	Boyce-Codd Normal Form — the normalisation level targeted for the relational schema (Section 3.3)
Clustered index	A table's storage physically ordered by its primary key (Postgres applies this implicitly via the primary-key B-tree) (Section 3.3)
CVSS	Common Vulnerability Scoring System — a standard scale for rating the severity of a security defect (Section 6.5)
Elicitation	The requirements-gathering step of identifying stakeholder needs before they are written as FRs/NFRs (Section 2.3)
ICE (WebRTC)	Interactive Connectivity Establishment — the protocol WebRTC uses to negotiate a direct media path between two peers (Appendix C, GET /human-consultations/:id/ice)
Idempotent	An operation that produces the same result no matter how many times it is repeated — a property BullMQ job handlers are designed to have, since a crashed job may be retried (Section 5.4)
JSDoc	The comment convention used to document the server's TypeScript functions, picked up by the OpenAPI generator (FR-14)
Migration (database)	A version-controlled, incremental change to the database schema, applied in order (Section 4.3; files under server/prisma/migrations)
pm2 / systemd	Process supervisors that restart a Node.js server automatically after a crash or reboot (Appendix D.5)
Prisma	The TypeScript ORM used for all database access in this project; schema.prisma (Appendix A) is its single source of truth for the schema
Rate limiting	Restricting how many requests a single client may make in a time window, to blunt brute-force and denial-of-service attempts (Section 6.1, 6.2)
Repository pattern	An architectural rule that only one layer (*.repository.ts) is allowed to import the database client, so every other layer stays storage-agnostic (Section 3.7, 4.4)
Risk register	A table listing a project's foreseeable risks with their likelihood, impact and mitigation (Table 2.2a)
Seed data	A small, fixed set of rows inserted into a fresh database so the application has something to show immediately after setup
Surrogate key	A primary key with no business meaning (e.g. a UUID or auto-increment id), used in place of a natural key (Section 3.3)
Token stream	A completion delivered incrementally, token-by-token, rather than as one final block of text — how the Legal Assistant's reply is rendered (Figure 2.8)
UUID	Universally Unique Identifier — a common surrogate-key format (Appendix A's schema in fact uses plain auto-incrementing integers instead, Table 2.4)
Watchdog / health check	A periodic probe confirming a running process or dependency is still responsive (noted as a Future Scope item, Section 8)
Feature-sliced architecture	Organising frontend code by domain (one folder per feature, each owning its own components/API calls/hooks) rather than by technical layer — the client's own structure (Section 3.2.1), in deliberate contrast to the server's layered module pattern (Figure 3.9)

Term	Definition
Firecrawl	The third-party web-scraping service used when ingesting a new statute source into the Legal Knowledge Base (Appendix B, Table 9.1)
Redux Toolkit	The state-management library backing the client's single global store (one auth slice); every other feature keeps its data as local state in a custom hook rather than in this store (Section 3.2.1)
React Context	A React built-in for passing data through the component tree without prop-drilling; used in this client for exactly one concern, the light/dark theme, and deliberately not for server data (Section 3.2.1)
Sweeper	An in-process timer (setInterval, not an external cron job) that periodically scans for consultations stuck past a timeout and moves them to EXPIRED (Table 2.5b) — the mechanism behind HUMAN_REQUEST_TTL_SECONDS and HUMAN_JOIN_TTL_SECONDS (Appendix B)
Tesseract.js	The open-source OCR library used for the server's OCR fallback (Section 3.6, Procedure 5); no cloud OCR API is used
TTL	Time To Live — how long a value or request stays valid before it expires automatically (Appendix B's HUMAN_REQUEST_TTL_SECONDS, HUMAN_JOIN_TTL_SECONDS)
STUN	Session Traversal Utilities for NAT — a lightweight server that helps two WebRTC peers discover a usable address to connect directly (Section 2.3.4); used alone when a direct connection is possible
TURN	Traversal Using Relays around NAT — a relay server WebRTC falls back to when a direct (STUN-only) connection cannot be established, e.g. on a strict corporate or mobile network (Appendix B)
AppError	The single generic error class every deliberately-thrown server error uses (message + HTTP status code); there is no NotFoundError/UnauthorizedError subclass hierarchy (Appendix F.1)
bcrypt	The password-hashing algorithm used for both User.password and the OTP code hash (OtpVerification.codeHash) — neither is ever stored or logged in plaintext (Section 6.1)
Docker Compose	Defines the local development stack's PostgreSQL and Redis containers (server/docker-compose.yml, Appendix D.1) — not used for the server or client process themselves, which run directly on the host
pino-http	Express middleware that logs every HTTP request/response through the same structured Pino logger as the rest of the server, with the same redaction rules applied (Section 6.4)
Supertest	The HTTP-assertion library system tests use to drive the real Express app (request(app).post(...)) without a separately running server process (Section 5.1, 5.8)
Vite	The client's build tool and development server (Appendix D.3) — npm run dev starts it, npm run build produces the static production bundle
Helmet	The Express middleware supplying the baseline HTTP security headers verified by TC-14 (Table 5.3) and listed in Table 6.1
Multer	The Express file-upload middleware behind POST /documents/upload's multipart handling (Appendix G.3) — upload.single("file") is its real field name
ioredis	The Redis client library BullMQ and the rate limiters are built on (Appendix F.5); REDIS_URL (Appendix B) is the connection string it reads
Axios	The HTTP client library the client's shared request instance is built on (client/src/services/api.ts, Section 3.2.1) — every *Api.ts wrapper calls through this one instance
Express	The Node.js web framework the entire server API is built on — every route in Appendix C is an Express router

Term	Definition
Tailwind CSS	The utility-class CSS framework the client's styling — including the risk badge tokens in Table 7.1a and the dark-mode variants throughout — is built on
TypeScript	The statically-typed superset of JavaScript both the server and client are written in; the Prisma client (Appendix A) and every Zod schema (Section 4.2) depend on its type system
Node.js	The JavaScript runtime the server process runs on (Appendix D.1's version requirement, Appendix D.5's deployment target)
npm	The package manager used to install, build, and run both the server and client (Appendix D.3's script reference)
CRUD	Create, Read, Update, Delete — the conventional shape of the ordinary, non-streaming REST endpoints in Appendix C (e.g. the Documents module)
Environment variable	A configuration value supplied to the running process rather than written into source code (Appendix B) — the mechanism behind every credential, port, and tunable limit in this system (Section 6.4)

COMPLETE SOURCE CODE — CORE MODULES

Supplementary material, placed after the Glossary and outside the 100–125 page project-documentation count the MCSP-232 guideline specifies (page 12: “excluding coding”). The four module groups below — Authentication/OTP, Document ingestion, the Analysis pipeline, and RAG grounding — are reproduced here in full, file by file, exactly as implemented; every other module in Figure 3.2 is already documented through the Use Case Specifications, sequence diagrams, procedural-design pseudocode, and the representative extracts earlier in this report, and is additionally available in full, runnable form on the CD attached to this report's last page.

I.1 Authentication / OTP Module (Complete)

Six files: routing, request validation, the controller, data access, the SMS gateway client, and the service that ties them together.

```
import { Router } from "express";
import { sendOtpHandler, verifyOtpHandler } from "../otp.controller";
import { validate } from "../../common/middleware/validate.middleware";
import { sendOtpBodySchema, verifyOtpBodySchema } from "../otp.schema";

const router = Router();

router.post("/send", validate({ body: sendOtpBodySchema }), sendOtpHandler);
router.post("/verify", validate({ body: verifyOtpBodySchema }), verifyOtpHandler);

export default router;
```

server/src/modules/otp/otp.routes.ts

```
import { z } from "zod";
import { phoneSchema } from "../../common/schemas/common.schema";

export const sendOtpBodySchema = z.object({
  phone: phoneSchema,
});

export const verifyOtpBodySchema = z.object({
  phone: phoneSchema,
  code: z
    .string({ error: "Enter the code you received" })
    .trim()
    .regex(/^\\d{6}$/, "Enter the 6-digit code you received"),
});
```

server/src/modules/otp/otp.schema.ts

```
import { Request, Response, NextFunction } from "express";
import * as otpService from "../otp.service";

export const sendOtpHandler = async (req: Request, res: Response, next: NextFunction) => {
  try {
```

```

    const result = await otpService.sendOtp(req.body.phone);
    res.status(200).json({ success: true, message: "OTP sent", data: result });
  } catch (err) {
    next(err);
  }
};

export const verifyOtpHandler = async (req: Request, res: Response, next: NextFunction) => {
  try {
    const result = await otpService.verifyOtp(req.body.phone, req.body.code);
    res.status(200).json({ success: true, data: result });
  } catch (err) {
    next(err);
  }
};

```

server/src/modules/otp/otp.controller.ts

```

import { prisma } from "../../config/db";

export const createOtp = async (phone: string, codeHash: string, expiresAt: Date) => {
  return prisma.otpVerification.create({
    data: { phone, codeHash, expiresAt },
  });
};

export const findLatestOtpForPhone = async (phone: string) => {
  return prisma.otpVerification.findFirst({
    where: { phone, consumed: false },
    orderBy: { createdAt: "desc" },
  });
};

export const incrementAttempts = async (id: number) => {
  return prisma.otpVerification.update({
    where: { id },
    data: { attempts: { increment: 1 } },
  });
};

export const markConsumed = async (id: number) => {
  return prisma.otpVerification.update({
    where: { id },
    data: { consumed: true },
  });
};

```

```
});  
};
```

server/src/modules/otp/otp.repository.ts

```
import { env } from "../../config/env";  
import { logger } from "../../config/logger";  
  
export const sendOtpSms = async (phone: string, code: string): Promise<void> => {  
  if (!env.MSG91_AUTH_KEY || !env.MSG91_TEMPLATE_ID) {  
    // Deliberately a plain string, not a structured `{ code }` field – the  
    // logger redacts any `code` property by default (see logger.ts), but  
    // this line only ever fires in dev/test mode and exists specifically  
    // so a developer can see the code without the API response's devCode.  
    logger.info(`[DEV OTP] ${phone}: ${code}`);  
    return;  
  }  
  
  const res = await fetch("https://api.msg91.com/api/v5/flow/", {  
    method: "POST",  
    headers: {  
      "Content-Type": "application/json",  
      authkey: env.MSG91_AUTH_KEY,  
    },  
    body: JSON.stringify({  
      template_id: env.MSG91_TEMPLATE_ID,  
      recipients: [  
        {  
          mobiles: phone.replace(/^\\+/, ""),  
          otp: code,  
        },  
      ],  
    })),  
  });  
  
  if (!res.ok) {  
    const body = await res.text();  
    logger.error({ status: res.status, body }, "MSG91 send failed");  
    throw new Error("Failed to send OTP SMS");  
  }  
};
```

server/src/modules/otp/otp.sms.ts

```

import crypto from "crypto";
import bcrypt from "bcrypt";
import * as otpRepo from "../otp.repository";
import * as userRepo from "../user/user.repository";
import { AppError } from "../../common/errors/AppError";
import { generateToken } from "../../common/utills/jwt";
import { sendOtpSms } from "../otp.sms";
import { env } from "../../config/env";
import { toSafeUser } from "../user/user.mapper";

const RESEND_COOLDOWN_MS = 30 * 1000;
const OTP_TTL_MS = 5 * 60 * 1000;
const MAX_ATTEMPTS = 5;

// Phone format is validated at the route (otp.schema.ts, shared with
// registration) before this ever runs – only the business rules
// (cooldown, expiry, attempt limits) belong here.
export const sendOtp = async (phone: string) => {
  const latest = await otpRepo.findLatestOtpForPhone(phone);
  if (latest && Date.now() - latest.lastSentAt.getTime() < RESEND_COOLDOWN_MS) {
    throw new AppError("A code was sent recently. Please wait a moment before requesting
another.", 429);
  }

  const code = crypto.randomInt(100000, 1000000).toString();
  const codeHash = await bcrypt.hash(code, 10);
  const expiresAt = new Date(Date.now() + OTP_TTL_MS);

  await otpRepo.createOtp(phone, codeHash, expiresAt);
  await sendOtpSms(phone, code);

  const isDevMode = !env.MSG91_AUTH_KEY || !env.MSG91_TEMPLATE_ID;
  return isDevMode ? { success: true, devCode: code } : { success: true };
};

export const verifyOtp = async (phone: string, code: string) => {
  const otp = await otpRepo.findLatestOtpForPhone(phone);
  if (!otp) {
    throw new AppError("This code was already used, or no code was requested for this number.
Please request a new one.", 400);
  }

  if (otp.expiresAt.getTime() < Date.now()) {
    throw new AppError("This code has expired. Please request a new one.", 400);
  }
}

```

```

    if (otp.attempts >= MAX_ATTEMPTS) {
      throw new AppError("Too many wrong attempts. Please request a new code.", 429);
    }

    const isMatch = await bcrypt.compare(code, otp.codeHash);
    if (!isMatch) {
      await otpRepo.incrementAttempts(otp.id);
      throw new AppError("That code isn't correct. Please check it and try again.", 401);
    }

    await otpRepo.markConsumed(otp.id);

    let user = await userRepo.findUserByPhone(phone);
    if (!user) {
      user = await userRepo.createUser({
        name: "Mobile User",
        phone,
        password: "OTP_AUTH_USER",
      });
    }

    const token = generateToken({ id: user.id, email: user.email });

    return { user: toSafeUser(user), token };
  };

```

server/src/modules/otp/otp.service.ts

I.2 Document Module (Complete)

Five files: schema validation, routing, the controller, data access, and the service — the upload/paste entry point for FR-4.

```
import { z } from "zod";

// Same 50-character minimum document.service.ts always enforced – moved
// here so it's caught before a request even reaches the service/database,
// and reported the same way every other route's bad input is.
export const createTextBodySchema = z.object({
  content: z
    .string({ error: "Pasted text is required" })
    .trim()
    .min(50, "Pasted text is too short. Please paste at least 50 characters."),
});

export const updateDocumentBodySchema = z
  .object({
    title: z
      .string()
      .trim()
      .min(1, "Title cannot be empty")
      .max(120, "Title must be 120 characters or fewer")
      .optional(),
    isFavorite: z.boolean({ error: "isFavorite must be true or false" }).optional(),
  })
  .refine((data) => data.title !== undefined || data.isFavorite !== undefined, {
    message: "Nothing to update – send a title or isFavorite",
  });
```

server/src/modules/document/document.schema.ts

```
import { Router } from "express";
import {
  createTextDocumentController,
  getDocuments,
  uploadDoc,
  getDocumentByIdHandler,
  getDocumentFileHandler,
  updateDocumentHandler,
  deleteDocumentHandler,
} from "../document.controller";
import { upload } from "../../config/multer";
```

```

import { authMiddleware } from "../../common/middleware/auth.middleware";
import { standardRateLimiter } from "../../common/middleware/rateLimit.middleware";
import { validate } from "../../common/middleware/validate.middleware";
import { idParamSchema } from "../../common/schemas/common.schema";
import { createTextBodySchema, updateDocumentBodySchema } from "./document.schema";

const router = Router();
const idParams = validate({ params: idParamSchema() });

router.post(
  "/upload",
  authMiddleware,
  standardRateLimiter,
  upload.single("file"),
  uploadDoc
);

router.post(
  "/text",
  authMiddleware,
  standardRateLimiter,
  validate({ body: createTextBodySchema }),
  createTextDocumentController
);
router.get("/get-documents", authMiddleware, getDocuments);
router.get("/:id/file", authMiddleware, idParams, getDocumentFileHandler);
router.get("/:id", authMiddleware, idParams, getDocumentByIdHandler);
router.patch(
 ("/:id",
  authMiddleware,
  standardRateLimiter,
  idParams,
  validate({ body: updateDocumentBodySchema }),
  updateDocumentHandler
);
router.delete("/:id", authMiddleware, standardRateLimiter, idParams, deleteDocumentHandler);

export default router;

```

server/src/modules/document/document.routes.ts

```

import fs from "fs";
import path from "path";
import { Request, Response, NextFunction } from "express";

```

```

import { createTextDoc, uploadDocument, updateDocument, deleteDocument } from
"./document.service";
import { User } from "../../types/userType";
import { getUserDocuments, getDocumentById } from "./document.repository";
import { extractTextFromPDF } from "../../common/utils/pdf";
import { AppError } from "../../common/errors/AppError";
import { logger } from "../../config/logger";

const CONTENT_TYPE_BY_EXT: Record<string, string> = {
  ".pdf": "application/pdf",
  ".png": "image/png",
  ".jpg": "image/jpeg",
  ".jpeg": "image/jpeg",
};

interface AuthRequest extends Request {
  user?: User;
}

export const uploadDoc = async (req: Request, res: Response, next: NextFunction) => {
  try {

    const authReq = req as AuthRequest;

    if(!authReq.user || !authReq?.user?.id) {
      throw new AppError("User not authenticated", 401);
    }

    const userId: number = authReq.user?.id;

    if (!authReq.file) {
      throw new AppError("No file uploaded", 400);
    }

    const result = await uploadDocument(userId, authReq.file.path);

    res.status(201).json({
      success: true,
      data: result,
    });
  } catch (err) {
    logger.error({ err }, "Error in uploadDoc controller");
    next(err);
  }
}

```

```

};

export const createTextDocumentController = async (req: Request, res: Response,
next: NextFunction) => {
  try {
    const authReq = req as AuthRequest;

    if (!authReq.user || !authReq.user.id) {
      throw new AppError("User not authenticated", 401);
    }

    const userId: number = authReq.user.id;
    const { content } = req.body;

    const result = await createTextDoc(userId, content);

    res.status(201).json({
      success: true,
      data: result,
    });
  } catch (err) {
    next(err);
  }
};

export const getDocuments = async (req: any, res: Response, next: NextFunction) => {
  try {
    const userId = req.user.id;

    const docs = await getUserDocuments(userId);

    res.json({
      success: true,
      data: docs,
    });
  } catch (err) {
    next(err);
  }
};

export const getDocumentByIdHandler = async (req: any, res: Response, next: NextFunction) => {
  try {
    const documentId = Number(req.params.id);
    const doc = await getDocumentById(documentId);
  }
};

```

```

    if (!doc) {
      throw new AppError("Document not found", 404);
    }
    if (doc.userId !== req.user?.id) {
      throw new AppError("Not authorized to access this document", 403);
    }

    // Text-pasted documents already have their content in the DB; file
    // uploads only store a path, so extract on demand for the viewer –
    // same pattern analysis.controller.ts uses when re-reading a document.
    const content = doc.filePath ? await extractTextFromPDF(doc.filePath) : doc.content ?? "";

    res.json({ success: true, data: { document: doc, content } });
  } catch (err) {
    next(err);
  }
};

export const updateDocumentHandler = async (req: any, res: Response, next: NextFunction) => {
  try {
    const document = await updateDocument(Number(req.params.id), req.user?.id, req.body ?? {});
    res.json({ success: true, data: { document } });
  } catch (err) {
    next(err);
  }
};

export const deleteDocumentHandler = async (req: any, res: Response, next: NextFunction) => {
  try {
    await deleteDocument(Number(req.params.id), req.user?.id);
    res.json({ success: true, message: "Document deleted" });
  } catch (err) {
    next(err);
  }
};

export const getDocumentFileHandler = async (req: any, res: Response, next: NextFunction) => {
  try {
    const documentId = Number(req.params.id);
    const doc = await getDocumentById(documentId);

    if (!doc) {
      throw new AppError("Document not found", 404);
    }
  }
};

```

```

    if (doc.userId !== req.user?.id) {
      throw new AppError("Not authorized to access this document", 403);
    }
    if (!doc.filePath || !fs.existsSync(doc.filePath)) {
      throw new AppError("No file available for this document", 404);
    }

    const ext = path.extname(doc.filePath).toLowerCase();
    res.setHeader("Content-Type", CONTENT_TYPE_BY_EXT[ext] || "application/octet-stream");
    res.setHeader("Content-Disposition", "inline");
    fs.createReadStream(doc.filePath).pipe(res);
  } catch (err) {
    next(err);
  }
};

```

server/src/modules/document/document.controller.ts

```

import { prisma } from "../../config/db";

export const createDocument = async (
  userId: number,
  filePath: string
) => {
  return prisma.document.create({
    data: { userId, filePath },
  });
};

export const createTextDocument = async (
  userId: number,
  content: string
) => {
  return prisma.document.create({
    data: { userId, content, status: "pending" },
  });
};

export const getDocumentById = async (documentId: number) => {
  return prisma.document.findUnique({ where: { id: documentId } });
};

export const getUserDocuments = async (userId: number) => {
  return prisma.document.findMany({

```

```

    where: { userId },
    orderBy: { createdAt: "desc" },
    // Only the risk summary – the Documents grid shows a verdict chip per
    // card without needing the full analysis payload.
    include: { analysis: { select: { riskLevel: true, riskScore: true } } },
    // The list never displays document text, and pasted documents can be
    // large – the Dashboard and Documents pages both fetch this on every visit.
    omit: { content: true },
  });
};

export const updateDocument = async (
  documentId: number,
  data: { title?: string; isFavorite?: boolean }
) => {
  return prisma.document.update({
    where: { id: documentId },
    data,
    omit: { content: true },
  });
};

// Analysis and ConversationDocument rows go with it (onDelete: Cascade).
export const deleteDocument = async (documentId: number) => {
  return prisma.document.delete({ where: { id: documentId } });
};

export const markDocumentAnalyzed = async (
  documentId: number,
  title: string | null,
  documentType: string
) => {
  // Documents start with no title, so an existing one was set by the user
  // (renamed before the first analysis) – never overwrite it with the AI's.
  const existing = await prisma.document.findUnique({
    where: { id: documentId },
    select: { title: true },
  });

  return prisma.document.update({
    where: { id: documentId },
    data: { status: "completed", title: existing?.title ?? title, documentType },
  });
};

```

```

export const markDocumentFailed = async (documentId: number) => {
  return prisma.document.update({
    where: { id: documentId },
    data: { status: "failed" },
  });
};

// Set right after a background analysis job is enqueued (before the worker
// has necessarily picked it up), so a concurrent request/poll knows work is
// already in flight and won't enqueue a second job for the same document.
export const markDocumentProcessing = async (documentId: number) => {
  return prisma.document.update({
    where: { id: documentId },
    data: { status: "processing" },
  });
};

```

server/src/modules/document/document.repository.ts

```

import * as docRepo from "../document.repository";
import { AppError } from "../../common/errors/AppError";
import { removeUploadedFile } from "../../common/utils/files";

const getOwnedDocument = async (documentId: number, userId: number) => {
  if (!Number.isInteger(documentId)) {
    throw new AppError("Invalid document id", 400);
  }

  const doc = await docRepo.getDocumentById(documentId);
  if (!doc) {
    throw new AppError("Document not found", 404);
  }
  if (doc.userId !== userId) {
    throw new AppError("Not authorized to access this document", 403);
  }
  return doc;
};

// Shape/format of `input` (title length, isFavorite is a real boolean, at
// least one field present) is already guaranteed by document.schema.ts's
// updateDocumentBodySchema at the route – this only does the business
// check (ownership) a database round trip is needed for.
export const updateDocument = async (
  documentId: number,
  userId: number,

```

```

    input: { title?: string; isFavorite?: boolean }
  ) => {
    await getOwnedDocument(documentId, userId);
    return docRepo.updateDocument(documentId, input);
  };

export const deleteDocument = async (documentId: number, userId: number) => {
  const doc = await getOwnedDocument(documentId, userId);

  await docRepo.deleteDocument(documentId);
  // After the row is gone, so a failed DB delete never orphans a live document.
  await removeUploadedFile(doc.filePath);
};

export const uploadDocument = async (userId: number, filePath: string) => {
  // Text is deliberately NOT extracted here: it used to call pdf-parse
  // unconditionally (breaking on any non-PDF upload – a real bug this
  // surfaced once image uploads were added), the resulting preview was
  // never actually read by the client, and it duplicated work that
  // analysis.textExtraction.ts already does properly (extension-aware,
  // with OCR fallback) when the background analysis job runs.
  const document = await docRepo.createDocument(userId, filePath);
  return { document };
};

// The 50-character minimum is enforced by document.schema.ts's
// createTextBodySchema at the route.
export const createTextDoc = async (userId: number, content: string) => {
  const document = await docRepo.createTextDocument(userId, content);

  return {
    document,
    preview: content.substring(0, 300),
  };
};

```

server/src/modules/document/document.service.ts

I.3 Analysis Module (Complete)

Nine files — this project's signature feature end to end: schema, routing, the controller, data access, the BullMQ queue definition, text extraction (Section 3.6, Procedure 5), the deterministic risk-score lookup (Section 3.6, Procedure 1), the AI service call, and the background worker that ties them together (Figure 2.7).

```
import { z } from "zod";

export const runAnalysisBodySchema = z.object({
  documentId: z.coerce
    .number({ error: "documentId is required" })
    .int("documentId must be a whole number")
    .positive("documentId must be a positive number"),
  // Only the client's first poll of a fresh "Analyze" click sets this –
  // see analysis.controller.ts for why a plain status-check poll must not.
  retry: z.boolean().optional().default(false),
});
```

server/src/modules/analysis/analysis.schema.ts

```
import { Router } from "express";
import { runAnalysis } from "../analysis.controller";
import { authMiddleware } from "../../common/middleware/auth.middleware";
import { analysisPollRateLimiter } from "../../common/middleware/rateLimit.middleware";
import { validate } from "../../common/middleware/validate.middleware";
import { runAnalysisBodySchema } from "../analysis.schema";
import { logger } from "../../config/logger";

const router = Router();

logger.debug("Analysis routes initialized");
router.post(
  "/run",
  authMiddleware,
  analysisPollRateLimiter,
  validate({ body: runAnalysisBodySchema }),
  runAnalysis
);

export default router;
```

server/src/modules/analysis/analysis.route.ts

```

import { Response, NextFunction } from "express";
import { getDocumentById } from "../analysis.repository";
import { markDocumentProcessing } from "../document/document.repository";
import { enqueueAnalysis } from "../analysis.queue";
import { AppError } from "../../common/errors/AppError";
import { logger } from "../../config/logger";

// Re-exported for backward compatibility – the derivation itself now lives
// in analysis.riskScore.ts (see that file for why).
export { riskScoreForLevel } from "../analysis.riskScore";

// documentId's shape (present, a positive integer) is guaranteed by
// analysis.schema.ts's runAnalysisBodySchema at the route.
//
// The actual analysis (text extraction, OCR fallback, the OpenAI call) runs
// in the background via BullMQ (see analysis.worker.ts) – this handler's
// job is just to report where things stand and, if nothing is in flight
// yet, kick a job off. It is deliberately safe to call repeatedly: the
// client polls this same endpoint while a document is "processing".
export const runAnalysis = async (req: any, res: Response, next: NextFunction) => {
  try {
    const { documentId, retry } = req.body;
    const doc = await getDocumentById(documentId);

    if (!doc) {
      throw new AppError("Document not found", 404);
    }

    if (doc.userId !== req.user?.id) {
      throw new AppError("Not authorized to access this document", 403);
    }

    // Already analyzed – return the stored result instead of spending
    // another OpenAI call (or another background job) on a document
    // we've already processed.
    if (doc.analysis) {
      return res.json({
        success: true,
        data: { analysis: doc.analysis, cached: true },
      });
    }

    // A job is already queued/running for this document – just report
    // that, rather than enqueueing a second one (the queue itself also
    // guards against this via a deterministic jobId, but checking here

```

```

    // avoids the extra round trip).
    if (doc.status === "processing") {
        return res.status(202).json({ success: true, data: { status: "queued" } });
    }

    // A previous run failed for good (all retries exhausted). Only
    // start a new attempt when the client explicitly asks for a retry
    // (the first call of a fresh "analyze" click) – not on every poll,
    // or a document that can never succeed would be silently
    // re-attempted forever instead of surfacing the failure.
    if (doc.status === "failed" && !retry) {
        return res.json({
            success: true,
            data: { failed: true, message: "Analysis failed. Please try again." },
        });
    }

    await markDocumentProcessing(documentId);
    await enqueueAnalysis(documentId);

    return res.status(202).json({ success: true, data: { status: "queued" } });
} catch (err) {
    logger.error({ err }, "Error occurred while running analysis");
    next(err);
}
};

```

server/src/modules/analysis/analysis.controller.ts

```

import { prisma } from "../../config/db";

export interface AnalysisData {
    summary: string;
    riskLevel: string;
    riskScore: number;
    clauses: string[];
    riskItems: any;
}

export const createAnalysis = async (documentId: number, data: AnalysisData) => {
    // A document can only have one analysis (documentId is unique), so
    // re-running analysis on an already-analyzed document must overwrite
    // the existing row instead of colliding with the unique constraint.
    return prisma.analysis.upsert({

```

```

        where: { documentId },
        update: data,
        create: { documentId, ...data },
    });
};

export const getDocumentById = async (documentId: number) => {
    return prisma.document.findUnique({
        where: { id: documentId },
        include: { analysis: true },
    });
};

```

server/src/modules/analysis/analysis.repository.ts

```

import { Queue } from "bullmq";
import { redisConnection } from "../../config/redis";

export const ANALYSIS_QUEUE_NAME = "document-analysis";

export interface AnalysisJobData {
    documentId: number;
}

export const analysisQueue = new Queue<AnalysisJobData>(ANALYSIS_QUEUE_NAME, {
    connection: redisConnection,
    defaultJobOptions: {
        attempts: 3,
        backoff: { type: "exponential", delay: 5000 },
        // Keep Redis from accumulating job history forever.
        removeOnComplete: { age: 24 * 60 * 60, count: 1000 },
        removeOnFail: { age: 7 * 24 * 60 * 60 },
    },
});

/**
 * Enqueues a background analysis run for a document. A deterministic jobId
 * (one per document) makes this safe to call more than once for the same
 * document while a job is already waiting/active – BullMQ refuses the
 * duplicate rather than running the analysis twice.
 */
export const enqueueAnalysis = async (documentId: number): Promise<void> => {
    await analysisQueue.add("analyze", { documentId }, { jobId: `analysis-${documentId}` });
};

```

```
import path from "path";
import { extractTextFromPDF } from "../../common/utils/pdf";
import { ocrPdf, ocrImage, IMAGE_EXTENSIONS } from "../../common/utils/ocr";

// Same threshold createTextDoc (document.service.ts) already uses for
// pasted text – below this there just isn't enough content for a
// meaningful analysis, AI or otherwise.
export const MIN_USABLE_TEXT_LENGTH = 50;

export interface ExtractableDocument {
  filePath: string | null;
  content: string | null;
}

export interface ExtractedText {
  text: string;
  /** True when the text-layer/pasted content wasn't enough and OCR filled the gap. */
  usedOcr: boolean;
}

/**
 * Gets analyzable text out of a document, however it was submitted:
 * pasted text as-is; a normal (born-digital) PDF via its text layer;
 * a scanned PDF or a photographed document via OCR.
 */
export const extractDocumentText = async (doc: ExtractableDocument): Promise<ExtractedText> => {
  if (doc.content) {
    return { text: doc.content, usedOcr: false };
  }

  if (!doc.filePath) {
    return { text: "", usedOcr: false };
  }

  const ext = path.extname(doc.filePath).toLowerCase();

  if (IMAGE_EXTENSIONS.includes(ext)) {
    const text = await ocrImage(doc.filePath);
    return { text, usedOcr: true };
  }

  // PDF path: try the fast, free text-layer extraction first.
```

```

const pdfText = await extractTextFromPDF(doc.filePath);
if (pdfText.trim().length >= MIN_USABLE_TEXT_LENGTH) {
  return { text: pdfText, usedOcr: false };
}

// Little to no text layer – likely a scanned/photographed PDF. Fall back
// to OCR rather than failing outright.
const ocrText = await ocrPdf(doc.filePath);
return { text: ocrText || pdfText, usedOcr: true };
};

```

server/src/modules/analysis/analysis.textExtraction.ts

```

const RISK_SCORE_BY_LEVEL: Record<string, number> = {
  Low: 30,
  Medium: 60,
  High: 90,
};

// Deterministic, server-side risk score derivation – kept in its own tiny,
// dependency-free module (rather than living in analysis.controller.ts)
// so analysis.worker.ts can use it without pulling in the controller's own
// imports (which reach the BullMQ queue / Redis connection) just to get a
// pure lookup function.
export const riskScoreForLevel = (level: string): number => RISK_SCORE_BY_LEVEL[level] ?? 60;

```

server/src/modules/analysis/analysis.riskScore.ts

```

import OpenAI from "openai";
import { env } from "../../config/env";
import { logger } from "../../config/logger";

const client = new OpenAI({
  apiKey: env.OPENAI_API_KEY,
});

const DOCUMENT_TYPES = [
  "Rental Agreement",
  "Employment Contract",
  "NDA",
  "Loan Agreement",
  "Privacy Policy",
  "Terms of Service",
  "Business Contract",
];

```

```

    "Other",
  ];

  const SEVERITIES = ["Low", "Medium", "High"];
  const RISK_CATEGORIES = ["Financial", "Termination", "Liability", "Privacy", "Other"];

  export const analyzeDocument = async (text: string) => {
    //Limit text (it's important to avoid token limits and high costs)
    const trimmedText = text.slice(0, 4000);

    const prompt = `
You are a legal assistant AI that helps non-lawyers understand legal documents.

Analyze the following legal document and return a JSON object with exactly these fields:

{
  "title": "A short descriptive title for this document (e.g. 'Rental Agreement - 123 Main St'),
under 60 characters",
  "documentType": "One of: ${DOCUMENT_TYPES.join(", ")}",
  "summary": "A concise summary in simple, non-legal English",
  "clauses": ["Important clause 1", "Important clause 2"],
  "riskLevel": "Low, Medium, or High - the OVERALL risk level of this document",
  "riskItems": [
    {
      "clause": "The specific clause or term that is risky",
      "severity": "Low, Medium, or High",
      "category": "One of: ${RISK_CATEGORIES.join(", ")}",
      "explanation": "Plain-English explanation of why this is risky and what it means for the
person signing/agreeing"
    }
  ]
}

Return ONLY valid JSON, no markdown formatting, no extra commentary.

Document:
${trimmedText}
`;

    const response = await client.chat.completions.create({
      model: "gpt-4.1-mini",
      messages: [
        {
          role: "user",

```

```

        content: prompt,
      },
    ],
    response_format: { type: "json_object" },
  });

const output = response.choices[0].message.content;

let parsed: any;
try {
  const cleanOutput = output
    ?.replace(/``json/g, "")
    ?.replace(/```/g, "")
    ?.trim();

  parsed = JSON.parse(cleanOutput || "{}");
} catch (error) {
  logger.error({ output }, "Failed to parse AI response");
  throw new Error("Invalid AI response format");
}

// Normalize/validate the AI's output so malformed or drifted responses
// can't crash downstream DB writes or UI rendering.
return {
  title:
    typeof parsed.title === "string" && parsed.title.trim()
      ? parsed.title.trim().slice(0, 120)
      : null,
  documentType: DOCUMENT_TYPES.includes(parsed.documentType)
    ? parsed.documentType
    : "Other",
  summary: typeof parsed.summary === "string" ? parsed.summary : "",
  clauses: Array.isArray(parsed.clauses)
    ? parsed.clauses.filter((c: unknown) => typeof c === "string")
    : [],
  riskLevel: SEVERITIES.includes(parsed.riskLevel) ? parsed.riskLevel : "Medium",
  riskItems: Array.isArray(parsed.riskItems)
    ? parsed.riskItems
      .filter(
        (r: any) =>
          r && typeof r.clause === "string" && typeof r.explanation === "string"
      )
    : []
    .map((r: any) => ({
      clause: r.clause,
      severity: SEVERITIES.includes(r.severity) ? r.severity : "Medium",
      category: RISK_CATEGORIES.includes(r.category) ? r.category : "Other",
    })),

```

```

        explanation: r.explanation,
      : [],
    )))
  };
};

```

server/src/modules/analysis/analysis.service.ts

```

import { Job, UnrecoverableError, Worker } from "bullmq";
import { redisConnection } from "../../config/redis";
import { ANALYSIS_QUEUE_NAME, AnalysisJobData } from "./analysis.queue";
import { getDocumentById, createAnalysis } from "./analysis.repository";
import { markDocumentAnalyzed, markDocumentFailed } from "../document/document.repository";
import { analyzeDocument } from "./analysis.service";
import { riskScoreForLevel } from "./analysis.riskScore";
import { extractDocumentText, MIN_USABLE_TEXT_LENGTH } from "./analysis.textExtraction";
import { logger } from "../../config/logger";

// How many analyses can run at once. Caps both the OpenAI request rate (a
// runaway burst of uploads can't turn into a wall of simultaneous API
// calls/costs) and local CPU load from OCR, which is genuinely heavy.
const ANALYSIS_CONCURRENCY = 3;

/**
 * Does the actual work for one document: extract text (falling back to OCR
 * when needed), send it to the AI, and persist the result. Exported
 * separately from the BullMQ wiring below so it can be unit tested by
 * calling it directly, with no real queue/Redis involved.
 *
 * Returns quietly (does nothing) if the document was deleted or already
 * analyzed by the time this runs – both are "nothing to do", not failures.
 */
export const processAnalysisJob = async (data: AnalysisJobData) => {
  const { documentId } = data;

  const doc = await getDocumentById(documentId);
  if (!doc) {
    logger.info({ documentId }, "Analysis job skipped: document no longer exists");
    return;
  }
  if (doc.analysis) {
    logger.info({ documentId }, "Analysis job skipped: already analyzed");
    return;
  }
}

```

```

let extracted;
try {
  extracted = await extractDocumentText(doc);
} catch (err: any) {
  // A file we genuinely can't read (corrupt PDF, unreadable image) will
  // never succeed on retry – fail it once, not three times.
  throw new UnrecoverableError(`Could not read this document's content: ${err?.message ??
err}`);
}

if (!extracted.text || extracted.text.trim().length < MIN_USABLE_TEXT_LENGTH) {
  throw new UnrecoverableError(
    "No readable text could be found in this document, even after OCR."
  );
}

// From here on, a failure (OpenAI network blip, rate limit, malformed
// response) is genuinely worth retrying – let it propagate as a normal
// Error so BullMQ's backoff applies.
const aiResult = await analyzeDocument(extracted.text);

const saved = await createAnalysis(documentId, {
  summary: aiResult.summary,
  riskLevel: aiResult.riskLevel,
  riskScore: riskScoreForLevel(aiResult.riskLevel),
  clauses: aiResult.clauses,
  riskItems: aiResult.riskItems,
});

await markDocumentAnalyzed(documentId, aiResult.title, aiResult.documentType);

return saved;
};

/**
 * The function actually registered with BullMQ. Wraps processAnalysisJob
 * with the DB bookkeeping for a failure: only flips the document to
 * "failed" once nothing more will be retried (an UnrecoverableError, or the
 * last configured attempt) – not on every transient retry in between,
 * which would otherwise flicker a client's polling view between
 * "processing" and "failed" while a retry is still pending.
 */
export const analysisJobProcessor = async (job: Job<AnalysisJobData>) => {
  try {
    return await processAnalysisJob(job.data);
  }

```

```

    } catch (err) {
      const maxAttempts = job.opts.attempts ?? 1;
      // `job.attemptsMade` counts attempts completed *before* this one (0 on
      // the first attempt, 1 on the second, ...), not the current attempt
      // number – confirmed empirically against a real BullMQ job, since this
      // is easy to get off-by-one on. The attempt this catch block is
      // handling is therefore attemptsMade + 1.
      const isFinalAttempt = err instanceof UnrecoverableError || job.attemptsMade + 1 >=
maxAttempts;

      if (isFinalAttempt) {
        await markDocumentFailed(job.data.documentId).catch((markErr) =>
          logger.error({ err: markErr, documentId: job.data.documentId }, "Could not mark document
as failed")
        );
      }

      throw err;
    }
  };

let worker: Worker<AnalysisJobData> | null = null;

export const startAnalysisWorker = (): Worker<AnalysisJobData> => {
  if (worker) return worker;

  worker = new Worker<AnalysisJobData>(ANALYSIS_QUEUE_NAME, analysisJobProcessor, {
    connection: redisConnection,
    concurrency: ANALYSIS_CONCURRENCY,
  });

  worker.on("failed", (job, err) => {
    logger.error({ err, documentId: job?.data.documentId }, "Analysis job failed");
  });

  logger.info({ concurrency: ANALYSIS_CONCURRENCY }, "Document analysis worker started");
  return worker;
};

export const shutdownAnalysisWorker = async (): Promise<void> => {
  await worker?.close();
  worker = null;
};

```

server/src/modules/analysis/analysis.worker.ts

I.4 RAG Grounding (Complete)

The citation-extraction and verification engine behind Procedure 4 (Section 3.6) and the Legal Assistant/Connect Advocate grounding guarantee (Section 2.5.4).

```
// Grounded legal retrieval for the AI advocate.
//
// Rules this module enforces in code (a prompt alone cannot):
// * only government-sourced text is returned – aggregator rows are excluded;
// * the client's state is injected by the server, never chosen by the model;
// * nothing below the similarity threshold is returned, so "no answer" is a
//   real outcome the advocate has to handle honestly;
// * every provision the advocate states can be checked against what was
//   actually retrieved (verifyCitations).

import { embedText } from "./rag.service";
import {
  findGroundedChunks,
  findSectionChunks,
  GroundedChunk,
  KnowledgeSourceType,
} from "./rag.repository";
import { actsExcludedForEra, LawEra, LEGACY_ACT_ALIASES, STATUTE_SOURCES } from "./rag.sources";

const GOVERNMENT_SOURCE_TYPES: KnowledgeSourceType[] = ["GOV_STATUTE", "GOV_JUDGMENT", "GOV_GAZETTE"];

// 0.30 comes from the golden-set calibration (npm run rag:eval): the weakest
// correct hit scored 0.336, clearly unrelated questions stay below ~0.2. A
// cutoff can't separate "adjacent topic" from "answerable" – that judgement
// stays with the model's prompt and the citation verifier.
export const DEFAULT_RAG = { k: 6, minSimilarity: 0.3 };

export interface LawPassage {
  /** Stable within one search result, e.g. "P1". */
  id: string;
  /** Human label, e.g. "BNSS s.482". */
  citation: string;
  actShort: string | null;
  act: string | null;
  section: string | null;
  text: string;
  sourceUrl: string;
  sourceDomain: string | null;
}
```

```

    jurisdiction: string;
    /** 1 = identical. Exact section lookups are 1. */
    similarity: number;
    exact: boolean;
}

export interface GroundedOptions {
    /** Two-letter state code (e.g. "MH"); central law ("IN") is always included. */
    state?: string | null;
    k?: number;
    minSimilarity?: number;
    sourceTypes?: KnowledgeSourceType[];
    /** Criminal-law regime; defaults to "current" (BNS/BNSS/BSA) so repealed codes are never quoted as current. */
    era?: LawEra;
}

// — Citation parsing (shared by retrieval and the verifier) —————

interface ActAlias {
    alias: string;
    actShort: string;
    ingested: boolean;
}

const ACT_ALIASES: ActAlias[] = [
    ...STATUTE_SOURCES.flatMap((s) => s.aliases.map((alias) => ({ alias, actShort: s.actShort, ingested: true }))),
    ...Object.entries(LEGACY_ACT_ALIASES).flatMap(([actShort, aliases]) =>
        aliases.map((alias) => ({ alias, actShort, ingested: false })))
],
].sort((a, b) => b.alias.length - a.alias.length); // longest first so "BNSS" beats "BNS"

const escapeRe = (s: string) => s.replace(/[*+?^$()|[\]\|\\]/g, "\\$&");
const ACT_RE = ACT_ALIASES.map((a) => escapeRe(a.alias)).join("|");
const NUM = String.raw`d{1,3}[A-Za-z]?(?:\s*\[^\]]{1,6}\)]*`;
const NUM_LIST = String.raw`${NUM}(?:\s*(?:,|and|&|to)\s*${NUM})*`;

// "Section 103 of BNS", "Sections 103 and 105 of the Bharatiya Nyaya Sanhita"
const SECTION_THEN_ACT = new RegExp(

String.raw`\b(?:sections?|secs?\.?|s\.$)\s*${NUM_LIST}\s*(?:,\s*)?(?:of\s+|under\s+)?(?:the\s+)?${ACT_RE})(?![a-z])`,
"gi"
);
// "BNS Section 103", "BNSS, s. 482"
const ACT_THEN_SECTION = new RegExp(

```

```

String.raw`(?<![a-z])(${ACT_RE})\s*,?\s*(?:sections?|secs?\.?|s\.|§)\s*(${NUM_LIST})`,
"gi"
);

const resolveAlias = (text: string): ActAlias | undefined =>
  ACT_ALIASES.find((a) => a.alias === text.toLowerCase().replace(/\s+/g, " ").trim());

// "103(1)" → "103"; "43a" → "43A"
const baseSection = (n: string) => n.replace(/\s*\(.?$/ , "").toUpperCase();

export interface SectionRef {
  actShort: string;
  section: string;
  ingested: boolean;
  /** The text as the speaker wrote it. */
  raw: string;
}

export const extractSectionRefs = (text: string): SectionRef[] => {
  const refs: SectionRef[] = [];
  const seen = new Set<string>();

  const add = (numberList: string, actText: string, raw: string) => {
    const alias = resolveAlias(actText);
    if (!alias) return;
    for (const num of numberList.split(/\s*(?:,|and|&|to)\s*(?=\d)/i)) {
      const section = baseSection(num.trim());
      if (!section) continue;
      const key = `${alias.actShort}|${section}`;
      if (seen.has(key)) continue;
      seen.add(key);
      refs.push({ actShort: alias.actShort, section, ingested: alias.ingested, raw });
    }
  };

  for (const m of text.matchAll(SECTION_THEN_ACT)) add(m[1], m[2], m[0]);
  for (const m of text.matchAll(ACT_THEN_SECTION)) add(m[2], m[1], m[0]);
  return refs;
};

// — Retrieval —————

const toPassage = (c: GroundedChunk, index: number, exact: boolean): LawPassage => ({
  id: `P${index + 1}`,

```

```

citation: c.actShort && c.section ? `${c.actShort} s.${c.section}` : c.sourceTitle,
actShort: c.actShort,
act: c.act,
section: c.section,
text: c.content,
sourceUrl: c.sourceUrl,
sourceDomain: c.sourceDomain,
jurisdiction: c.jurisdiction,
similarity: exact ? 1 : Math.max(0, 1 - c.distance),
exact,
});

export const jurisdictionsFor = (state?: string | null): string[] =>
  state && /^[A-Za-z]{2}$/.test(state) ? ["IN", state.toUpperCase()] : ["IN"];

export const retrieveGrounded = async (query: string, options: GroundedOptions = {}): Promise<LawPassage[]> => {
  const k = options.k ?? DEFAULT_RAG.k;
  const minSimilarity = options.minSimilarity ?? DEFAULT_RAG.minSimilarity;
  const jurisdictions = jurisdictionsFor(options.state);
  const sourceTypes = options.sourceTypes ?? GOVERNMENT_SOURCE_TYPES;

  // 1. Provisions the query names outright are fetched exactly – no chance
  // of a near-miss section from the embedding search.
  const exact: GroundedChunk[] = [];
  for (const ref of extractSectionRefs(query).filter((r) => r.ingested)) {
    exact.push(...(await findSectionChunks(ref.actShort, ref.section, jurisdictions)));
  }

  // 2. Semantic search, filtered by jurisdiction + source type, thresholded.
  const embedding = await embedText(query);
  const semantic = await findGroundedChunks(embedding, {
    jurisdictions,
    sourceTypes,
    k,
    maxDistance: 1 - minSimilarity,
    excludeActs: actsExcludedForEra(options.era ?? "current"),
  });

  const seen = new Set<number>();
  const merged: { chunk: GroundedChunk; exact: boolean }[] = [];
  for (const chunk of exact) if (!seen.has(chunk.id)) (seen.add(chunk.id), merged.push({ chunk, exact: true }));
  for (const chunk of semantic) if (!seen.has(chunk.id)) (seen.add(chunk.id), merged.push({ chunk, exact: false }));
});

return merged.slice(0, Math.max(k, exact.length)).map((m, i) => toPassage(m.chunk, i, m.exact));

```

```

};

// — Tool the live advocate calls —————

export const SEARCH_LAW_TOOL = {
  type: "function" as const,
  name: "search_law",
  description:
    "Search the official Indian statute text (BNS, BNSS, BSA and other ingested government sources) for provisions relevant to the client's situation. " +
    "You MUST call this before stating any section number, punishment, procedure or time limit. Pass the legal issue in plain words, or name a provision (e.g. 'Section 482 BNSS'). " +
    "Only cite what this tool returns.",
  parameters: {
    type: "object",
    properties: {
      query: { type: "string", description: "The legal issue or provision to look up." },
      era: {
        type: "string",
        enum: ["current", "before_2024_07_01"],
        description:
          "Criminal-law regime. Use 'current' when the offence happened on or after 1 July 2024 (BNS/BNSS/BSA). " +
          "Use 'before_2024_07_01' for earlier offences (IPC/CrPC/Evidence Act). Ask the client when it happened if you do not know. Civil laws are unaffected.",
      },
    },
    required: ["query"],
    additionalProperties: false,
  },
};

export interface SearchLawResult {
  found: boolean;
  passages: LawPassage[];
  /** Instruction for the model, not shown to the client. */
  note: string;
}

export const searchLaw = async (
  query: string,
  ctx: { state?: string | null; k?: number; minSimilarity?: number; era?: LawEra }
): Promise<SearchLawResult> => {
  const passages = await retrieveGrounded(query, ctx);

```

```

    if (passages.length === 0) {
      return {
        found: false,
        passages: [],
        note:
          "No sufficiently relevant provision was found in the knowledge base. Do NOT state a section number
or punishment from memory. " +
          "Tell the client you cannot confirm the exact provision, and advise them to consult an enrolled
advocate.",
      };
    }

    return {
      found: true,
      passages,
      note: "Cite only the provisions in `passages`, using their `citation` label. Do not add section numbers that
are not listed here.",
    };
  };

  // — Citation verification —————

  export type CitationStatus = "verified" | "unverified_not_retrieved" | "unverified_act_not_loaded";

  export interface CheckedCitation {
    actShort: string;
    section: string;
    status: CitationStatus;
    raw: string;
  }

  // Compares every "Section N of <Act>" in what the advocate said against the
  // passages actually retrieved in this consultation.
  export const verifyCitations = (
    advocateText: string,
    retrieved: Pick<LawPassage, "actShort" | "section">[]
  ): CheckedCitation[] => {
    const shown = new Set(retrieved.filter((p) => p.actShort && p.section).map((p) =>
`${p.actShort}|${p.section}`));

    return extractSectionRefs(advocateText).map((ref) => ({
      actShort: ref.actShort,
      section: ref.section,
      raw: ref.raw,
      status: !ref.ingested
    }

```

```

      ? "unverified_act_not_loaded"
      : shown.has(`${ref.actShort}|${ref.section}`)
        ? "verified"
        : "unverified_not_retrieved",
    ));
  });
};

```

server/src/modules/rag/rag.grounded.ts

I.5 Shared Middleware and Error Handling (Complete)

The six cross-cutting files every route in Appendix C passes through — `AppError`, the central error handler (Appendix F.1), the auth/admin guards, the rate limiter (Appendix F.5), and the Zod validation middleware walked through in Section 5.8.

```

export class AppError extends Error {
  statusCode: number;

  constructor(message: string, statusCode: number) {
    super(message);
    this.statusCode = statusCode;

    Error.captureStackTrace(this, this.constructor);
  }
}

```

server/src/common/errors/AppError.ts

```

import { Request, Response, NextFunction } from "express";
import { AppError } from "../errors/AppError";
import { logger } from "../../config/logger";

export const errorHandler = (
  err: any,
  req: Request,
  res: Response,
  next: NextFunction
) => {
  if (err instanceof AppError) {
    return res.status(err.statusCode).json({
      success: false,
      message: err.message,
    });
  }
  logger.error(err);
  res.status(500).json({
    success: false,
    message: "Internal Server Error",
  });
};

```

```

    }

    logger.error({ err }, "Unhandled error");

    return res.status(500).json({
      success: false,
      message: "Internal Server Error",
    });
  });
};

```

server/src/common/middleware/error.middleware.ts

```

import { Request, Response, NextFunction } from "express";
import jwt from "jsonwebtoken";
import { env } from "../../config/env";
import { AppError } from "../../errors/AppError";

// Verifies the JWT and attaches its payload as req.user. Deliberately logs
// nothing about the request: the Authorization header and token are secrets.
export const authMiddleware = (req: Request, res: Response, next: NextFunction) => {
  try {
    const authHeader = req.headers.authorization;

    if (!authHeader) {
      throw new AppError("Unauthorized request", 401);
    }

    const token = authHeader.split(" ")[1];
    const decodedToken = jwt.verify(token, env.JWT_SECRET);

    req.user = decodedToken as unknown as Express.User;
    next();
  } catch (err) {
    next(err instanceof AppError ? err : new AppError("Invalid token", 401));
  }
};

```

server/src/common/middleware/auth.middleware.ts

```

import { Response, NextFunction } from "express";
import { prisma } from "../../config/db";
import { AppError } from "../../errors/AppError";

```

```

// Use after authMiddleware. The role is re-read from the database on every
// call – never taken from the JWT, which lives 7 days – so demoting an admin
// takes effect immediately.
export const requireAdmin = async (req: any, res: Response, next: NextFunction) => {
  try {
    const userId = req.user?.id;
    if (!userId) throw new AppError("Unauthorized request", 401);

    const user = await prisma.user.findUnique({ where: { id: userId }, select: { role: true } });
    if (user?.role !== "ADMIN") {
      throw new AppError("Admin access required", 403);
    }

    next();
  } catch (err) {
    next(err);
  }
};

```

server/src/common/middleware/admin.middleware.ts

```

import rateLimit, { ipKeyGenerator } from "express-rate-limit";
import { RedisStore } from "rate-limit-redis";
import { AppError } from "../../errors/AppError";
import { redisConnection } from "../../config/redis";

// Backed by Redis instead of express-rate-limit's default in-memory store,
// so limits survive a `nodemon` restart / deploy instead of silently
// resetting, and would stay correct if this ever ran as more than one
// process (the in-memory store can't be shared across processes at all).
//
// One store per limiter, each with its own key prefix – express-rate-limit
// normally gives each `rateLimit()` call its own isolated MemoryStore
// automatically; sharing a single RedisStore (and therefore one Redis key
// namespace) between aiRateLimiter and standardRateLimiter would merge
// their counts for the same user instead of tracking them separately.
const makeRedisStore = (prefix: string) =>
  new RedisStore({
    // rate-limit-redis's Redis client contract is just "a function that
    // sends raw commands" – ioredis's `.call(command, args[])` overload
    // satisfies it (the spread-args overload trips up TS's overload
    // resolution here, so the array form is used instead).
    sendCommand: (...args: string[]) => redisConnection.call(args[0], args.slice(1)) as any,
  });

```

```

    prefix,
  });

// Keyed by authenticated user id rather than IP – every endpoint this is
// applied to sits behind authMiddleware, and IP-based keying would let one
// user burn through the limit from multiple networks or unfairly throttle
// several users behind the same NAT/proxy. Falls back to IP only if
// somehow unauthenticated; express-rate-limit v8 requires IPv6 addresses
// (e.g. dev's ":::1") to go through ipKeyGenerator so they're normalized
// consistently rather than compared as raw strings.
// Exported (not just used inline) so it can be unit tested directly instead
// of only indirectly through a live rate-limit trip.
export const keyByUser = (req: any) => (req.user?.id ? String(req.user.id) :
ipKeyGenerator(req.ip));

const handler = (req: any, res: any, next: any) => {
  next(new AppError("Too many requests – please slow down and try again shortly.", 429));
};

// Guards the OpenAI/Firecrawl-cost endpoints (chat messages, voice
// transcription+chat, RAG ingestion) – the ones a runaway client or script
// could turn into a real API bill.
export const aiRateLimiter = rateLimit({
  windowMs: 15 * 60 * 1000,
  limit: 30,
  standardHeaders: true,
  legacyHeaders: false,
  keyGenerator: keyByUser,
  handler,
  store: makeRedisStore("ratelimit:ai:"),
});

// Looser guard for cheaper-but-still-real-cost endpoints (TTS synthesis is
// cached after the first call, document upload/analysis already caches).
export const standardRateLimiter = rateLimit({
  windowMs: 15 * 60 * 1000,
  limit: 100,
  standardHeaders: true,
  legacyHeaders: false,
  keyGenerator: keyByUser,
  handler,
  store: makeRedisStore("ratelimit:standard:"),
});

// POST /analysis/run is polled repeatedly by the client while a background

```

```

// analysis job runs (every ~2s, for up to a few minutes on a slow/OCR'd
// document – see client/src/features/document/useDocumentAnalysis.ts) –
// unlike aiRateLimiter's other endpoints, most of those calls don't spend
// any OpenAI cost at all: the actual AI call only happens once per document
// (the queue's deterministic jobId already prevents a duplicate job, and an
// already-analyzed document just returns the cached row). The limit here is
// sized for "polling volume", not "AI spend" – cost control for the AI call
// itself is the worker's concurrency cap (analysis.worker.ts), not this.
export const analysisPollRateLimiter = ratelimit({
  windowMs: 15 * 60 * 1000,
  limit: 300,
  standardHeaders: true,
  legacyHeaders: false,
  keyGenerator: keyByUser,
  handler,
  store: makeRedisStore("ratelimit:analysis-poll:"),
});

```

server/src/common/middleware/rateLimit.middleware.ts

```

import { Request, Response, NextFunction } from "express";
import { ZodError, ZodType } from "zod";
import { AppError } from "../../errors/AppError";

interface ValidationTargets {
  body?: ZodType<any>;
  params?: ZodType<any>;
  query?: ZodType<any>;
}

// One place every route's input shape/format check goes through, so a
// malformed request fails the same way everywhere – a clear 400 naming the
// first problem – instead of each handler hand-rolling its own checks (or,
// for several routes before this layer existed, not checking at all and
// letting bad input fall through to a confusing Prisma error, or worse, an
// unhandled one that surfaces as a raw 500).
//
// On success, req.body/params/query are replaced with the *parsed* value –
// so a schema's .default()s, trims, and z.coerce conversions (e.g. an ":id"
// route param string becoming a real number) are what the controller
// actually sees, not the raw request data.
export const validate = (targets: ValidationTargets) => {
  return (req: Request, res: Response, next: NextFunction) => {
    try {
      if (targets.body) {

```

```

        req.body = targets.body.parse(req.body);
    }
    if (targets.params) {
        req.params = targets.params.parse(req.params) as any;
    }
    if (targets.query) {
        req.query = targets.query.parse(req.query) as any;
    }
    next();
} catch (err) {
    next(toAppError(err));
}
};

export const zodIssueToMessage = (err: ZodError): string => {
    const first = err.issues[0];
    if (!first) return "Invalid request.";
    const field = first.path.length ? `${first.path.join(".")}: ` : "";
    return `${field}${first.message}`;
};

const toAppError = (err: unknown): AppError => {
    if (err instanceof ZodError) {
        return new AppError(zodIssueToMessage(err), 400);
    }
    return err instanceof AppError ? err : new AppError("Invalid request.", 400);
};

```

server/src/common/middleware/validate.middleware.ts